


*Al-Balqa Applied University*

*Faculty of Artificial Intelligence*

*Salt – Jordan*

# Smart Know Your Customer (SKYC) Based Financial Risk and Suspicious Activity Detection System

**By:**

**Najd Raid Hwarat**

**Hala Nedal Amoor**

**Joud Salah Saleh**

**Supervised By:**

**Dr. Nisrean Thalji**

***A graduation project submitted in fulfillment of the requirements for the degree of Bachelor's in Data Science***

**2025 / 2026**

# TABLE OF CONTENTS

Dedication ..... [9]
Acknowledgements ..... [10]
Abstract ..... [11]

**CHAPTER 1: INTRODUCTION**

1.1 Background of the Study ..... [11]
1.2 Problem Statement ..... [13]
1.3 Objectives ..... [14]
1.3.1 General Objective ..... [14]
1.3.2 Specific Objectives ..... [15]
1.4 Significance of the Study ..... [15]
1.5 Scope and Limitations ..... [16]

**CHAPTER 2: LITERATURE REVIEW**

2.1 Know Your Customer (KYC) Overview ..... [18]
2.2 Related Works ..... [18]
2.3 Related Works Summary ..... [21]
2.4 Banking Systems and Industry Applications ..... [23]
2.5 Research Gap ..... [28]

**CHAPTER 3: METHODOLOGY**

3.1 Overview of the Proposed Methodology ..... [30]
3.2 Data Collection and Sources ..... [30]

3.3 Dataset Feature Analysis ..... [34]
3.3.1 Credit Card Fraud Dataset Feature Analysis ..... [34]
3.3.2 PaySim Dataset Feature Analysis ..... [35]
3.3.3 IBM AML Transaction Dataset Feature Analysis ..... [36]
3.3.4 IBM AML Account Dataset Feature Analysis ..... [38]
3.4 Data Preprocessing ..... [39]
3.4.1 Data Sampling Techniques ..... [39]
3.4.2 Handling Missing Values ..... [39]
3.4.3 Class Imbalance Treatment ..... [40]
3.4.4 Outlier Retention Strategy ..... [40]
3.4.5 Feature Encoding and Scaling ..... [40]
3.5 Machine Learning Algorithms Used ..... [41]
3.5.1 Logistic Regression ..... [41]
3.5.2 Random Forest ..... [42]
3.5.3 XGBoost ..... [42]
3.5.4 Isolation Forest ..... [42]
3.5.5 One-Class SVM ..... [43]
3.5.6 Autoencoder ..... [43]
3.6 Justification for Dataset and Algorithm Selection ..... [44]
3.7 System Architecture Style ..... [45]
3.8 Tools and Technologies ..... [45]
3.8.1 Python ..... [45]
3.8.2 Pandas and NumPy ..... [46]
3.8.3 Scikit-learn ..... [46]
3.8.4 XGBoost ..... [46]
3.8.5 TensorFlow / Keras ..... [46]
3.8.6 Jupyter Notebook ..... [46]
3.8.7 Flask / Streamlit ..... [46]
3.8.8 SQLite / PostgreSQL ..... [47]
3.9 System Methodology Flow ..... [47]
3.10 Data Flow Across System Modules ..... [49]

3.11 Machine Learning Model Integration Flow ..... [50]

**References ..... [52]**

**CHAPTER 4: DESIGN AND IMPLEMENTATION**

4.1 System Architecture ..... [59]
4.2 Data Preprocessing Pipeline ..... [61]
4.2.1 IBM AML Dataset Preprocessing ..... [61]
4.2.2 Missing Value Handling ..... [62]
4.2.3 Feature Encoding ..... [64]
4.2.4 Credit Card Dataset Preprocessing ..... [65]
4.2.5 PaySim Dataset Preprocessing ..... [66]
4.3 Feature Engineering ..... [69]
4.3.1 IBM AML Feature Engineering ..... [69]
4.3.2 Feature Selection Rationale ..... [71]
4.3.3 Unsupervised Learning: Exploration and Rationale for Exclusion ..... [72]
4.4 Model Training Pipeline ..... [73]
4.4.1 Scaling ..... [73]
4.4.2 Class Balancing with SMOTE ..... [74]
4.4.3 Model Architectures ..... [76]
4.4.4 Validation Strategy ..... [81]
4.5 Risk Scoring System ..... [83]
4.5.1 Transaction Risk Score (0-100) ..... [83]
4.5.2 Customer Risk Score (0-100) ..... [83]
4.5.3 Risk Level Classification ..... [84]
4.5.4 Explainable AI ..... [84]
4.6 Application Implementation ..... [85]
4.6.1 Technology Stack ..... [85]
4.6.2 Application Structure ..... [85]
4.7 Challenges Encountered and Solutions ..... [91]
4.7.1 Memory and Computational Constraints ..... [91]

4.7.2 Feature Engineering Iteration ..... [91]
4.7.3 Variable Scoping in Jupyter Notebooks ..... [92]
4.7.4 Class Imbalance at Multiple Scales ..... [92]
4.7.5 Unsupervised Model Exploration and Abandonment ..... [92]

**CHAPTER 5: RESULTS AND DISCUSSION**

5.1 Comparison with Related Work ..... [93]
5.1.1 Credit Card Dataset ..... [93]
5.1.2 PaySim Dataset ..... [93]
5.1.3 IBM AML Dataset ..... [94]
5.1.4 Key Differentiators of the SKYC Approach ..... [94]
5.2 IBM AML Dataset Results ..... [95]
5.2.1 Train/Test Split Performance ..... [95]
5.2.2 Overfitting Analysis (Train/Test Split) ..... [95]
5.2.3 K-Fold Cross Validation Results ..... [96]
5.2.4 K-Fold Overfitting Analysis ..... [97]
5.3 Credit Card and PaySim Experimental Comparison ..... [99]
5.3.1 Four-Configuration Experiment Design ..... [99]
5.3.2 Credit Card Results ..... [99]
5.3.3 PaySim Results ..... [100]
5.3.4 Cross-Dataset Comparison ..... [101]
5.4 Feature Importance Analysis ..... [103]
5.5 Limitations ..... [104]

**CHAPTER 6: CONCLUSION AND FUTURE WORK**

6.1 Conclusion ..... [105]
6.2 Future Work ..... [106]

**REFERENCES ..... [107]**

# LIST OF FIGURES

**Chapter 1: Introduction**

Figure 1.1: System Background Overview Diagram ..... [12]

Figure 1.2: Problem Statement Conceptual Diagram ..... [13]

**Chapter 2: Literature Review**

Figure 2.1: Proposed Smart KYC-Based System Architecture ..... [27]

**Chapter 3: Methodology**

Figure 3.1: Overall System Methodology Flow ..... [48]

Figure 3.2: Data Flow Across System Modules ..... [49]

Figure 3.3: Machine Learning Model Integration Flow ..... [50]

**CHAPTER 4: DESIGN AND IMPLEMENTATION**

Figure 4.1: System Architecture Diagram ..... [60]

Figure 4.2: IBM AML Dataset: Class Distribution and Sampling Strategy ..... [62]

Figure 4.3: Credit Card Dataset: PCA Feature Distribution Overview ..... [66]

Figure 4.4: PaySim Dataset: Transaction Type Distribution ..... [66]

Figure 4.5: PaySim Dataset: Fraud vs. Normal Transaction Proportions ..... [67]

Figure 4.6: PaySim Dataset: isFlaggedFraud Rule-Based System Performance ..... [67]

Figure 4.7: PaySim Dataset: Feature Engineering-Fraud Hour ..... [68]

Figure 4.8: PaySim Dataset: Fraud by Destination Type ..... [69]

Figure 4.9: IBM AML Feature Engineering: Engineered Feature Categories ..... [71]

Figure 4.10: Unsupervised Anomaly Detection Models: Exploration and Comparison ..... [72]

Figure 4.11: StandardScaler Preprocessing Pipeline Across All Datasets ..... [73]

Figure 4.12: IBM AML - Random Forest: Confusion Matrix ..... [77]

Figure 4.13: IBM AML - XGBoost: Confusion Matrix ..... [77]

Figure 4.14: IBM AML - Logistic Regression: Confusion Matrix ..... [78]

Figure 4.15: Credit Card Dataset - Confusion Matrices for All Three Models ..... [79]

Figure 4.16: Credit Card Dataset - ROC and Precision-Recall Curves ..... [80]

Figure 4.17: PaySim Dataset - Confusion Matrices for All Three Models ..... [80]
Figure 4.18: PaySim Dataset - ROC and Precision-Recall Curves ..... [81]
Figure 4.19: IBM AML - 5-Fold Cross-Validation: AUC Learning Curves ..... [82]
Figure 4.20: Explainable AI Alert Cards: Severity-Coded Transaction Risk ..... [85]
Figure 4.21: Application Step 1: Dataset Selection Interface ..... [86]
Figure 4.22: Application Step 2: Account Selection Interface ..... [86]
Figure 4.23: Application Step 3 - Risk Assessment: Customer Risk Gauge ..... [87]
Figure 4.24: Application Step 3 - Risk Assessment: Per-Transaction Risk Table ..... [87]
Figure 4.25: Application Step 4 - Analytics Dashboard: Behavioural Features ..... [88]
Figure 4.26: Application Step 4 - Analytics Dashboard: Monthly Analysis ..... [88]
Figure 4.27: Application Step 4 - Analytics Dashboard: Network Features ..... [89]
Figure 4.28: Application Step 4 - Analytics Dashboard: Additional Feature Plots ..... [89]
Figure 4.29: Application Step 4 - Analytics Dashboard: Monthly Tracking ..... [89]
Figure 4.30: Application Step 5 - Model Performance ..... [90]
Figure 4.31: Application Step 5 - Model Performance: Metric Charts ..... [90]
Figure 4.32: Feature Importance Visualisation ..... [90]
Figure 4.33: Application Step 5 - Cross-Dataset Comparison ..... [91]

**Chapter 5: Results and Discussion**

Figure 5.1: IBM AML - Model Comparison: Test Metrics, K-Fold ROC-AUC and Radar ..... [95]
Figure 5.2: IBM AML - Overfitting Analysis: Train vs. Test AUC Gap per Model ..... [96]
Figure 5.3: IBM AML - K-Fold Cross-Validation vs. Train/Test Split ..... [97]
Figure 5.4: IBM AML - K-Fold Overfitting Analysis: Train vs. Validation AUC ..... [99]
Figure 5.5: Credit Card Dataset - Model Comparison ..... [100]
Figure 5.6: PaySim Dataset - Model Comparison ..... [101]
Figure 5.7: Cross-Dataset Model Performance Comparison ..... [101]
Figure 5.8: Model Performance Across 3 Datasets ..... [102]
Figure 5.9: Average Performance per Dataset and per Model ..... [103]
Figure 5.10: PaySim - Feature Importance Rankings ..... [104]

# LIST OF TABLES

**Chapter 1: Introduction**

Table 1.1: Comparison between Traditional KYC and AI-Based KYC ..... [14]

**Chapter 2: Literature Review**

Table 2.1: KYC & Fraud Systems vs. Proposed Risk ..... [23]

Table 2.2: Banking Systems vs. Smart KYC & Fraud ..... [25]

**Chapter 3: Methodology**

Table 3.1: Overview of datasets used in the proposed system ..... [31]

Table 3.2: Dataset sizes before and after sampling ..... [33]

Table 3.3: Dataset roles within system modules ..... [34]

Table 3.4: Credit Card Fraud dataset feature analysis ..... [35]

Table 3.5: PaySim dataset feature analysis ..... [36]

Table 3.6: IBM AML Transaction dataset feature analysis ..... [38]

Table 3.7: IBM AML Account dataset feature analysis ..... [39]

Table 3.8: Summary of preprocessing steps applied to each dataset ..... [41]

Table 3.9: Machine learning algorithms and their applications in the proposed system ..... [44]

**CHAPTER 4: DESIGN AND IMPLEMENTATION**

Table 4.1: AML – Missing Value Handling ..... [64]

Table 4.2: Credit Card Fraud class (class 1) — all models × SMOTE vs No SMOTE ..... [75]

Table 4.3: Paysim Fraud class (class 1) — all models × all configs. ..... [75]

## Dedications

To our families,

We dedicate this work to you our constant source of strength, inspiration, and unwavering support. Your patience through our long nights of study, your encouragement during moments of doubt, and your unconditional love throughout this journey have made all the difference.

To our parents, who believed in us before we believed in ourselves, who sacrificed so much to provide us with the opportunities we have today, and who never stopped reminding us that we were capable of achieving our dreams.

This achievement is not ours alone it is a testament to your love, your faith in us, and the values you have instilled in us. We hope to make you proud, not only through this project, but in all that we do moving forward.

With deepest gratitude and love,

Najd, Hala, and Joud

## Acknowledgements

We would like to express our sincere gratitude to our supervisor, Dr. Nisrean Thalji, for her guidance, patience, and expertise throughout this project. Her valuable feedback and continuous support have been instrumental in shaping our research and analysis.

We also extend our appreciation to the faculty members of our department for providing us with the knowledge and resources necessary to complete this work.

Special thanks to our university for offering an environment conducive to research and innovation.

Finally, we acknowledge the contributions of all researchers and practitioners whose work in financial risk management and fraud detection has laid the groundwork for our study.

## Abstract

The present rising trend in the digitization of financial services has led to increased complexity in the process of verification of customer identity as well as the identification of any fraudulent activity. The traditional method presently used in the verification process of Know Your Customer applies the method of verification by human means, which is not only time-consuming but also prone to human errors. The proposed project aims to create a "Smart KYC-Based Financial Risk and Suspicious Activity Detection System."

The implemented system applies three supervised machine learning algorithms (Logistic Regression, Random Forest and XGBoost) to transaction data from three datasets: IBM AML, Credit Card Fraud and PaySim. Unsupervised methods (Isolation Forest, One-Class SVM and Autoencoder) were explored during design but excluded from the final system. Model probabilities are combined into transaction- and customer-level risk scores (0–100) with rule-based explanations, aiming to reduce operational costs and make banking safer.

In this paper, the background study made about KYC, the issues related to KYC in the financial market, and different models created in previous studies regarding the systems for fraud detection, among others, shall be tackled in order to come up with a holistic approach in the creation of the proposed system. There are different modules that are linked to the structure of the proposed system.

This research finding is one among many that has been accumulating to form a body of knowledge in the area of finance technology. The above solution indicates just how well ML algorithms can be employed in changing the face of banking in order to safeguard finance from dangers that exist in this age.

# CHAPTER 1: INTRODUCTION

## 1.1 Background of the Study

The global finance industry has experienced a dramatic shift within the last two decades because of the incorporation of technology and digitalization [1]. Banks are currently faced with the problem of handling a large number of transactions on a daily basis, against a backdrop of laws and legislation that safeguard against identity theft [2].

Know Your Customer (KYC) is an important regulatory compliance requirement that necessitates banks to authenticate clients' identities and qualifications before engaging with them on financial matters [3]. KYC was implemented to prevent financial fraud, terrorism financing, and money laundering by

ensuring that proper customer identification and financial details were known to financial establishments [2] [3]. Conventional KYC practices consist of authenticating documents and customer background analysis and have functioned somewhat but not adequately for contemporary financial fraud and electronic financial transactions [2] [4]

The latest developments in machine learning and data analysis bring about the possibility of optimizing KYC processes [5]. It is possible for algorithms to analyze customer data in order to look for patterns that suggest risks or suspicious activities [6]. Financial institutions are thus able to effectively identify fraud, make assessments in relation to risks, and flash alerts for suspicious activities [7] [8].

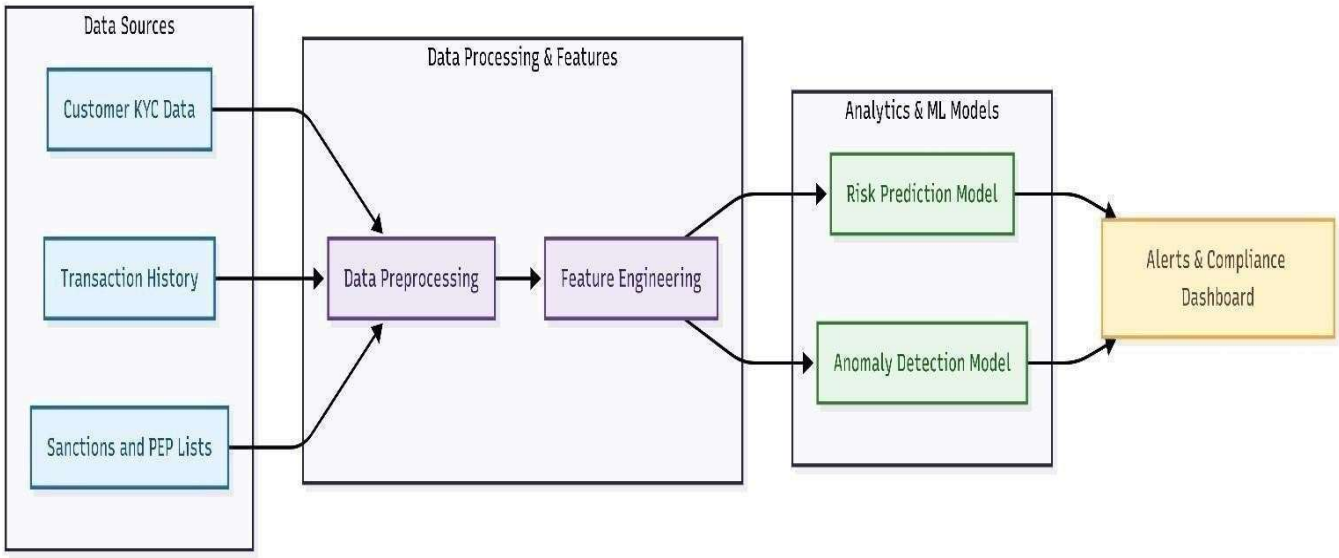



Figure 1.1: System Background Overview Diagram

## 1.2 Problem Statement

Even with the adoption of conventional KYC processes, contemporary financial institutions are faced with an ever-growing challenge in combating fraud [4]. The number of online transactions increases with each day, while fraudsters become smarter with regard to exploitation [5].

The price of traditional KYC processes is on the rise, and the cost of financial fraud attacks is also rising exponentially [4] [5] . There is an immense requirement for more intelligent verification processes that will ensure lower operating expenses and higher efficacy of fraud control [6] .

Machine learning techniques, combined with sophisticated data analysis, will be used to overcome these issues, allowing for automated verification, the detection of unusual patterns, and real-time alerts [6] [7] . For this reason, the proposed project aims to develop a Smart KYC-Based Financial Risk/Suspicious Activity Detection System, focusing on automated customer screening, risk evaluation, as well as intelligent fraud detection for improved financial operation security and efficiency [7] [8] .

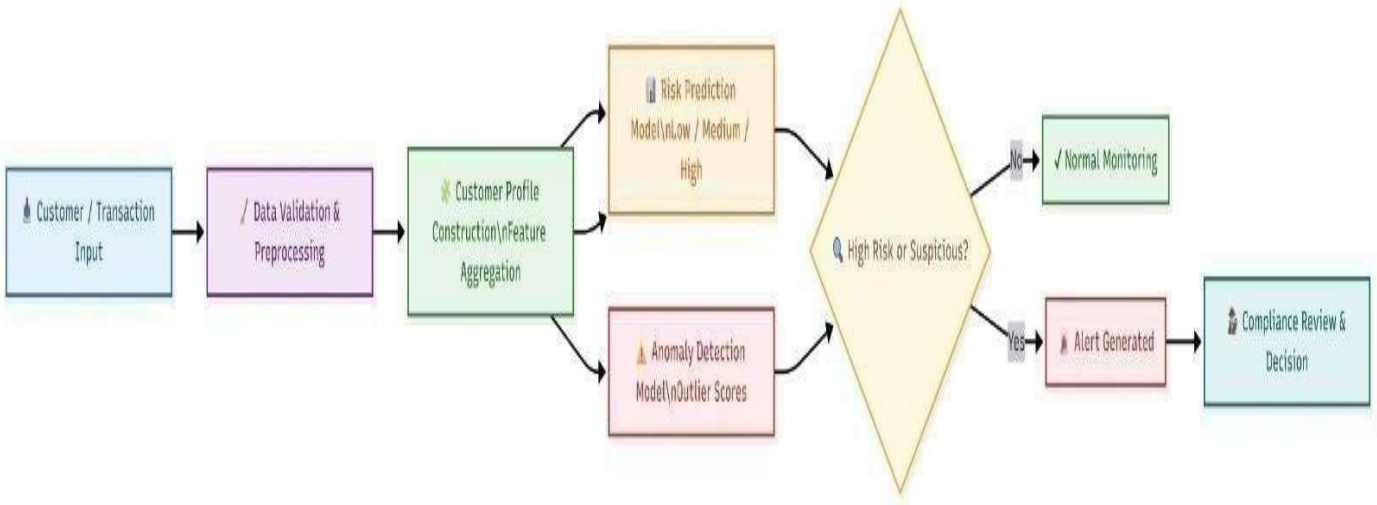



Figure 1.2: Problem Statement Conceptual Diagram

| Aspect | Traditional KYC | AI-Based KYC |
|---|---|---|
| Speed | Days to weeks for customer onboarding | Minutes to hours with automated verification |
| Accuracy | 70-80% (prone to human error) | 90-95% (consistent algorithmic analysis) |
| Scalability | Limited by human resources | Easily scalable to millions of customers |
| Strengths | Human judgment for complex cases; established processes; regulatory familiarity | Rapid processing; pattern recognition; continuous learning; reduced operational costs; 24/7 operation |
| Weaknesses | Labor-intensive; slow; expensive; inconsistent; limited fraud detection; poor customer experience | Requires quality training data; initial implementation costs; potential algorithmic bias; regulatory uncertainty |

Table 1.1 : Comparison between Traditional KYC and AI-Based KYC

## 1.3 Objectives

### 1.3.1 General Objective

This project aims to design and implement a Smart KYC-Based Financial Risk and Suspicious Activity Detection System using machine learning algorithms for the automation of customer verification, financial risk assessment, and suspicious activity identification in real time, which will increase the level of security, efficiency, and compliance of banking operations [6] [7] .

### 1.3.2 Specific Objectives

1. Analysis and understanding the present-day limitations and challenges associated with the conventional KYC procedure followed in financial institutions [2] [3] .
2. To develop an automated KYC verification process with minimal human intervention and turnaround time, while ensuring maximal accuracy in the verification of the customer's identity [1] [4] .
3. Development of a risk assessment model capable of assessing customer financial information and assigning risk scores based on criteria such as transactions, account behavior, and history [5] .
4. Implementation of machine learning techniques to identify possible fraud or suspicious behaviors through transaction analysis by pinpointing unusual patterns within such transactions [6] [8] .
5. Develop an intelligent alert generation tool that alerts the compliance team about high risk customers, as well as suspicious transactions that need urgent investigation [4] [5] .
6. Comparing the performance of machine learning techniques, namely Random Forest, XGBoost and Logistic Regression (with Isolation Forest, One-Class SVM and Autoencoders explored as unsupervised alternatives), in the context of financial risk analysis [6].
7. To make sure that the new system adheres to Anti-Money Laundering regulations concerning client due diligence [2] [3] .
8. Proving the applicability and utility of the system by performing comprehensive testing using realistic financial data scenarios [5] [6] .

## 1.4 Significance of the Study

The significance of Know Your Customer (KYC) systems in financial institutions and society has been highlighted [2] [3] . The adoption of an intelligent system for risk detection on the basis of KYC helps combat problems that emerge during traditional verification processes and boosts financial security [2] [5] .

KYC processes in **financial organizations** also experience cost savings and cycle time reduction through the application of an advanced KYC tool [1] [2] . The use of an intelligent KYC tool increases accuracy in identifying fraudulent activities and optimizes the utilization of human resources in decision-making [5] . The technology also increases adherence to Anti-Money Laundering rules [2] [3]

For **customers** also, smart KYC solutions enable faster and more convenient customer onboarding processes without compromising high levels of security [4][5]. This is an approach towards more customer satisfaction in the context of digital finance [4] [5] .

**Academia-wise**, this research makes an equal contribution to financial technology based on the assessment of machine learning algorithms for KYC and financial risk identification [6] [7] [8] . The results of this research are very helpful for future studies related to intelligent KYC system development [6] [7] .

The above discussion highlights that this study proves the essential use of intelligent KYC systems in fighting financial crimes and ensuring safe, scalable, and efficient financial services [2] [5] [6] .

## 1.5 Scope and Limitations

### Scope

This project involves designing, developing, and evaluating a 'Smart KYC-Based Financial Risk and Suspicious Activity Detection System' specially designed for banks and financial institutions. The main areas that will be covered in this project are as follows:

- Design of an automated KYC-oriented risk assessment workflow in line with KYC guidelines [1][2].
- Development and application of supervised machine learning algorithms (Random Forest, XGBoost and Logistic Regression), with Isolation Forest, One-Class SVM and Autoencoder explored as unsupervised alternatives [6] [8].
- Analysis of customer transaction data to detect signs of suspicious activity or possible fraud [4] [5] .
- Development of a risk scoring model that rates customers according to various factors, taking different levels of risk [5] .
- Development of an alert generation system to notify compliance teams about high-risk scenarios that demand an investigation [4] [5] .
- Analysis of the performance of the systems against appropriate variables to measure accuracy, precision, recall, and effectiveness [6] .
- Compliance with AML regulations when designing the system [2] [3] .

### Limitations

Although the extensive approach has been conducted in this study, it has some limitations, which include:

- The effectiveness of the system is dependent on the quality and completeness of data used in creating models for machine learning. Biased or incomplete data sources can potentially influence data predictions, along with data generality [1] .

- The study mainly targets transactional information and patterns related to customer behavior. Variables that could affect risk analysis, including geopolitical issues and macroeconomic trends, are not fully accounted for by the model [2] .

- To successfully implement the system, integration with the current banking structure would be necessary, which may be differing from one bank to another. The project gives a conceptual background but is not focused on all the technical integration difficulties [3] .

- The nature of financial crime being adversarial means that the effectiveness of the fraud detection system will always be undermined by the adversaries who are constantly trying to change their approaches in order to evade fraud detection [4] .

- The effectiveness of the system needs to continually be updated [4] . Additionally, privacy and data protection issues limit what can be gathered and analyzed for customers. It is important for the system to accommodate security needs without compromising privacy and data requirements [5] .

- It is an academic research project with limited exposure to real-world banking data. Although the performance of the system using synthetic and public data sources is demonstrated, it can perform differently in the real world of finance and banking, as claimed in [6] .

- The project scope does not include the legal and change management for the implementation of such a system within the banking environment. This is because it involves training and awareness creation among various actors within this sector [7] .

- The mechanism of alert creation in the system has the potential to give false positives, which have to be validated by human analysis in order to detect suspicious behavior. The trade-off between sensitivity and specificity needs to be appropriately set so as not to overwhelm the compliance team with alerts [8] .

# CHAPTER 2: LITERATURE REVIEW

## 2.1 Know Your Customer (KYC) Overview

Know Your Customer (KYC) operations currently play a crucial role in contemporary financial regulation and are the first line of defense against money laundering, terrorism financing, as well as other financial crimes. The main objectives of existing KYC operations are currently limited to identification and verification at the opening of accounts, without the ability to monitor potential money laundering practices that develop after account openings .As money crimes develop in sophistication, there has been concern by financial regulators, including the Financial Action Task Force (FATF) and the Basel Committee of Banking Supervision, to improve due diligence and ongoing transaction monitoring [24] , [30] , [31] .

The integration of the recent technological advancements in the fields of machine learning and artificial intelligence has also brought new opportunities with regard to the improvement of financial risk management systems. Such advancements in technological fields allow financial institutions to analyze huge amounts of transactions and detect patterns that may be identified with suspicion and enable the institutions to react accordingly in a real-time environment [9] [16] [31] . Yet, despite the advancements in technologies with regards to the management of risks in the financial institutions, many of the current systems remain in silos with regards to the verification of the KYC and the transaction analysis and risk management systems [9] [16] .

This chapter examines the existing literature surrounding KYC systems, anti-money laundering technology, financial fraud detection methods, and the use of Machine Learning in financial risk management. The chapter has been able to recognize the gaps in the existing methods and serves as the starting point of the new Smart KYC-Based Financial Risk and Suspicious Activity Detection System.

## 2.2 Related Works

The detection of financial crimes and suspicious activities has been studied extensively in recent years, with researchers exploring various machine learning and deep learning approaches. This section reviews key studies that have contributed to the development of intelligent fraud detection, risk assessment, and KYC enhancement systems.

**Kumar and Ravi (2019)** conducted a comprehensive survey on machine learning techniques for fraud detection in banking systems [9] . Their study examined supervised learning methods including decision trees, random forests, and support vector machines, as well as unsupervised techniques such as clustering and anomaly detection. The authors found that ensemble methods, particularly random forests, demonstrated superior performance in identifying fraudulent transactions due to their ability to handle imbalanced datasets. However, their research highlighted that most existing systems operate in isolation, focusing solely on transaction monitoring without integrating customer verification

processes , This means that fraud detection systems typically analyze transactions independently, without integrating customer identity verification or KYC risk profiles, which limits their effectiveness in comprehensive financial crime prevention [9] .

In a related study, **Chawla et al. (2020)** proposed a hybrid approach combining synthetic minority over-sampling technique (SMOTE) with gradient boosting for credit card fraud detection. Their methodology addressed the class imbalance problem common in financial datasets, where fraudulent transactions represent only a small fraction of total transactions. The system achieved a 96.3% detection rate with a false positive rate of 2.1%. Despite these promising results, the authors acknowledged that their approach did not corporate customer behavioral patterns or KYC data, which could potentially improve accuracy [10] ,The study utilized the publicly available European credit card transaction dataset.

Zhang and colleagues (2021) introduced a deep learning architecture for credit card fraud detection using a novel feature engineering methodology called HOBA (homogeneity-oriented behavior analysis). Their system processed transaction sequences using Long Short-Term Memory (LSTM) networks combined with attention mechanisms to identify temporal patterns indicative of fraud. The model achieved state-of-the-art performance on the ULB Credit Card dataset with an Area Under the Curve (AUC) score of 0.989. However, the computational complexity of the deep learning approach made real-time implementation challenging for institutions with high transaction volumes [11].

Focusing specifically on anti-money laundering, Konstantinidis and Gegov (2024) developed an explainable AI framework that addressed the “black box” problem in deep learning models. Their research emphasized the importance of model interpretability for regulatory compliance, as financial institutions must justify their suspicious activity reports to regulators. The authors implemented SHAP (SHapley Additive exPlanations) values to provide transparent explanations for model predictions. While their approach enhanced interpretability, it did not address the integration of KYC verification with transaction monitoring [12]. The study was evaluated on a synthetic dataset generated from real financial logs rather than on proprietary bank data.

Ahmed, Mahmood, and Islam (2016) presented a comprehensive survey of anomaly detection techniques specifically designed for the financial domain. Their review categorized methods into statistical approaches, machine learning techniques, and hybrid systems. The authors highlighted isolation forest and one-class SVM as particularly effective for detecting previously unseen fraud patterns. They noted that most existing systems treat customer onboarding and transaction monitoring as separate processes, creating gaps in comprehensive risk assessment [13].

Lopez-Rojas and Axelsson (2014) introduced BANKSIM, a synthetic bank payment dataset designed specifically for fraud detection research. Their simulation framework generated realistic transaction patterns including various types of fraud scenarios. This work addressed a critical challenge in financial fraud research: the lack of publicly available datasets due to privacy concerns. However, the

synthetic nature of the data meant that models trained on BANKSIM might not capture all complexities of real-world financial crime patterns [14] .

**Pourhabibi and colleagues (2020)** conducted a systematic literature review on graph-based anomaly detection approaches for fraud detection . Their research examined how network analysis techniques could identify suspicious patterns by analyzing relationships between entities such as accounts, merchants, and IP addresses. Graph neural networks showed particular promise in detecting organized fraud rings and money laundering networks. The limitation of these approaches was their computational expense and difficulty in incorporating traditional KYC attributes into graph structures [15] .

**Ryman-Tubb, Krause, and Garn (2018)** performed an industry benchmark study examining how artificial intelligence and machine learning research impacts payment card fraud detection in practice [16] . Their survey of major financial institutions revealed a significant gap between academic research and industry implementation. While academic studies often achieved high accuracy rates on benchmark datasets, real-world systems struggled with concept drift, where fraud patterns evolve over time. To address this challenge, the authors emphasized the need for adaptive learning systems capable of continuously updating models, incorporating online or incremental learning approaches, monitoring concept drift, and integrating human expertise into the fraud detection process.

**Weber and colleagues (2019)** explored scalable graph learning techniques specifically for anti-money laundering applications . Their approach used graph neural networks to analyze transaction flows and identify suspicious patterns at scale. The system could process millions of transactions efficiently using distributed computing frameworks. However, the authors noted that their system operated independently of KYC processes [17] , meaning that customer risk profiles were not incorporated into the graph analysis.

**Baesens, Van Vlasselaer, and Verbeke (2015)** provided a comprehensive framework for fraud analytics that integrated descriptive, predictive, and social network techniques. Their methodology emphasized the importance of combining multiple analytical approaches rather than relying on a single technique. The framework included customer segmentation, predictive modeling, and network analysis. While comprehensive in scope, their approach did not specifically address how to integrate these techniques with regulatory KYC requirements [18] .

**Jurgovsky and colleagues (2018)** developed a sequence classification approach for credit card fraud detection using Recurrent Neural Networks (RNNs) , specifically Long Short-Term Memory (LSTM) networks, to capture temporal patterns in transaction sequences. While the system outperformed traditional transaction-level models, it required extensive historical data for each customer [19] , making it less effective for new accounts due to the cold start problem**.**

**Carcillo et al. (2018**) proposed Scalable Real-time Fraud Finder (SCARFF) , a scalable framework for streaming credit card fraud detection using Apache Spark. Their system addressed the real-time processing requirements of modern payment systems, capable of analyzing thousands of transactions per second. The framework implemented online learning algorithms that adapted to evolving fraud patterns without requiring complete model retraining. Despite its efficiency, SCARFF focused exclusively on transaction analysis and did not incorporate customer verification or KYC data [20] .

**Dal Pozzolo and colleagues (2015)** investigated the challenge of calibrating probability estimates in fraud detection systems with highly imbalanced datasets [21] . Their research demonstrated that undersampling techniques, while improving classification performance, often produced poorly calibrated probability estimates. They proposed calibration methods specifically designed for imbalanced classification problems. This work highlighted an important consideration for practical fraud detection systems, where probability estimates guide decision-making processes.

**Yousefi, Alaghband, and Garibay (2022)** conducted a comprehensive survey on machine learning techniques and user authentication approaches for credit card fraud detection [22] . Their review examined over 150 studies and identified emerging trends including behavioral biometrics, device fingerprinting, and multi-factor authentication. The authors emphasized the importance of combining transaction monitoring with continuous authentication mechanisms. However, they noted that most existing systems did not integrate these authentication mechanisms with comprehensive KYC processes.

**Seiffert and colleagues (2010)** introduced Random Under-Sampling Boosting (RUSBoost) [23] , a hybrid approach combining random under sampling with boosting to address class imbalance in fraud detection. Their algorithm selectively removed majority class examples while iteratively building an ensemble of weak learners. RUSBoost demonstrated superior performance compared to traditional boosting methods on several imbalanced datasets. The technique has since been widely adopted in financial fraud detection systems, though it remains focused on transaction-level analysis rather than comprehensive customer risk assessment.

## 2.3 Related Works Summary

The studies reviewed above represent significant contributions to financial fraud detection and risk assessment. To facilitate comparison, Table 2.1 summarizes the key methodologies, features, and limitations of these work

| Reference | Methodology | Features | Limitations |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Kumar & Ravi (2019) [9] | Random Forest, SVM, Decision Trees | Transaction amount, frequency, merchant category | No integration with KYC data; operates in isolation |
| Chawla et al. (2020) [10] | SMOTE + Gradient Boosting | Transaction features, temporal patterns | Does not incorporate customer behavioral patterns |
| Zhang et al. (2021) [11] | LSTM + Attention Mechanism | Transaction sequences, temporal dependencies | High computational complexity; difficult for real-time use |
| Konstantinidis & Gegov (2024) [12] | Deep Learning + SHAP | Transaction patterns, explainable predictions | No integration with KYC verification processes |

| | | | |
|---|---|---|---|
| Ahmed et al. (2016) [13] | Isolation Forest, One-Class SVM | Anomaly detection, unsupervised learning | Treats KYC and monitoring as separate processes |
| Lopez-Rojas and Axelsson (2014) [14] | Simulation-based approach | Synthetic transaction data, various fraud scenarios | Synthetic data may not capture real-world complexity |
| Pourhabibi et al. (2020) [15] | Graph Neural Networks | Network relationships, entity connections | Computationally expensive; difficult to incorporate KYC |
| Ryman-Tubb et al. (2018) [16] | Industry benchmark study | Multiple ML techniques comparison | Gap between academic research and industry implementation |

| | | | |
|---|---|---|---|
| Weber et al. (2019) [17] | Graph Learning for AML | Transaction flows, network analysis | Operates independently of KYC processes |
| Baesens et al. (2015) [18] | Integrated analytics framework | Descriptive, predictive, network techniques | Does not address integration with regulatory KYC |
| Jurgovsky et al. (2018) [19] | RNN for sequence classification | Transaction sequences, temporal patterns | Requires extensive training data; ineffective for new accounts |
| Carcillo et al. (2018) [20] | Apache Spark streaming | Real-time processing, online learning | only on Focuses transactions; no KYC integration |
| Dal Pozzolo et al. (2015) [21] | Calibration with under-sampling | Probability calibration, imbalanced data handling | Limited to transactionlevel classification |
| Yousefi et al. (2022) [22] | Survey of ML and authentication | Behavioral biometrics, device fingerprinting | Authentication not integrated with comprehensive KYC |
| Seiffert et al. (2010) [23] | RUSBoost ensemble method | Handles class imbalance, boosting | Transaction-level focus; no customer risk assessment |

Table 2.1: KYC & Fraud Systems vs. Proposed Risk

## 2.4 Banking Systems and Industry Applications

While academic research has advanced the theoretical foundations of fraud detection and risk assessment, several major financial institutions have implemented practical systems for KYC and transaction monitoring. This section examines existing banking systems and highlights how our proposed Smart KYC-Based system differs from current industry implementations.

| Bank/ Institution | Country | System Focus | Technologies Used | Limitations | Difference from Our Proposed System |
|---|---|---|---|---|---|
| HSBC | United Kingdom | AML transaction monitoring | Rule-based systems, basic ML classifiers | High false positive rates; separate KYC and monitoring systems | Our system integrates KYC verification with continuous risk assessment using ensemble ML methods |
| JPMorgan Chase | United States | Fraud detection and prevention | Neural networks, pattern recognition | Limited explainability; operates post-transaction | We provide real time risk scoring with explainable predictions at both KYC and transaction stages |
| Barclays | United Kingdom | Customer due diligence | Document verification, manual review | Labor-intensive KYC process; reactive monitoring | Our system automates KYC verification and provides proactive continuous monitoring through ML models |

| | | | | | |
|---|---|---|---|---|---|
| ING Bank | Netherlands | Transaction monitoring | Statistical analysis, threshold based alerts | Static rules cannot adapt to evolving patterns; separate from KYC | We implement adaptive learning algorithms that evolve with new patterns while maintaining KYC transaction integration |
| DBS Bank | Singapore | Digital KYC verification | Biometric authentication, document AI | Strong KYC but limited ongoing risk assessment capabilities | Our system extends beyond initial verification to provide continuous risk assessment across the customer lifecycle |
| Standard Chartered | United Kingdom | Enhanced due diligence | Risk scoring models, watchlist screening | Primarily focuses on high risk customer s; limited automation | We apply comprehensive risk assessment to all customers with automated classification and monitoring |
| Citibank | United States | Compliance and AML | Network analysis, entity resolution | Complex implementation; high operational costs | Our lightweight ensemble approach reduces complexity while maintaining detection accuracy |

Table 2.2: Banking Systems vs. Smart KYC & Fraud

The comparison above reveals that current banking systems typically separate KYC verification from transaction monitoring, creating operational silos. Our proposed Smart KYC-Based Financial Risk and Suspicious Activity Detection System addresses this fragmentation by integrating customer

verification, risk classification, and continuous transaction monitoring within a unified framework. This integration enables more comprehensive risk assessment while reducing false positives through contextual understanding of customer profiles.

Figure 2.1 provides a conceptual overview of the proposed Smart KYC-Based system, highlighting how customer onboarding, transaction monitoring, and risk assessment components are integrated within a unified framework, in contrast to existing industry implementations.

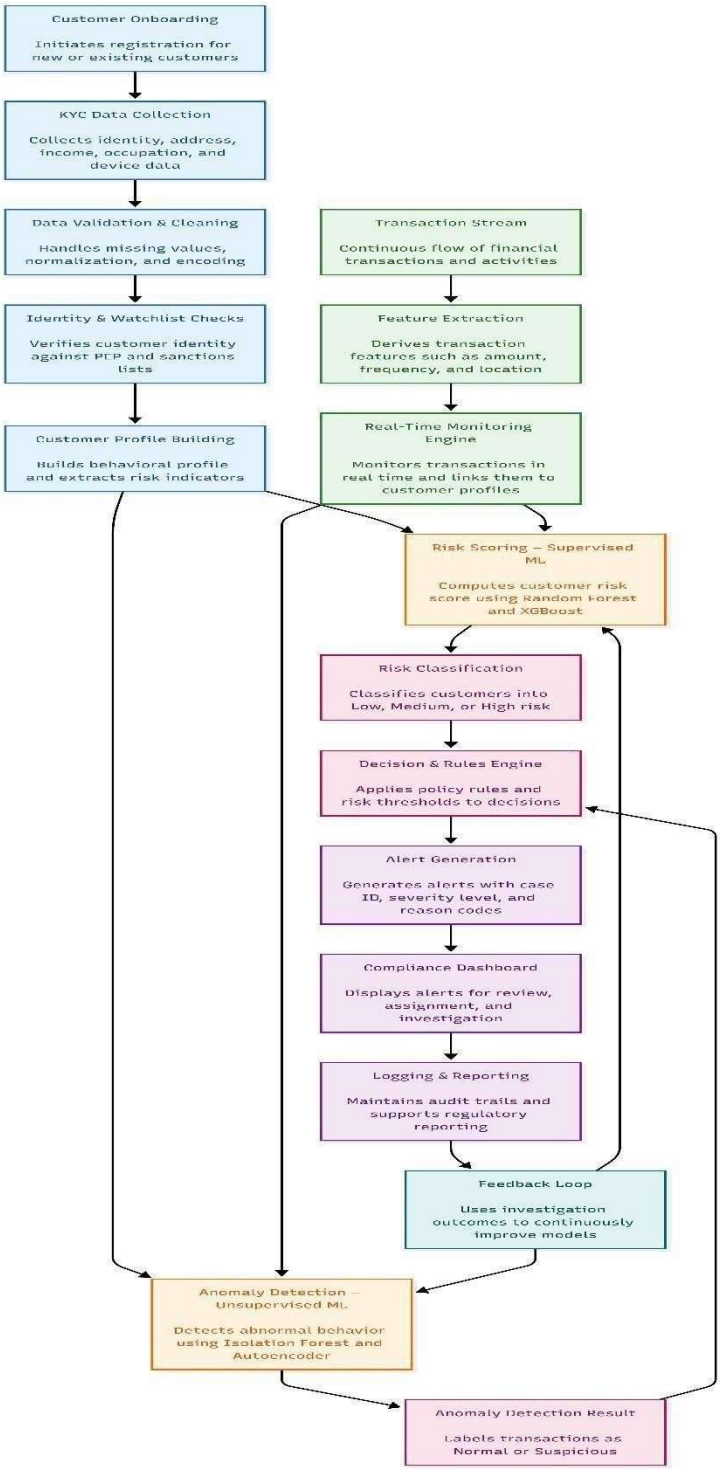



Figure 2.1: Proposed Smart KYC-Based System Architecture

## 2.5 Research Gap

Although considerable progress has been made in the detection of financial fraud and KYC processes, some key areas remain unaddressed in the existing research and literature on this subject. This highlights the need for a comprehensive system like Smart KYC-Based that can fill this gap [31] .

To begin with, most existing systems view the processes of KYC verification and transaction monitoring in an independent and isolated manner. While KYC verification is carried out only once at account opening, transaction monitoring takes place in real time without taking note of the updated risk profiles of customers. The proposed system addresses this by combining transaction-level risk scoring and customer-level risk aggregation in a single framework.

Second, the existing systems have a high percentage of false positives, which can be over 90% in some cases. These false notifications result in major workload challenges for the compliance department, which consequently has to investigate every alert manually. This arises from the fact that rule-based systems are not aware of the context in which the normal activity of the customer takes place. Our proposal aims to reduce this false positive ratio by considering customer risk scores and behaviour when evaluating alerts.

Thirdly, there is little work done using a hybrid machine learning technique that leverages a variety of algorithms optimized for different aspects of financial crime detection. Most work is centered on a single algorithm or on comparing algorithms, although different kinds of financial activity show patterns that could be better discovered using different algorithms. The proposed model combines three supervised classifiers (Logistic Regression, Random Forest and XGBoost) into an ensemble risk score; unsupervised detectors (Isolation Forest, One-Class SVM and Autoencoders) were explored but not retained (Section 4.3.3).

Fourthly, the issue of explainability has been a problem in the application of machine learning in designing fraud detection solutions. As a regulation in the financial industry, banks must be able to give proper explanations regarding suspicious transactions reported. However, the application of deep learning in the design of the model has promoted the model towards the status of a black box since it is not able to give proper explanations. The design of the model emphasizes the use of explanations in AI.

Fifth, there is a lack of integration of the various information sources that are useful in risk identification. Existing systems are often limited to transaction information and are not integrated with KYC and behavioural risk information. Our proposed system integrates customer (account and entity type) information with behavioural transaction patterns and transaction-level information to provide risk information.

Finally, there is a lack of existing research on real-time risk scoring systems capable of working at such a high volume as in modern-day financial institutions. Most of the highly sophisticated machine

learning algorithms for research purposes have been found to computationally intensive for such high transaction volumes. Our risk assessment system has been designed keeping this in mind. By closing the research gaps identified below, our Smart KYC-Based Financial Risk and Suspicious Activity Detection System seeks to achieve a real-world, comprehensive approach designed to improve the efficiency and effectiveness of financial crime prevention.

# Chapter 3: Methodology

## 3.1 Overview of the Proposed Methodology

The proposed system uses supervised machine learning to detect financial fraud, predict transaction risk and identify activity that could point to money laundering. The methodology gathers data from several financial transaction datasets; pre-processes it (sampling, class-imbalance handling, encoding and scaling); engineers behavioural risk indicators; trains Logistic Regression, Random Forest and XGBoost models; and deploys them in a Streamlit application. Unsupervised anomaly detection and a modular API-based architecture belonged to the initial design: the former was explored and excluded (Section 4.3.3) and the latter is future work (Section 6.2). The system processes transaction- and account-level information to produce risk scores and flag suspicious patterns for further investigation.

## 3.2 Data Collection and Sources

Data acquisition comprises the core concept of the proposed system. Five different datasets were chosen depending on their utility for Know Your Customer (KYC), Anti-Money Laundering (AML) and fraud analysis tasks. Each dataset tackles a different issue related to financial risk analysis, ranging from credit card fraud analysis to transaction-level laundering detection and customer profiling.

| Dataset Name | Source | File Name | Purpose |
|---|---|---|---|
| Credit Dataset Card Fraud [34] | Kaggle – European Credit Card Transactions Dataset | creditcard.csv | Supervised fraud detection and model benchmarking |
| PaySim Dataset [35] | Kaggle – PaySim Mobile Money Simulation Dataset | paysim.csv (PaySim log file) | Financial risk prediction and transaction behavior analysis |

| | | | |
|---|---|---|---|
| IBM AML Transaction Dataset (High Intensity) [36] | IBM Anti-Money Laundering Synthetic Dataset | HI-Large_Trans.csv | Supervised laundering detection (primary dataset) |
| IBM AML Transaction Dataset (Low Intensity) [36] | IBM Anti-Money Laundering Synthetic Dataset | LI-Large_Trans.csv | Supervised laundering detection (primary dataset) |
| IBM AML Account Dataset (High Intensity) [37] | IBM AML Synthetic Dataset | HI-Large_accounts.csv | Entity type of each account (entity_type_score) |
| IBM AML Account Dataset (Low Intensity) [37] | IBM AML Synthetic Dataset | LI-Large_accounts.csv | Entity type of each account (entity_type_score) |
| PEPs and Sanctions Dataset[38] | Open Sanctions | targets.simple.csv | Conceptual KYC screening (not used in model training) |

Table 3.1: Overview of datasets used in the proposed system

The **Credit Card Fraud Dataset** from Kaggle comprises anonymized credit card transactions by European cardholders. Features have been transformed using Principal Component Analysis (PCA) to avoid disclosing actual customer information. The dataset contains 284,807 transactions out of which 492 are fraud cases, which means the fraud rate is 0.172%, and hence it is a highly imbalanced dataset. This benchmark dataset for evaluating a supervised fraud detection model is widely used in academic research and industrial practice.

PaySim Dataset emulates mobile money transactions extracted from real financial logs: different transaction types (CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER) with amount, sender and receiver balances, along with fraud labels. The original dataset contains 6,362,620 transactions (8,213 fraudulent); a 30,000-transaction working sample keeping all fraud cases was drawn. PaySim allows the analysis of patterns in customer behavior and risk profiling.

The IBM AML Transaction Dataset consists of synthetic transaction records generated to mimic money laundering networks. It is available in High Intensity (HI) and Low Intensity (LI) variations; the Large files of both were used (355,768,786 transactions in total). All transactions of the 187,294 fraud-linked accounts and of 50,000 normal accounts were filtered (81.1 million rows), and a 30,000-transaction sample (10,000 laundering, 20,000 normal) was drawn for modelling. Every transaction carries a ground-truth label (Is Laundering), so supervised learning was used.

The IBM AML Account files (HI-Large_accounts.csv and LI-Large_accounts.csv) provide, for every account, the bank name, bank ID, account number, entity ID and entity name (4,109,786 unique accounts after merging). They are joined to the transactions by account number, and the entity type derived from the entity name is used as the entity_type_score feature, linking transaction-level risk to specific accounts.

The **PEPs and Sanctions Dataset** by Open Sanctions is a compilation of lists of Politically Exposed Persons(PEPs), sanctioned individuals, and entities out under regulatory scrutiny. Conceptually, it is used for KYC screening in order to illustrate capabilities of compliance but does not feed into training machine learning models. It provides the reference for determining high-risk customers at onboarding and during ongoing monitoring.

| Dataset | Original Size | Sampled Size | Sampling Method |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Credit Card Fraud | 284,807 transactions | 30,000 transactions | All 492 fraud + 29,508 random normal |
| PaySim | 6,362,620 transactions | 30,000 transactions | All 8,213 fraud + 21,787 random normal |
| IBM AML Transactions (HI) | 179,702,229 transactions | 30,000 transactions (HI and LI combined) | Fraud-linked + 50,000 normal accounts, then 10,000 fraud + 20,000 normal |
| IBM AML Transactions (LI) | 176,066,557 transactions | (combined with HI) | (combined with HI) |
| IBM AML Accounts (HI) | 4,109,786 unique accounts (HI + LI) | 18,201 accounts in the sample | Left join on account number |
| IBM AML Accounts (LI) | (merged with HI) | (merged with HI) | (merged with HI) |
| PEPs and Sanctions | Variable | Not sampled | Used as reference only |

Table 3.2: Dataset sizes before and after sampling

| Dataset | System Module | Primary Function |
|---|---|---|
| Credit Card Fraud | Fraud Detection Module | Training supervised classification models |

| | | |
|---|---|---|
| PaySim | Risk Prediction Module | Transaction risk scoring and customer behavior analysis |
| IBM AML Transactions | AML Detection Module | Supervised detection of laundering transactions (primary dataset) |
| IBM AML Accounts | Customer Profiling Module | Entity type of each account, joined to its transactions |
| PEPs and Sanctions | KYC Screening Module | Reference for high-risk customer identification (conceptual; not implemented) |

Table 3.3: Dataset roles within system modules

## 3.3 Dataset Feature Analysis

An examination of the attributes in the dataset was performed in the quest for the most appropriate columns to use for the development of the model. The choice of attributes was informed by expert knowledge of financial fraud patterns.

### 3.3.1 Credit Card Fraud Dataset Features

The Fraud Dataset in Credit Cards contains 31 columns: Time, V1-V28 PCA-transformed features, Amount, and Class fraud labels. The PCA transformation was done by the creators of this dataset so that no sensitive information about the cardholder would be disclosed and still retain valuable statistical relationships between features.

| Column Name | Status | Justification |
|---|---|---|
| V1 to V28 | Used | PCA-transformed features capturing transaction characteristics and patterns |

| | | |
|---|---|---|
| Amount | Used | Transaction amount is a critical feature for fraud detection; high-value transactions exhibit different fraud patterns |
| Class | Used (Target) | Binary label indicating fraudulent (1) or legitimate (0) transactions |
| Time | Excluded | Seconds elapsed since first transaction; temporal patterns are captured by V1-V28; including Time may introduce data leakage |

Table 3.4: Credit Card Fraud dataset feature analysis

### 3.3.2 PaySim Dataset Features

This dataset contains the description of transaction type, monetary amount, sender and receiver information, balance changes, and fraud indications. Feature selection consisted of transaction specific attributes, excluding redundant identifiers.

| Column Name | Status | justification |
|---|---|---|
| type | Used | Transaction type (CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER) influences fraud risk patterns |
| amount | Used | Transaction amount is a primary risk indicator; unusual amounts may signal fraud |
| oldbalanceOrg | Used | Sender's balance before transaction enables detection of inconsistent balance patterns |

| | | |
|---|---|---|
| newbalanceOrig | Used | Sender's balance after transaction; balance discrepancies indicate potential fraud |
| oldbalanceDest | Used | Receiver's balance before transaction supports identification of mule accounts |
| newbalanceDest | Used | Receiver's balance after transaction; unusual balance accumulation patterns suggest money laundering |
| isFraud | Used (Target) | Binary fraud label for supervised learning |
| isFlaggedFraud | Excluded | Legacy rule-based flag; caught only 16 of 8,213 frauds and would leak rule knowledge (Section 4.2.5) |
| step | Excluded | Not used directly; only used to derive the hour of day and the is_night feature |
| nameOrig | Excluded | Sender identifier; anonymized strings do not provide meaningful features without aggregation |
| nameDest | Excluded | Receiver identifier; excluded for the same reason as nameOrig |

Table 3.5: PaySim dataset feature analysis

### 3.3.3 IBM AML Transaction Dataset Features

The IBM AML Transaction Dataset includes features such as timestamps, sender and target entity information, amounts, currency identifiers, and forms of money. The choice of features involved considerations for those with continuous and categorical data that represent money laundering activity.

| Column Name | Status | Justification |
|---|---|---|
| Timestamp | Used | Day of month and hour are extracted from it as model features |
| From Bank | Excluded | Not a model input; used with To Bank for the is_cross_bank dashboard indicator |
| To Bank | Excluded | Not a model input; used with From Bank for the is_cross_bank dashboard indicator |
| Amount Received | Used | Received amount in transaction; structuring behavior involves repeated similar amounts |
| Receiving Currency | Used | Currency of received amount; currency conversion patterns may obscure fund sources |
| Amount Paid | Used | Paid amount in transaction; discrepancies between paid and received amounts suggest intermediary fees or conversion losses |
| Payment Currency | Used | Currency of paid amount; multiple currency transactions complicate audit trails |
| Payment Format | Used | Method of payment (e.g., wire, ACH); certain formats are preferred for laundering due to speed or anonymity |
| From Account | Excluded | Account identifier; anonymized strings require aggregation-based features for meaningful analysis |

| | | |
|---|---|---|
| To Account | Excluded | Account identifier; excluded for the same reason as From Account |
| Transaction ID | Excluded | Unique identifier for transactions; does not provide predictive information |

Table 3.6: IBM AML Transaction dataset feature analysis

### 3.3.4 IBM AML Account Dataset Features

The IBM AML Account Dataset is an account-level reference file with five columns: Bank Name, Bank ID, Account Number, Entity ID and Entity Name. It has no demographics, opening dates or transaction statistics (these were engineered, Section 4.3).

| Column Name | Status | Justification |
|---|---|---|
| Account Number | Used | Join key linking each transaction to its account record |
| Bank Name | Used | Bank of the account; used with Payment Format for the dashboard segment thresholds |
| Bank ID | Excluded | Numeric bank identifier; redundant with Bank Name |
| Entity ID | Excluded | Entity identifier; no predictive meaning on its own |
| Entity Name | Used | Entity type (Corporation, Partnership, Sole Proprietorship, Individual, ...) mapped to entity_type_score |

| | | |
|---|---|---|
| Demographics, Opening Date | N/A | Not provided by the IBM AML account files |
| Transaction Count | Derived | Not in the account file; computed from the transactions (Section 4.3) |
| Amount and Volume Statistics | Derived | Not in the account file; computed from the transactions as monthly, daily and yearly aggregates (Section 4.3) |

Table 3.7: IBM AML Account dataset feature analysis

## 3.4 Data Preprocessing

Data preprocessing refers to the processing of raw data into a format that can be used in training a machine learning model. Data preprocessing techniques include data quality issues, class imbalance problems.

### 3.4.1 Random Sampling

Each dataset was reduced to a 30,000-transaction working sample to keep computation feasible. For Credit Card and PaySim, all fraud cases were kept (492 and 8,213) and the remaining rows were drawn at random from normal transactions, giving fraud rates of 1.64% and 27.38%. For IBM AML, the transactions of all fraud-linked accounts and of 50,000 normal accounts were filtered first, and 10,000 laundering and 20,000 normal transactions were then drawn at random (33.33% fraud). The fraud rate is therefore deliberately higher than in the original data.

### 3.4.2 Handling Missing Values

None of the three datasets contains missing values in its original columns. Missing values arise only in the engineered IBM AML features (for example, the first transaction of an account has no previous amount or time), and each was filled with a context-specific value such as the current amount, 0, 1.0 or the population median (Table 4.1). No imputation was needed for Credit Card or PaySim.

### 3.4.3 Addressing Class Imbalance

Fraud detection datasets are characterized by class imbalance: the fraud rate is about 0.172% in the Credit Card Fraud Dataset and about 0.13% in PaySim. Imbalance was handled in two stages. First, the working samples keep all fraud cases (Section 3.4.1). Second, for IBM AML the Synthetic Minority Over-sampling Technique (SMOTE) was applied to the training partition only, to reach a 1:1 ratio, while for Credit Card and PaySim SMOTE was not applied and class_weight='balanced' was used for Logistic Regression and Random Forest (XGBoost was trained without class weighting). This choice followed a four-configuration experiment (Section 5.3).

### 3.4.4 Outlier Retention

Contrary to usual machine learning approaches where outliers are typically eliminated to facilitate generalization, in fraud detection, it is essential to preserve outliers. This is because outliers contain characteristics that represent suspected transactions, and if they are eliminated, it becomes difficult to identify fraud from legitimate transactions. Also, for analytical purposes, detection of outliers is essential, and their elimination is not required during preprocessing.

### 3.4.5 Feature Encoding and Scaling

The categorical variables in the PaySim and IBM AML datasets needed to be encoded in order to be compatible with machine learning algorithms. One-hot encoding needed to be carried out on nominal categorical variables (for instance, transaction type, payment format, Currency Code) to generate indicator variables. Label encoding needed to be carried out on an ordinal categorical variable as appropriate; however, nominal categorical variables dominated in this case.

Feature scaling was performed on numerical features to normalize their ranges for better convergence of algorithms. Standardization (z-score: mean 0, variance 1) was applied to all model inputs with a StandardScaler fitted on the training partition only. Although tree-based models (Random Forest, XGBoost) are scale-invariant, the same scaled inputs were used for all three models to keep the pipeline identical and to give SMOTE meaningful distances (Section 4.4.1).

| Preprocessing Step | Credit Card Fraud | PaySim | IBM AML Transactions | IBM AML Accounts |
|---|---|---|---|---|
| Random Sampling | All fraud + random normal (30k) | All fraud + random normal (30k) | Account filter, then 10k fraud + 20k normal (30k) | Joined to sampled transactions |
| Missing Value Imputation | No missing values | No missing values | Context-specific fill (Table 4.1) | No missing values |
| Class Imbalance Handling | class_weight (LR, RF); no SMOTE | class_weight (LR, RF); no SMOTE | SMOTE on the training split | Not applicable |
| Outlier Treatment | Retained | Retained | Retained | Retained |
| Categorical Encoding | Not applicable (numerical only) | One-hot encoding | One-hot encoding | Ordinal mapping (entity_type_score) |
| Feature Scaling | Standardization (all models) | Standardization (all models) | Standardization (all models) | Scaled with the transactions |

Table 3.8: Summary of preprocessing steps applied to each dataset

## 3.5 Machine Learning Algorithms Used

The final system uses three supervised algorithms on all three labelled datasets (IBM AML, Credit Card Fraud and PaySim). Three unsupervised algorithms were also studied during the design phase; they are described below for completeness but were excluded from the final system (Section 4.3.3).

### 3.5.1 Logistic Regression

Logistic regression is a linear supervised learning algorithm used for probability-based classification problems. This algorithm can be seen as applying the logistic regression function to a linear combination of inputs. This leads to probabilities being predicted for possible classes. This is quite

efficient and is therefore used as a baseline for other algorithms for certain problems, including fraud detection.

In the proposed system, Logistic Regression is used on all three datasets for binary fraud detection. The feature weights are learned by maximizing the likelihood of the observed fraud labels, which is then used to predict the fraud likelihood of new transactions, and an L2 penalty (C = 1.0) is used to reduce overfitting.

### 3.5.2 Random Forest

Random Forest is an ensemble supervised learning algorithm that constructs multiple decision trees during training and outputs the mode (for classification) or mean (for regression) of the predictions from individual trees. Each tree is trained on a bootstrap sample of the training data, and at each split, a random subset of features is considered; this provides diversity among the trees and reduces overfitting.

Random Forest is applied to all three datasets for fraud detection. The algorithm captures non-linear relationships and feature interactions that linear models cannot represent. Feature importance metrics derived from Random Forest provide an insight into which attributes of the transactions contribute most to fraud prediction, thereby supporting model interpretability and refinement of feature engineering.

### 3.5.3 XGBoost

XGBoost is a more advanced ensemble supervised learning algorithm based on the principle of gradient boosting. Unlike Random Forest, which builds trees independently, XGBoost constructs trees sequentially, each tree correcting errors from previously built trees. The algorithm minimizes a differentiable loss function by using gradient descent along with regularization terms for model complexity.

XGBoost has performed well in several fraud detection benchmarks. In this system it is applied to all three datasets. No scale_pos_weight was used: imbalance was handled by the sampling design and, for IBM AML, by SMOTE. XGBoost gave the best ROC-AUC on IBM AML and Credit Card (Chapter 5).

### 3.5.4 Isolation Forest

Isolation Forest is an unsupervised anomaly detection algorithm that is based on the fact that anomalies are rare and their feature values lie far away from the normal observations. The algorithm constructs

an ensemble of isolation trees through recursive partitioning of data by randomly selecting a feature and then a split value. Anomalies are few and easy to isolate, meaning they have shorter path lengths in the trees.

Isolation Forest was explored on the IBM AML transaction data to uncover suspicious patterns without using labels. Each transaction receives an anomaly score from the average path length in the ensemble of trees, and shorter paths indicate more anomalous transactions. It was excluded from the final system because it flagged too many normal transactions (Section 4.3.3).

### 3.5.5 One-Class SVM

One-Class Support Vector Machine (SVM) is an anomaly detection algorithm from unsupervised learning that trains a decision boundary to enclose data points in feature space. This algorithm transforms data into a high-dimensional feature space via a kernel function (for example, radial basis function) and plots a hyperplane that separates normal data from the origin with maximal margin. Any point lying beyond this hyperplane is labeled as an anomaly.

One-Class SVM was explored on the IBM AML transaction data to find transactions that deviate from normal patterns. It works well when normal examples form a dense cluster and anomalies are scattered, and it requires tuning of the kernel and the nu parameter. It was excluded because training was too slow on the 30,000-transaction sample (Section 4.3.3).

### 3.5.6 Autoencoder

An autoencoder is an unsupervised deep learning algorithm that includes an encoder neural network that compresses the data into a latent space of a lower dimension and a decoder neural network that decodes the compressed latent structure into the original high-dimensional structure. This algorithm learns the reconstruction errors for the normal data and marks the normal data that has high reconstruction errors because the anomaly pattern tends to be significantly different from the normal pattern.

An Autoencoder was explored on the IBM AML transaction data to capture non-linear patterns, using fully connected layers with activation functions (such as ReLU or sigmoid) and flagging transactions whose reconstruction error exceeds a threshold. It was excluded because it did not improve on the supervised models despite extensive tuning (Section 4.3.3).

| Algorithm | Type | Dataset(s) | Task | Output |
|---|---|---|---|---|
| Logistic Regression | Supervised | IBM AML, Credit Card, PaySim | Binary fraud classification | Fraud probability (0-1) |
| Random Forest | Supervised | IBM AML, Credit Card, PaySim | Binary fraud classification | Fraud probability and class label |
| XGBoost | Supervised | IBM AML, Credit Card, PaySim | Binary fraud classification | Fraud probability and class label |
| Isolation Forest | Unsupervised | IBM AML (explored, excluded) | Anomaly detection | Anomaly score (-1 to 1) |
| One-Class SVM | Unsupervised | IBM AML (explored, excluded) | Anomaly detection | Anomaly label (normal/anomaly) |
| Autoencoder | Unsupervised | IBM AML (explored, excluded) | Anomaly detection | Reconstruction error |

Table 3.9: Machine learning algorithms and their applications in the proposed system

## 3.6 Justification for Algorithm and Dataset Selection

The choice of datasets and machine learning models is grounded in existing and proven methods in financial fraud analysis and anti-money laundering research from 2019 to 2024. European Credit Card Fraud Dataset is employed in a comparison of supervised fraud detection methods because of its acute class imbalance and anonymous nature. Various analyses between 2019 and 2024 have proved that the accuracy of fraud detection and reasonable false positives can be achieved in these methods at a higher level compared with Logistic Regression using both Random Forest and XGBoost.

PaySim is chosen as a representative of mobile money fraud patterns as it has been found in past studies that a supervised learning model trained on PaySim can successfully distinguish between a genuine and a fraudulent transaction when appropriate feature engineering is used. PaySim is a simulation dataset that can accommodate scalability with recent payment technologies.

The IBM AML datasets (Altman et al., 2023) are synthetic but provide a ground-truth laundering label for every transaction, generated from realistic laundering typologies. This makes them suitable for supervised learning, which was adopted as the core of the system; unsupervised methods such as Isolation Forest, One-Class SVM and Autoencoders were explored as complementary detectors but did not improve the results (Section 4.3.3).

## 3.7 System Architecture Style

The target production architecture is a RESTful API-based design in which separate microservices handle fraud detection, risk evaluation and customer analysis, with containerization (Docker), orchestration (Kubernetes) and OAuth 2.0 for secure access to sensitive financial data. This architecture was not implemented in this project: the delivered prototype is a single Streamlit application (Section 4.1), and a scoring API is listed as future work (Section 6.2).

The API-based design avoids tight coupling seen in ABI architectures, allowing flexibility, languageindependent integration, and easy updates to models or feature pipelines without system recompilation.

## 3.8 Tools and Technologies

The proposed system implementation will make use of an integrated set of open-source tools and technologies that are well suited to data science, machine learning, and web application development.

### 3.8.1 Python

Python will be used as the programming language for this entire system because of the rich data science ecosystem that it offers. The reason why Python was chosen is that it can be easily prototyped in a short span of time due to the fact that it can be interpreted directly.

### 3.8.2 Pandas and NumPy

Pandas is used for manipulation and analysis tasks, involving high-level data structures called Data Frames for managing tabled data. Pandas helps perform load, filter, aggregate, and transform operations on financial transactions needed during preprocessing. Other libraries such as NumPy offer basic mathematical operations needed during feature calculations and analysis processes for generating new business features relevant for machine learning modeling tasks.

### 3.8.3 Scikit-learn

Scikit-learn is the main library used to implement the machine learning algorithms. It offers a common API to train and test a model or make predictions, with modules such as Logistic Regression, Random Forest Classifier, Isolation Forest and One-Class SVM, preprocessing modules such as StandardScaler, and metrics including precision, recall, F1-score and ROC-AUC. SMOTE was taken from the companion imbalanced-learn library. Version 1.9.0 was used to train and save all models.

### 3.8.4 XGBoost

XGBoost is used as a gradient boosting library that comes optimized for higher performance and faster execution speeds compared to the Scikit-learn library's techniques for gradient boosting. This is due to the optimized procedures and the ability to perform parallel processing as well as the ability to handle missing values that XGBoost possesses.

### 3.8.5 TensorFlow or Keras

TensorFlow/Keras was used only during the exploration phase, to build the Autoencoder for anomaly detection. These frameworks support flexible neural network architectures with automatic differentiation and GPU acceleration, and Keras makes encoder and decoder networks easy to define. The Autoencoder is not part of the final system.

### 3.8.6 Jupyter Notebook

Jupyter Notebook is the interactive development environment that allows exploratory analysis of data, model experimentation, and result visualization. Notebooks make it easy to document analysis workflows to make them reproducible. Results of analysis can be visualized using Matplotlib or Seaborn in Jupyter Notebooks to make them interpretable in terms of model performances or feature distributions.

### 3.8.7 Flask or Streamlit

Streamlit was used to build the interactive web application of this project (Section 4.6). Flask is a lightweight framework for designing RESTful APIs that would provide access to the

model inference service; it is the candidate framework for the future scoring API (Section 6.2). Streamlit enables rapid prototyping of data dashboards and applications that anyone can use without front-end development expertise, which suited the demonstration and analytics goals of this project.

### 3.8.8 SQLite or PostgreSQL

No database was used in the prototype: data, engineered features and models are stored as Parquet and joblib files. For a production version, SQLite (simple deployments) or PostgreSQL (transactions, concurrency and scalability, with indexes on account number and time) is planned for storing transactions, customer profiles, model predictions and audit trails.

## 3.9 System Methodology Flow

The overall system design methodology comprises a holistic workflow from data ingestion to risk analysis and report generation. The graphical representation depicted below illustrates the end-to-end workflow of the proposed system end-to-end workflow of the proposed system.

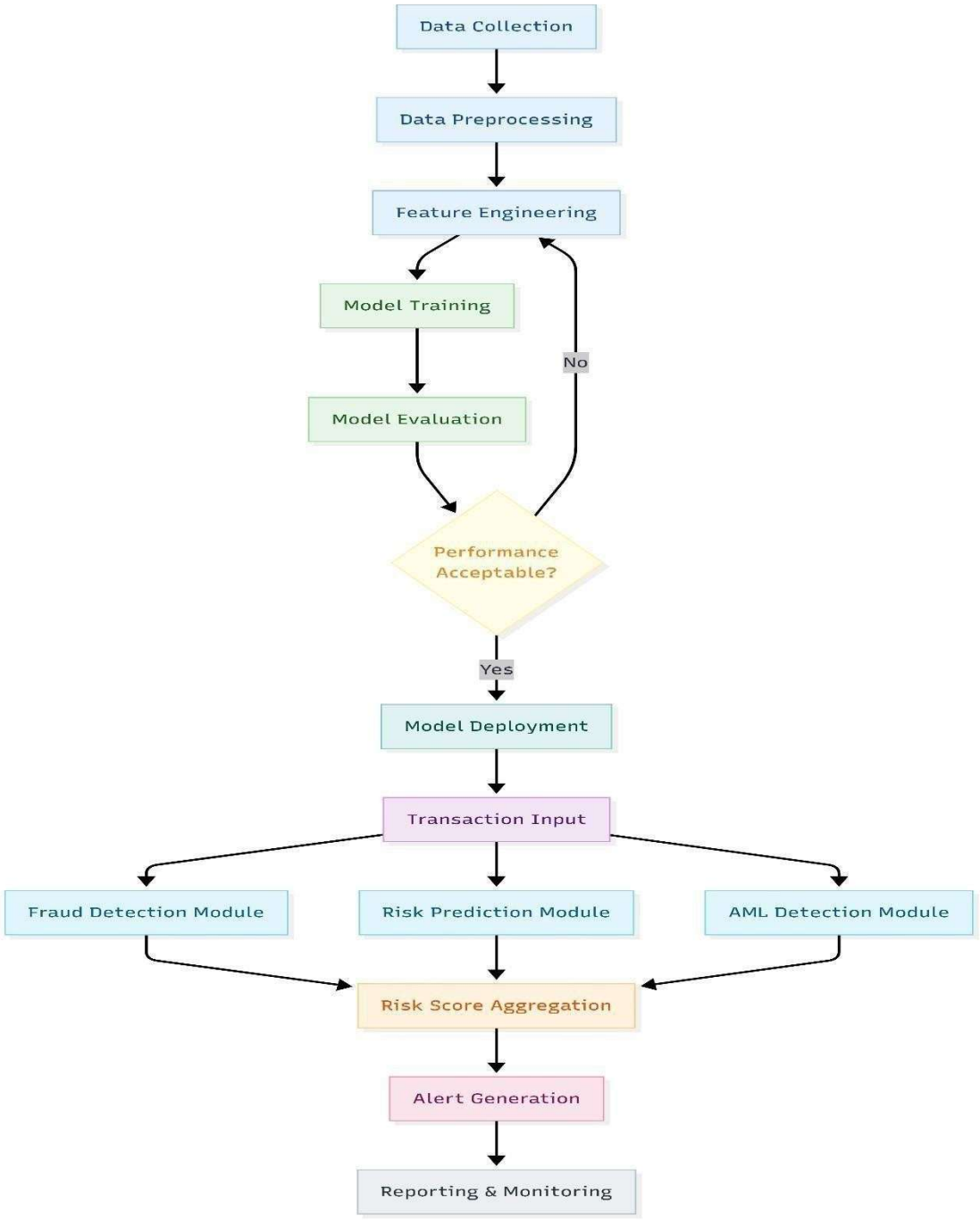



Figure 3.1: Overall System Methodology Flow

It first collects data from multiple sources, followed by preprocessing to handle class imbalance and to encode and scale features. Feature engineering extracts relevant risk indicators from raw transaction attributes. Model training applies supervised algorithms (Logistic Regression, Random Forest and XGBoost) to learn fraud patterns; unsupervised anomaly detection was explored and excluded. Model performance is gauged with appropriate metrics. If the results do not meet the

performance criteria, refinements are carried out iteratively. Models that satisfy the criteria are deployed in the application, where they score the transactions of each dataset. In the proposed production design, fraud detection, risk prediction and AML detection would run in parallel on incoming transactions and consolidate their scores into cumulative risk assessments, with high-risk cases generating alerts for analyst review and reporting dashboards monitoring system performance.

## 3.10 Data Flow Across System Modules

The system architecture includes a number of specialized modules that are involved in the processing of various aspects of financial risk. The below diagram explains the movement of data from one module to the other module.

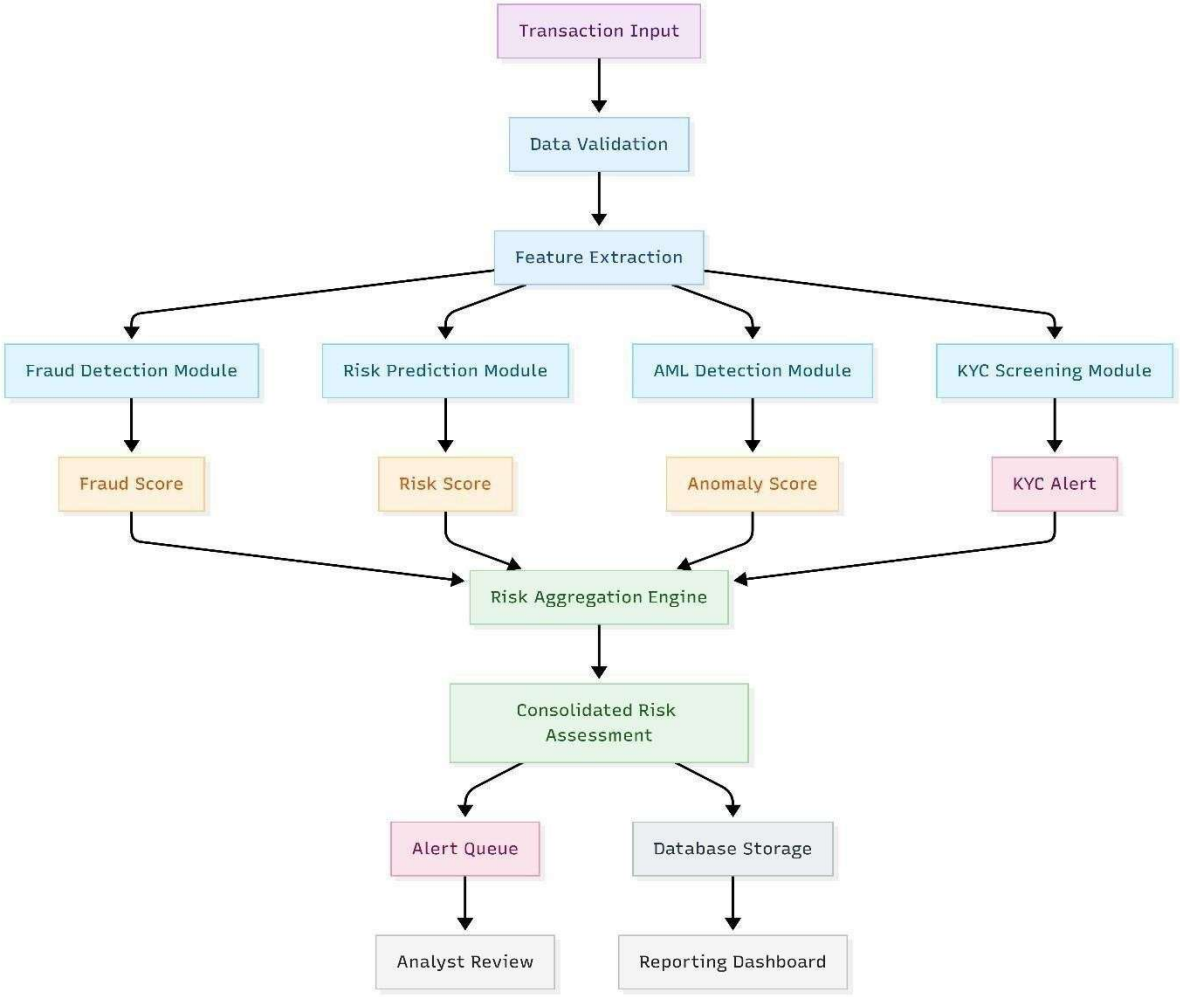



Figure 3.2: Data Flow Across System Modules

In the implemented prototype, each dataset has its own pipeline: the selected transactions are encoded and scaled, scored by the dataset's three supervised models (Logistic Regression, Random Forest and XGBoost), and the probabilities are averaged into a transaction risk score that is aggregated per account into a customer risk score. Rule-based indicators from the engineered features explain each alert, and dashboards show risk distributions and model performance. KYC screening against PEPs and sanctions lists, cross-module score aggregation and persistent storage of assessments in a database are part of the proposed design (Figure 3.2) but were not implemented in this project.

## 3.11 Machine Learning Model Integration Flow

The ingestion of machine learning models into the operational system is a delicate procedure coordinating model loading, feature processing, model inference, and result interpretation. The following diagram describes the model ingestion flow:

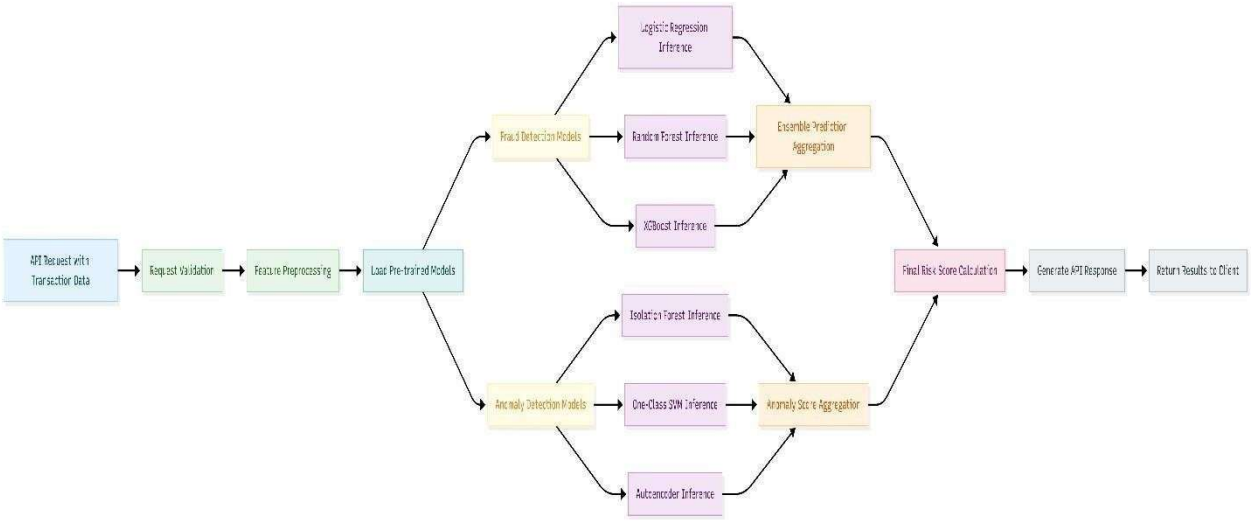



Figure 3.3: Machine Learning Model Integration Flow

In the implemented prototype, models are integrated directly into the Streamlit application rather than behind an API. For the selected dataset, the application loads the saved scaler, the three models and the stored feature list (joblib and JSON files) and applies the same scaling used during development, so that features are identical in both phases. Logistic Regression, Random Forest and XGBoost then score every transaction; anomaly models (Isolation Forest, One-Class SVM, Autoencoder) are not used in the final system. The three fraud model

probabilities are averaged with equal weights into a 0–100 transaction risk score, which is aggregated per account into a customer risk score and shown with rule-based explanations. The API-based flow in Figure 3.3, which returns a risk decision to a client application, is the proposed production design.

## Chapter 1 References


[1] Financial Action Task Force. (2021). Guidance on Digital Identity. FATF, Paris. https://www.fatfgafi.org/publications/fatfrecommendations/documents/guidancedigitalidentity.ht m l

[2] Basel Committee on Banking Supervision. (2020). Sound management of risks related to money laundering and financing of terrorism. Bank for International Settlements.

https://www.bis.org/bcbs/publ/d505.htm

[3] World Bank Group. (2019). Global Standards and Financial Inclusion. World Bank, Washington.

DC. https://www.worldbank.org/en/topic/financialinclusion/brief/kyc

[4] PricewaterhouseCoopers. (2022). Global Economic Crime and Fraud Survey. PwC. https://www.pwc.com/gx/en/forensics/gecc-2022/index.html

[5] McKinsey & Company. (2021). The Future of Banking: A $20 Trillion Opportunity. McKinsey Global Institute.

https://www.mckinsey.com/industries/financialservices/ourinsights/banking-matters

[6] Association of Certified Anti-Money Laundering Specialists. (2020). ACAMS Risk Assessment Guide. ACAMS. https://www.acams.org/en/resources/aml-glossary-of-terms

[7] Deloitte. (2021). Regtech Universe 2021: The Annual Regtech Report. Deloitte Centre for Regulatory Strategy.

https://www2.deloitte.com/uk/en/pages/financialservices/articles/regtech-universe.html

[8] International Monetary Fund. (2020). Financial Technology and Supervision: Risks and

Challenges. IMF Working Papers. https://www.imf.org/en/Publications/WP/Issues/2020/05/15/Fintech-andFinancialServices49831

## Chapter 2 References

[9] Kumar, R. R., & Ravi, V. (2019). A Survey on Machine Learning Techniques for Detection in Banking. International Journal of Computer Applications, 182(44), 1–6. https://doi.org/10.48550/arXiv.1912.02629

[10] Chawla, S., Kaur, H., Singh, K., & Goyal, M. (2020). Hybrid Approach for Credit Card Fraud Detection Using SMOTE and Gradient Boosting. Journal of Financial Crime, 27(3), 819–833.

https://semarakilmu.com.my/journals/index.php/applied_sciences_eng_tech/article/download/3291/23 82/22013

[11] Zhang, X., Han, Y., Xu, W., & Wang, Q. (2021). HOBA: A novel feature engineering methodology for credit card fraud detection with a deep learning architecture. Information Sciences, 557, 302–316.

https://doi.org/10.1016/j.ins.2019.05.023

[12] Konstantinidis, G., & Gegov, A. (2024). Deep neural networks for anti-money laundering using explainable artificial intelligence. University of Portsmouth.

https://pure.port.ac.uk/ws/portalfiles/portal/93893711/GIANNIS_PAPER.pdf

[13] Ahmed, M., Mahmood, A. N., & Islam, M. R. (2016). A survey of anomaly detection techniques in financial domain. Future Generation Computer Systems, 55, 278–288.

https://www.semanticscholar.org/paper/A-survey-of-anomaly-detectiontechniques-in-domainAhmed-Mahmood/5b55d69c4b5f9a371f04944453283ed36ddb90ce

[14] Lopez-Rojas, E. A., & Axelsson, S. (2014). BankSim dataset – synthetic bank payments for fraud research. Dataset (Kaggle): https://www.kaggle.com/datasets/ealaxi/banksim1

[15] Pourhabibi, T., Ong, K.-L., Kam, B. H., & Boo, Y. L. (2020). Fraud detection: A systematic literature review of graph-based anomaly detection approaches. Decision Support Systems, 133, 113303.
https://doi.org/10.1016/j.dss.2020.113303

[16] Ryman-Tubb, N. F., Krause, P., & Garn, W. (2018). How Artificial Intelligence and machine learning research impacts payment card fraud detection: A survey and industry benchmark. Engineering Applications of Artificial Intelligence, 76, 130–157.
https://www.sciencedirect.com/science/article/abs/pii/S0952197618301520

[17] Weber, M., Chen, J., Suzumura, T., et al. (2018/2019). Scalable Graph Learning for Anti-Money Laundering: A First Look. arXiv: https://arxiv.org/abs/1812.00076
PDF: https://jiechenjiechen.github.io/pub/aml.pdf

[18] Baesens, B., Van Vlasselaer, V., & Verbeke, W. (2015). Fraud Analytics Using Descriptive, Predictive, and Social Network Techniques: A Guide to Data Science for Fraud Detection. Wiley.
https://onlinelibrary.wiley.com/doi/book/10.1002/9781119146841

[19] Jurgovsky, J., Granitzer, M., Ziegler, K., et al. (2018). Sequence classification for credit-card fraud detection. Expert Systems with Applications, 100, 234–245. https://doi.org/10.1016/j.eswa.2018.01.037

[20] Carcillo, F., Le Borgne, Y.-A., Caelen, O., et al. (2018). SCARFF: A Scalable Framework for Streaming Credit Card Fraud Detection with Spark. Information Fusion. https://doi.org/10.1016/j.inffus.2017.09.005

[21] Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating Probability with Undersampling for Unbalanced Classification (Fraud Detection context). https://www.researchgate.net/publication/283349138_Calibrating_Probability_with_Undersampling_for_Unbalanced_Classification

[22] Yousefi, N., Alaghband, M., & Garibay, I. (2022). Survey on ML techniques and user authentication approaches for credit card fraud detection. https://arxiv.org/pdf/1912.02629.pdf

[23] Seiffert, C., Khoshgoftaar, T. M., Van Hulse, J., & Napolitano, A. (2010). RUSBoost: A hybrid approach combining random undersampling with boosting for imbalanced classification. https://sci2s.ugr.es/keel/pdf/algorithm/articulo/2010-IEEE%20TSMCpartA-RUSBoost%20A%20Hybrid%20Approach%20to%20Alleviating%20Class%20Imbalance.pdf

[24] Ngai, E. W. T., Hu, Y., Wong, Y. H., Chen, Y., & Sun, X. (2011). The application of data mining techniques in financial fraud detection. Decision Support Systems, 50(3), 559–569. https://doi.org/10.1016/j.dss.2010.08.006

[25] Deng, X., Huang, Z., & Cheng, X. (2019). eCreditRisk: A real-time credit rating system. IEEE Transactions on Knowledge and Data Engineering, 33(6), 2346–2359. https://www.researchgate.net/profile/Ming-Chang-Lee-2/publication/235351722_Enterprise_Credit_Risk_Evaluation_models_A_Review_of_Current_Research_Trends/links/00b7d5344c1b825e31000000/Enterprise-Credit-Risk-Evaluation-models-A-ReviewofCurrentResearch-T

[26] Van Vlasselaer, V., Bravo, C., Caelen, O., et al. (2015). APATE: A novel approach for automated credit card transaction fraud detection. Decision Support Systems, 75, 38–48. https://doi.org/10.1016/j.dss.2015.04.013

[27] Kou, Y., Lu, C.-T., Sirwongwattana, S., & Huang, Y.-P. (2004). Survey of fraud detection techniques. IEEE International Conference on Networking, Sensing and Control. https://doi.org/10.1109/ICNSC.2004.1297040

[28] Carneiro, N., Figueira, G., & Costa, M. (2017). A data mining based system for credit-card fraud detection in e-tail. Decision Support Systems, 95, 91–101. https://doi.org/10.1016/j.dss.2017.01.002

[29] Bhattacharyya, S., Jha, S., Tharakunnel, K., & Westland, J. C. (2011). Data mining for credit card fraud. Decision Support Systems, 50(3), 602–613. https://doi.org/10.1016/j.dss.2010.08.008

[30] Bolton, R. J., & Hand, D. J. (2002). Statistical fraud detection: A review. Statistical Science, 17(3), 235–255. https://doi.org/10.1214/ss/1042727940

[31] West, J., & Bhattacharya, M. (2016). Intelligent financial fraud detection. Computers & Security, 57, 47–66. https://doi.org/10.1016/j.cose.2015.09.005 [32] Randhawa, K., Loo, C. K., Seera, M., Lim, C. P., & Nandi, A. K. (2018). Credit card fraud detection using AdaBoost. IEEE Access, 6, 14277–14284. https://doi.org/10.1109/ACCESS.2018.2806420

[33] Xuan, S., Liu, G., Li, Z., Zheng, L., Wang, S., & Jiang, C. (2018). Random forest for credit card fraud detection. IEEE International Conference on Networking, Sensing and Control (ICNSC). https://doi.org/10.1109/ICNSC.2018.8361343

## Chapter 3 References

[34] Credit Card Fraud Dataset

Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. IEEE Symposium Series on Computational Intelligence. https://doi.org/10.1109/SSCI.2015.33

[35] PaySim Dataset

Lopez-Rojas, E., Elmir, A., & Axelsson, S. (2016). PaySim: A financial mobile money simulator for fraud detection. European Modeling and Simulation Symposium (EMSS).

https://www.researchgate.net/publication/307858636_PaySim_A_Financial_Mobile_Money_Simulator_for_Fraud_Detection

[36] IBM AML Transactions Dataset (High & Low Risk)

Altman, E., et al. (2023). Realistic Synthetic Financial Transactions for Anti-Money Laundering Models. NeurIPS 2023.

https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml

[37] IBM AML Accounts Dataset (High & Low Risk)

Altman, E., et al. (2023). Account files (HI-Large_accounts.csv, LI-Large_accounts.csv) of the IBM AML dataset. https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml

[38] PEPs and Sanctions

Financial Action Task Force (FATF). (2012). International standards on combating money laundering and the financing of terrorism & proliferation. https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Fatf-recommendations.html

# CHAPTER 4: DESIGN AND IMPLEMENTATION

## 4.1 System Architecture

The SKYC Anti-Money Laundering Detection Platform follows a modular, layered architecture designed for maintainability, scalability, and compliance with banking-grade standards. The system is composed of four primary layers: Data Layer, Model Layer, Risk Engine Layer, and Presentation Layer.

The Data Layer handles ingestion, transformation, and storage of transaction data from three distinct financial datasets: IBM AML as the primary anti-money laundering dataset, and Credit Card Fraud Detection and PaySim as validation datasets. Raw CSV files exceeding 16 GB each are converted to Parquet format for efficient columnar storage and memory-optimized loading.

The Model Layer encapsulates nine machine learning models (three per dataset), each trained independently with standardized preprocessing pipelines. Models are serialized using joblib for persistent storage and rapid loading at application startup.

The Risk Engine Layer implements the unified risk scoring system, combining per-model probabilities into ensemble transaction scores and aggregating these into customer-level risk assessments. This layer also generates explainable AI outputs for compliance reporting.

The Presentation Layer is implemented as a Streamlit web application with a step-based navigation system. The interface supports three datasets, providing dataset-specific analytics, risk assessment, and model performance views.

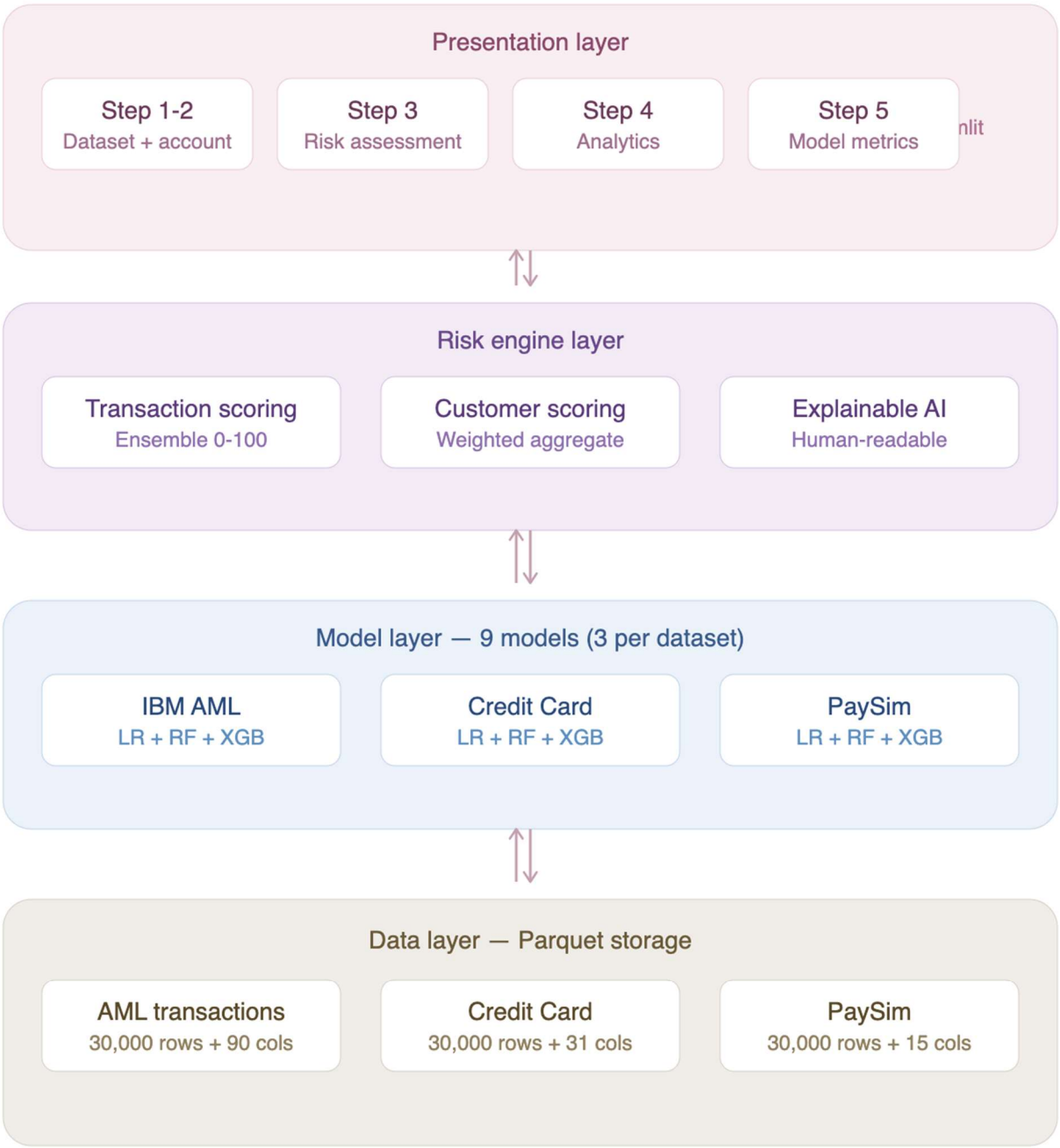



**Figure 4.1: System Architecture Diagram**

*Shows the four-layer architecture: Data, Model, Risk Engine, and Presentation layers with data flow arrows.*

## 4.2 Data Preprocessing Pipeline

### 4.2.1 IBM AML Dataset Preprocessing

The IBM AML dataset was constructed by combining two large-scale transaction files, HI-Large_Trans.csv and LI-Large_Trans.csv. The merged dataset comprised 355,768,786 transactions distributed across 4,029,619 unique accounts. Of these, 187,294 accounts (4.65%) were associated with at least one fraudulent transaction, while the remaining 3,842,325 accounts (95.35%) exhibited exclusively legitimate activity. Given the prohibitive volume of the raw data relative to available memory, both CSV files were converted into Parquet format using chunk-based processing, with batch sizes of five million rows for the HI dataset and one million rows for the LI dataset, yielding 213 Parquet files in total (36 HI + 177 LI). This conversion substantially reduced memory consumption, improved storage efficiency, and accelerated subsequent data loading during preprocessing and feature engineering.

Account selection followed a two-tier strategy designed to preserve the complete fraud signal while retaining a representative sample of legitimate customer behaviour. All 187,294 fraudulent accounts were included without filtering. For normal accounts, eligibility was restricted to those with a transaction history spanning at least two calendar months (MIN_MONTHS ≥ 2), ensuring that behavioural features derived from temporal patterns would be meaningful. From the eligible normal accounts, 50,000 were selected based on the richness of their historical activity.

From this combined account pool of 237,294 accounts, an analytical sample of 30,000 transactions was extracted for model development, comprising 10,000 fraudulent transactions (33.33%) and 20,000 normal transactions (66.67%). Although the original dataset exhibited extreme class imbalance — with only 326,150 fraudulent transactions (0.092%) out of 355,768,786 total transactions — the sampling strategy deliberately increased fraud representation to facilitate effective model training while preserving realistic transaction characteristics. To further address residual class imbalance during training, the Synthetic Minority Oversampling Technique (SMOTE) was applied exclusively within the training partition, ensuring no leakage into the evaluation set.

The transaction records were subsequently enriched by merging with the corresponding account metadata files, HI-Large_accounts.csv and LI-Large_accounts.csv, introducing attributes including BankName, BankID, AccountNumber, EntityID, and EntityName. From the EntityName field, an ordinal feature designated entity_type_score was derived, assigning a value of 3 to Corporation, 2 to Sole Proprietorship, Partnership, and Country, and 1 to Individual and Direct entity types. This encoding captures the ordinal relationship between entity categories while reflecting their differential risk profiles within the AML domain, where corporate and partnership structures are generally associated with greater exposure to layering and integration-stage laundering activity.






**Figure 4.2: IBM AML Dataset: Class Distribution and Sampling Strategy**

4.2.2 Missing Value Handling

Missing values were handled using context-appropriate strategies rather than uniform imputation. Each feature received a fill value justified by its statistical or domain meaning:

| Feature | Cause of Missing | Fill Strategy | Justification |
|---|---|---|---|
| prev_mean_amount<br>prev_max_amount<br>Certain | First transaction per account — expanding().shift(1) always yields NaN | Current Amount Paid | The first observation has no prior history; it becomes its own baseline (Bahnsen et al., 2016). |
| amount_to_mean_ratio<br>amount_to_max_ratio<br>Edge case | Division by zero — prior amount equals exactly 0 after fill | 1.0 (neutral ratio) | A ratio of 1.0 encodes no deviation from the historical baseline, introducing no artificial anomaly signal. |

| | | | |
|---|---|---|---|
| upper_limit<br>Edge case | BankName × Payment Format segment has insufficient observations for P95 | Account P95 → Global P95 | Hierarchical two-level fallback preserves contextual granularity at each level before resorting to a population-level bound. |
| time_diff<br>Certain | First transaction per account — .diff() always yields NaN for the first row | 0 seconds | No elapsed time exists before the first event; zero is the only semantically correct value. |
| avg_time_diff<br>Certain | Accounts with a single transaction — mean of a lone NaN from .diff() is NaN | Population median | The median is robust to outliers in intertransaction intervals |
| | | | and avoids the distortion introduced by extreme high-frequency accounts (Sterne & White, 2009). |
| prev_time<br>Certain | First transaction per account — .shift(1) always yields NaN | Curre nt timestamp string | Self-reference for the first event; prevents null propagation in downstream temporal features while remaining interpretable. |
| amount_ratio_recv_paid<br>Edge case | Amount Paid equals zero — division undefined | 1.0 (neutral ratio) | A ratio of 1.0 implies no currency conversion discrepancy, representing a neutral exchange. |
| monthly_*_change_pct<br>Certain | First calendar month per account — .shift(1) yields NaN; no prior period exists | 0 (no change) | No prior period means no measurable change. Zero encodes the neutral baseline and prevents spurious spike detection on the first observed month. |

| | | | |
|---|---|---|---|
| monthly_amount_std monthly_tx_std std_daily_* std_hourly_tx std_yearly_tx Certain | Groups with a single observation — .std() is mathematically undefined (returns NaN) | 0 | The standard deviation of a single observation is defined as zero in descriptive statistics; this is the only mathematically correct imputation. |
| peak_hour Certain | Accounts absent from hourly peak after left join (singletransaction accounts have no peak) | Curr ent transaction hour | With no activity history, the current transaction's hour is the best available estimate of the account's activity peak (mode imputation proxy). |

Table 4.1 AML – Missing Value Handling

### 4.2.3 Feature Encoding

Categorical features were encoded using two distinct strategies based on the nature of each variable:

One-Hot Encoding was applied to three categorical features that lack inherent ordinal relationships: Payment Format (7 categories: ACH, Bitcoin, Cash, Cheque, Credit Card, Reinvestment, Wire), Payment Currency (15 currencies), and Receiving Currency (15 currencies). This produced 37 binary indicator columns. One-Hot Encoding was selected because these features are purely nominal; applying ordinal encoding would incorrectly impose a rank ordering (e.g., implying that Wire > Cash), which has no semantic meaning in the AML domain.

For the entity_type_score feature, a domain-driven ordinal mapping was applied manually rather than using automated label encoding. Each entity type was assigned a numerical score reflecting its relative risk level within the AML context: Corporation = 3, Sole Proprietorship, Partnership, and Country = 2, and Individual and Direct = 1. This mapping is grounded in the Financial Action Task Force (FATF) guidelines and established Know Your Customer (KYC) practices, which recognise that corporate and partnership structures are generally associated with higher exposure to layering and integration-stage laundering activity due to the complexity of their ownership structures. Unlike label encoding, which assigns arbitrary integers to categorical values based on alphabetical or insertion order, this manual ordinal mapping preserves a meaningful risk hierarchy that is directly interpretable within the AML domain.

The final feature set comprised 42 features: 5 base features (Amount Paid, Amount Received, day, hour, entity_type_score) plus 37 One-Hot encoded columns.

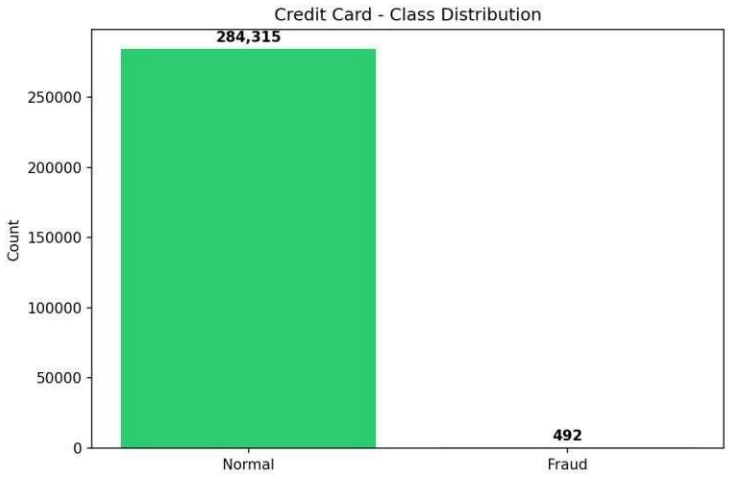
### 4.2.4 Credit Card Dataset Preprocessing

The Credit Card Fraud Detection dataset, sourced from Kaggle, consists of 284,807 anonymised credit card transactions made by European cardholders over a two-day period in September 2013. The dataset contains 31 features: 28 PCA-transformed components (V1 through V28) generated by the dataset creators to protect cardholder privacy while preserving statistical relationships, along with the original Time and Amount features and a binary target variable Class (1 = fraud, 0 = normal). The dataset exhibits extreme class imbalance, with only 492 fraudulent transactions (0.172%) out of 284,807 total transactions.

A sample of 30,000 transactions was drawn from the original dataset for model development. All 492 fraudulent transactions were retained in the sample to preserve the complete fraud signal, alongside 29,508 randomly selected normal transactions, yielding a fraud rate of 1.64% in the working sample.

The Time feature was excluded from model inputs. Since Time represents the elapsed seconds from the first transaction in the dataset rather than an absolute timestamp, it functions as a sequential index that encodes positional information rather than meaningful temporal behaviour. Including it risks introducing data leakage, as the model could exploit temporal ordering patterns that would not generalise to real-world deployment where transactions arrive in arbitrary order. The 28 PCA-transformed features (V1 through V28) plus the Amount field were retained, yielding 29 input features.

No additional feature engineering was performed on this dataset. Because the V1 through V28 features have already undergone Principal Component Analysis by the dataset creators, the variance structure of the original feature space is already captured in the transformed components. Applying further transformations such as interaction terms or polynomial expansions on PCA-transformed features would not recover additional information and could introduce noise.



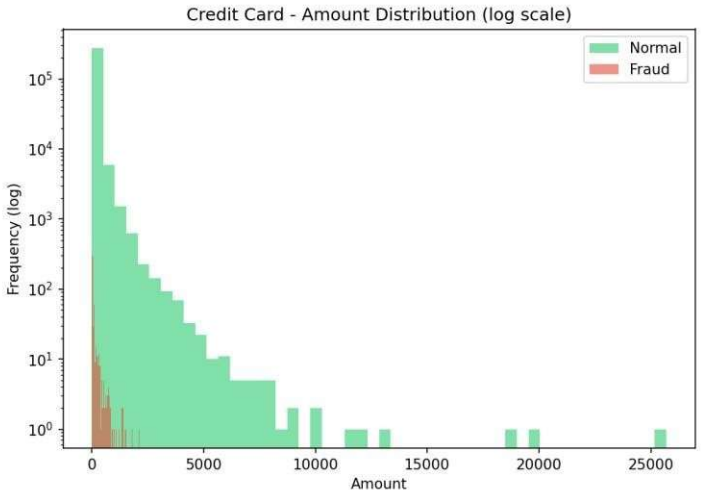



Figure 4.3: Credit Card Dataset: PCA Feature Distribution Overview

### 4.2.5 PaySim Dataset Preprocessing

The PaySim dataset is a synthetic dataset that simulates mobile money transactions based on real transaction logs from an African mobile money service. It contains 6,362,620 transactions with 11 original features, including step (a unit of time representing one hour of real-world simulation), type (five transaction categories: CASH_IN, CASH_OUT, DEBIT, PAYMENT, and TRANSFER), amount, sender and receiver account identifiers, and pre- and post-transaction balances for both origin and destination accounts. The dataset also includes two target-related columns: isFraud (the ground-truth label) and isFlaggedFraud (the legacy rule-based system flag). Out of the 6,362,620 transactions, 8,213 are labelled as fraudulent, yielding a fraud rate of approximately 0.13%.

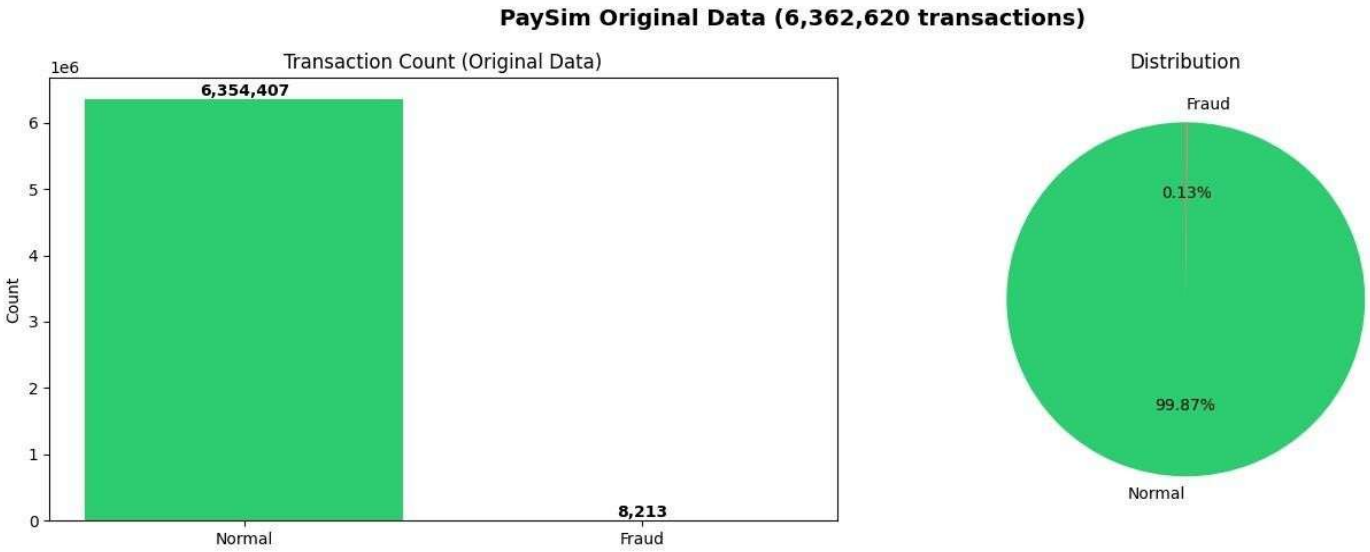



**Figure 4.4: PaySim Dataset: Transaction Type Distribution**

A notable characteristic of this dataset is that fraudulent transactions occur exclusively in two transaction types: CASH_OUT and TRANSFER. No fraud is observed in CASH_IN, DEBIT, or PAYMENT transactions. This concentration is a structural property of the PaySim simulation design rather than an

artefact of the analysis pipeline, and it reflects the real-world observation that money laundering and fraud in mobile money systems predominantly exploit fund withdrawal and peer-to-peer transfer mechanisms.

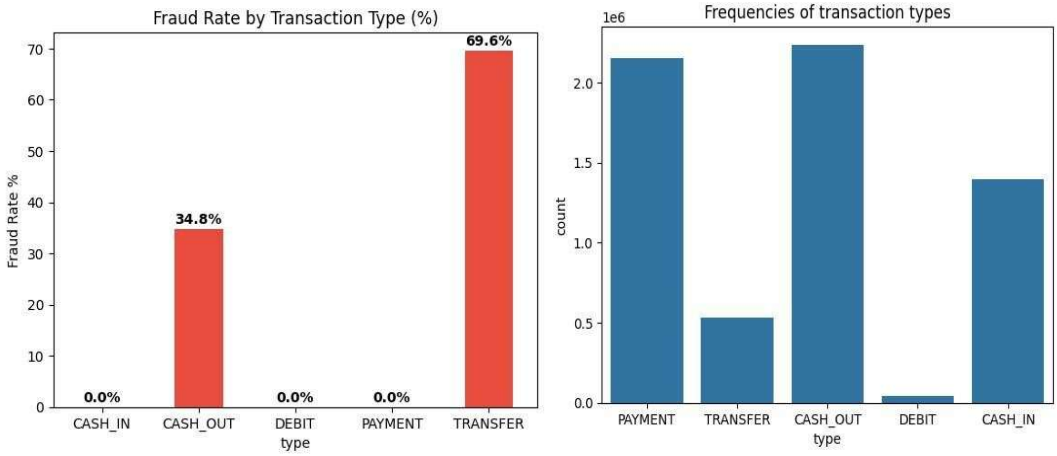



**Figure 4.5: PaySim Dataset: Fraud vs. Normal Transaction Proportions**

A sample of 30,000 transactions was drawn from the original dataset. All 8,213 fraudulent transactions were included in the sample to preserve the complete fraud signal, alongside 21,787 randomly selected normal transactions. This yielded a fraud rate of approximately 27.38% in the working sample, substantially higher than the original 0.13% to facilitate effective model training on minority-class patterns.

The isFlaggedFraud column was excluded from the model inputs. This feature represents the output of a simplistic legacy rule-based system that flags transactions exceeding a fixed monetary threshold. Upon analysis, it was found that this system correctly identified only 16 fraudulent transactions out of 8,213 total fraud cases in the dataset, highlighting its extremely limited effectiveness and the overly simplistic nature of the rule-based approach.

```
    print("isFlaggedFraud = 1 حالات:")
    flagged = df_ps_sample[df_ps_sample['isFlaggedFraud'] == 1]
    print(f"العدد: {len(flagged)}")
    print(f"\nالأنواع:")
    print(flagged['type'].value_counts())
    print(f"\nأقل مبلغ: {flagged['amount'].min():,.0f}")
    print(f"أكبر مبلغ: {flagged['amount'].max():,.0f}")
```

```
isFlaggedFraud = 1 حالات:
16 :العدد

الأنواع:
type
TRANSFER    16
Name: count, dtype: int64

353,874 :أقل مبلغ
10,000,000 :أكبر مبلغ
```

```
print(f"(6 مليون) بالبيانات الأصلية:")
print(f"  isFlaggedFraud = 1: {df_ps['isFlaggedFraud'].sum()}")
print(f"  إجمالي الاحتيال: {df_ps['isFraud'].sum()}")
print(f"  نسبة الكشف: {df_ps['isFlaggedFraud'].sum() / df_ps['isFraud'].sum():.2%}")
#Very poor detection rule-based system
```

```
(6 مليون) بالبيانات الأصلية:
  isFlaggedFraud = 1: 16
  إجمالي الاحتيال: 8213
  نسبة الكشف: 0.19%
```

**Figure 4.6: PaySim Dataset: isFlaggedFraud Rule-Based System Performance**

Including this feature as an input would introduce a form of label leakage, as it embeds prior knowledge about suspicious transactions that the model is expected to learn independently. Moreover, the poor detection performance of this feature further justifies the need for machine learning-based approaches, which are capable of capturing complex and non-linear fraud patterns that traditional rule-based systems fail to detect.

Feature engineering for PaySim produced four derived features. The balance_diff_orig feature captures the net change in the sender account balance (oldbalanceOrg minus newbalanceOrig), which reveals complete account draining patterns commonly associated with fraud. The balance_diff_dest feature captures the corresponding change in the receiver account balance, detecting unusual accumulation indicative of mule account behaviour. The amount_vs_balance feature computes the ratio of the transaction amount to the sender's available balance, identifying disproportionately large transfers relative to account holdings. The is_night feature was introduced as a binary indicator flagging transactions occurring between 12:00 AM and 8:00 AM (hours 0 through 7 in the step-derived hour field), motivated by the empirical observation that fraudulent transactions in the dataset are disproportionately concentrated during overnight hours when legitimate user activity is minimal.

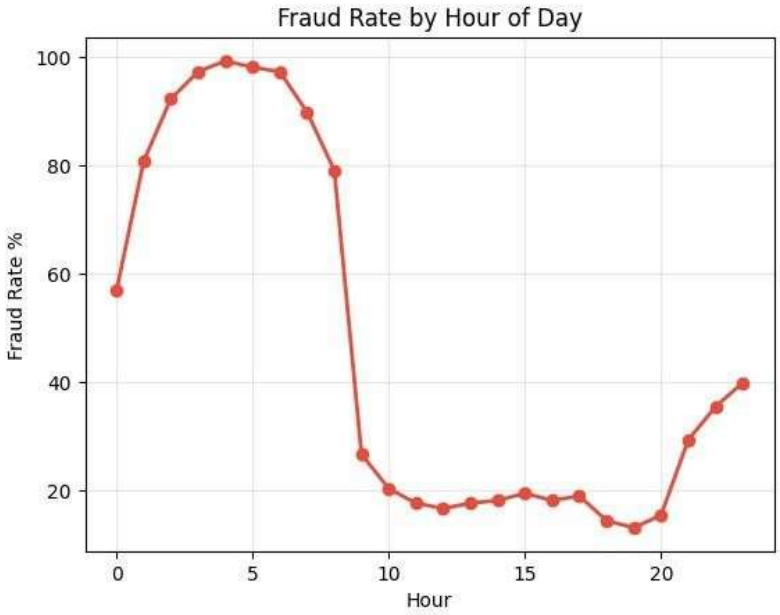



**Figure 4.7: PaySim Dataset: Feature Engineering-Fraud Hour**

Additionally, One-Hot Encoding was applied to the type column, producing five binary features (type_CASH_IN, type_CASH_OUT, type_DEBIT, type_PAYMENT, type_TRANSFER). This encoding was preferred over ordinal encoding because the five transaction types have no inherent rank ordering. The dest_is_customer feature, which flags whether the destination account belongs to a customer rather than a merchant, is closely tied to the fraud label: every fraudulent transaction in the PaySim dataset is sent to a customer account, so a merchant destination rules fraud out entirely. Because this pattern is a structural artefact of the simulator that largely follows from the transaction type, including it would introduce a risk of label leakage (data leakage). Therefore, to ensure a realistic and unbiased learning process, this feature was excluded from the model inputs.




**Figure 4.8: PaySim Dataset: Fraud by Destination Type**

No additional feature engineering beyond the features described above was performed on this dataset. Unlike the IBM AML dataset, PaySim does not contain account-level transaction histories spanning multiple months, which precludes the construction of temporal behavioural features such as monthly aggregations, velocity tracking, or spike detection. The feature set was therefore limited to transactionlevel attributes and the four engineered features, yielding 14 model input features in total.

## 4.3 Feature Engineering

### 4.3.1 IBM AML Feature Engineering

Over 60 behavioural features were engineered from the IBM AML transaction data, organised into six categories. These features were designed exclusively for the analytics dashboard and were NOT used as model inputs, to prevent temporal and label leakage:

**Amount and Threshold Features:** prev_mean_amount and prev_max_amount represent the expanding-window mean and maximum of all prior transaction amounts for each account, computed using a cumulative calculation with a one-step shift to exclude the current transaction. From these baselines, two ratio features were derived: amount_to_mean_ratio and amount_to_max_ratio, which measure the current transaction amount relative to the account's historical average and peak, respectively. The upper_limit feature is the account's own 95th-percentile amount when the account has more than five transactions in the sample, and otherwise the 95th percentile of its bank-payment format segment, with account-level and global fallbacks. Two binary flags complete this group: is_large_amount, which activates when the transaction amount exceeds the

account-level 95th percentile, and is_abnormal, which activates when the amount exceeds upper_limit.

**Time and Velocity Features:** time_diff captures the elapsed seconds since the previous transaction for the same account, while avg_time_diff represents the mean inter-transaction interval across the account's full history. The binary flag is_faster_than_normal activates when time_diff falls below avg_time_diff, identifying transactions occurring at above-average velocity. Two frequency measures, transactions_per_hour and transactions_per_day, count the number of transactions within the same account-hour and account-day combinations, respectively.

**Network Features:** unique_receivers counts the number of distinct destination accounts per sender. The recv_count feature records how many times a specific sender-receiver pair has transacted, and repeat_receiver is a binary flag indicating whether the receiver has been observed before. The is_cross_bank flag activates when the sender and receiver banks differ. The receiver_risk_count feature tallies the total number of incoming transactions for each destination account across the entire dataset. The currency_mismatch flag activates when the payment currency differs from the receiving currency, and amount_ratio_recv_paid captures the ratio of the received amount to the paid amount, highlighting currency conversion discrepancies.

**Monthly Tracking Features:** Monthly aggregations were computed per account per calendar month, including monthly_tx_count, monthly_total_amount, monthly_avg_amount, monthly_max_amount, monthly_unique_receivers, and monthly_cross_bank. For each of these metrics, a month-over-month percentage change was calculated by comparing the current month to the previous month. Three binary spike indicators (amount_spike, tx_count_spike, receiver_spike) were derived, activating when the absolute month-over-month change exceeded 200%. Cumulative spike totals (total_amount_spikes, total_tx_spikes, total_receiver_spikes) were computed to quantify each account's historical anomaly frequency. Monthly volatility was captured through monthly_amount_std and monthly_tx_std, and a months_active counter recorded the number of distinct calendar months in which the account appeared.

**Daily and Hourly Features:** Daily aggregations include avg_daily_tx_count, max_daily_tx_count, std_daily_tx_count, avg_daily_amount, max_daily_amount, std_daily_amount, avg_daily_receivers, max_daily_receivers, and active_days. Hourly aggregations include avg_hourly_tx, max_hourly_tx, std_hourly_tx, peak_hour (the hour with the highest transaction frequency), and active_hours. A night

activity analysis was performed by counting transactions occurring between 23:00 and 05:00, producing night_tx_count and night_tx_ratio (night transactions divided by total transactions per account).

**Yearly Features:** Yearly aggregations include avg_yearly_tx, max_yearly_tx, std_yearly_tx, avg_yearly_amount, max_yearly_amount, active_years, and yearly_amount_change_pct, which measures the year-over-year percentage change in total transaction volume.

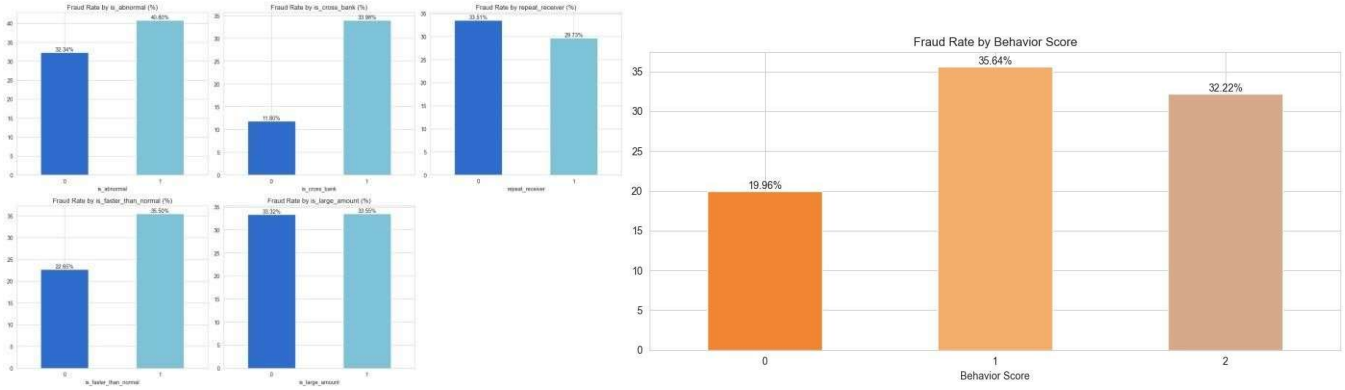



**Figure 4.9: IBM AML Feature Engineering: Engineered Feature Categories Overview**

## 4.3.2 Feature Selection Rationale

Only 5 base features plus One-Hot encoded categoricals (totalling 42 features) were selected as model inputs for the IBM AML dataset. All 60+ engineered features were explicitly excluded from model training for the following reasons:

Temporal Leakage: Features computed from the full account history (e.g., monthly aggregations, unique_receivers) include information from transactions that may appear in the test set, violating the train-test independence assumption. In a production system, these features would require proper temporal splitting to ensure that only past data informs each prediction.

Sampling Leakage: fraud and normal transactions were sampled separately, so per-account counts computed inside the sample (e.g., transactions_per_day) partly reflect the sampling design and therefore the label.

Redundancy: Features like currency_mismatch are already captured by the combination of One-Hot encoded Payment Currency and Receiving Currency columns. Including both the derived flag and the underlying encoded features would introduce multicollinearity without adding discriminative power.

The engineered features remain in the dataset (df_merged_features.parquet) and are used extensively in the application dashboard for behavioural analytics, customer profiling, and compliance reporting. This design separates the model's predictive scope from the dashboard's analytical scope, ensuring that the model's performance metrics reflect genuine generalisation rather than information leakage.

The models nevertheless achieved strong results with the base features alone (test ROC-AUC 0.90–0.92), so the engineered features were kept for the dashboard. Whether they would add predictive value under a leakage-free temporal split is left for future work (Section 6.2).

### 4.3.3 Unsupervised Learning: Exploration and Rationale for Exclusion

During the initial design phase, unsupervised anomaly detection models, including Isolation Forest, OneClass SVM, and Autoencoder, were explored as complementary components to the supervised pipeline. The motivation was that unsupervised methods could detect novel fraud patterns not represented in the labelled training data, since they learn the boundary of normal behaviour without relying on explicit fraud labels.

However, after implementation and evaluation, the unsupervised models were excluded from the final system for several reasons. First, all three datasets used in this project provide ground-truth fraud labels, which makes supervised learning the natural and more effective approach. Second, the unsupervised models produced substantially lower precision on the IBM AML dataset, generating a high volume of false positives that would be impractical for compliance teams to review. Third, the evaluation of unsupervised models on labelled data is inherently problematic: metrics like ROC-AUC assume access to ground truth, which contradicts the unsupervised premise. Fourth, the computational cost of One-Class SVM on 30,000 samples was prohibitive, and the Autoencoder required extensive hyperparameter tuning without yielding improvements over the supervised baselines.

Consequently, the final SKYC system relies exclusively on three supervised models (Logistic Regression, Random Forest, and XGBoost) across all three datasets, with the unsupervised approach documented as a planned extension for future deployment scenarios where labelled data is scarce or unavailable.

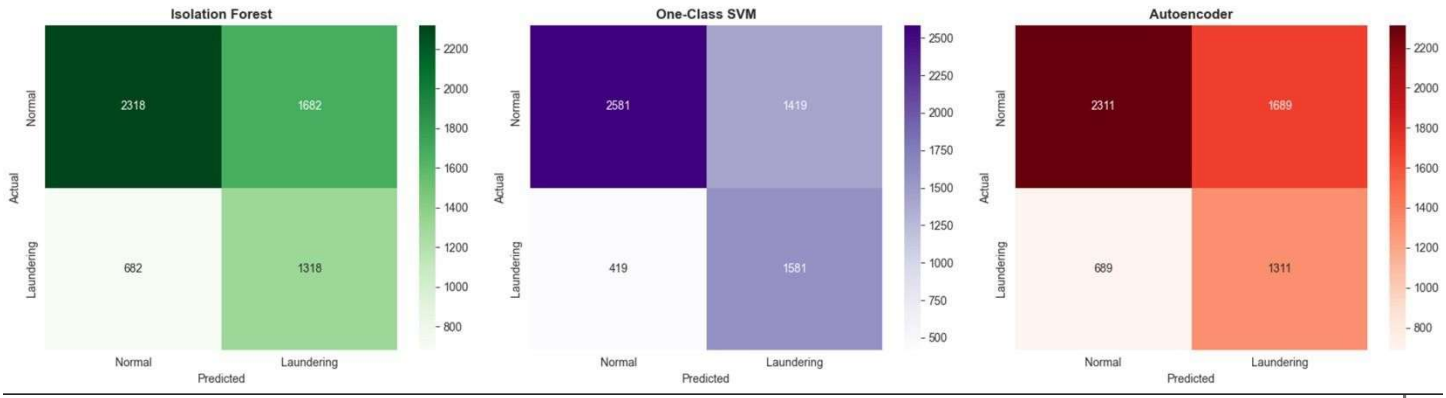



**Figure 4.10: Unsupervised Anomaly Detection Models: Exploration and Comparison**

## 4.4 Model Training Pipeline

### 4.4.1 Scaling

StandardScaler (zero mean, unit variance) was applied to all features before model training. The scaler was fit exclusively on the training data and used to transform both training and test sets, preventing data leakage. Scaling was applied uniformly across all three models and across all datasets to ensure fair comparison and consistent preprocessing behaviour throughout the entire system.

StandardScaler was specifically chosen because it standardises features by removing the mean and scaling to unit variance, ensuring that all variables contribute equally to the learning process regardless of their original scale. This is particularly important in fraud detection datasets, where features such as transaction amounts and ratio-based engineered variables exist on significantly different scales. Unlike Min-Max scaling, StandardScaler is more robust to outliers, which are common in financial transaction data.

The scaler was fitted only on the training data to avoid data leakage, as using the test set during the fitting process would introduce future information into the learning pipeline and lead to overly optimistic performance estimates. Therefore, the mean and standard deviation are computed solely from the training set, and the same transformation is then applied to the test set to ensure that the model does not gain any prior knowledge from unseen data.

While tree-based models (Random Forest and XGBoost) are mathematically scale-invariant, scaling was applied for three main reasons: first, to ensure pipeline consistency across all models, including Logistic Regression, which requires scaled inputs for stable gradient convergence; second, to maintain a fair comparison by eliminating preprocessing as a confounding variable; and third, because SMOTE generates synthetic samples through interpolation in feature space, and unscaled features could disproportionately bias the synthetic sample generation toward high-magnitude features.

```
# ── 2. Scale – fit ONLY on training data ──────────────────
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)   # fit + transform
X_test_scaled  = scaler.transform(X_test)        # transform only (no fit)
print(f"\n✅ Scaling done (fit on train, transform on test)")
```

**Figure 4.11: StandardScaler Preprocessing Pipeline Across All Datasets**

### 4.4.2 Class Balancing with SMOTE

Class imbalance was addressed using different strategies across the three datasets, informed by the characteristics of each dataset and the results of systematic experimentation.

For the IBM AML dataset, the Synthetic Minority Over-sampling Technique (SMOTE) was applied to the scaled training data, generating synthetic fraud samples by interpolating between existing minority-class instances in feature space. SMOTE was applied exclusively within the training partition and was never applied to validation or test data, preventing optimistic bias in performance estimates. To avoid doublepenalising the majority class, the class_weight parameter was set to None for all three models on this dataset, since combining SMOTE with class_weight=balanced would apply two correction mechanisms simultaneously, effectively over-compensating for the imbalance and biasing the decision boundary toward the minority class. For Stratified K-Fold Cross Validation, SMOTE was applied independently within each fold to prevent information from synthetic samples leaking into validation folds. A fresh StandardScaler and SMOTE instance were created for each fold, ensuring complete independence between the preprocessing of training and validation partitions.

For the Credit Card and PaySim datasets, SMOTE was not applied. The four-configuration experiments (FE+SMOTE, FE+NO_SMOTE, NO_FE+SMOTE, NO_FE+NO_SMOTE) conducted on both datasets showed that SMOTE gave at most a small Recall gain (0–6 percentage points on Credit Card, under 1 point on PaySim) at the cost of lower Precision (for example, XGBoost on Credit Card fell from 1.00 to 0.87). Additionally, the Credit Card features are PCA-transformed components, and interpolating between instances in PCA space does not necessarily produce samples that correspond to realistic transaction patterns, potentially introducing noise rather than useful training signal.

For the Credit Card dataset, class_weight='balanced' was used for Logistic Regression and Random Forest (XGBoost was trained without class weighting) to compensate for the low fraud rate of 1.64% in the working sample. For the PaySim dataset, class_weight was also set to balanced for these two models for consistency; with a fraud rate of 27.38% this is only a modest 2.65x weight adjustment. The near-perfect performance of the tree models across all configurations (F1 > 0.99) confirms that the PaySim class distribution was sufficiently balanced for effective training regardless of the weighting strategy employed.

| CONFIG | MODEL | PRECISION | RECALL | F1 | ROC-AUC | PR-AUC |
|---|---|---|---|---|---|---|
| **No SMOTE** | LR | 0.43 | 0.90 | 0.58 | 0.9820 | 0.9081 |
| | RF | 0.97 | 0.87 | 0.91 | 0.9856 | 0.9208 |
| | XGB | 1.00 | 0.83 | 0.91 | 0.9862 | 0.9256 |

| | | | | | | |
|---|---|---|---|---|---|---|
| SMOTE | LR | 0.42 | 0.91 | 0.58 | 0.9793 | 0.9075 |
| | RF | 0.92 | 0.87 | 0.89 | 0.9822 | 0.9272 |
| | XGB | 0.87 | 0.89 | 0.88 | 0.9826 | 0.9147 |

No FE for Credit Card (PCA features as-is); LR and RF use class_weight='balanced' in both rows. SMOTE raised XGB Recall 0.83→0.89 but cut its Precision 1.00→0.87. Adopted: No SMOTE. Best model: XGBoost (Precision 1.00, AUC 0.9862).

Table 4.2:Credit Card Fraud class (class 1) — all models × SMOTE vs No SMOTE

| CONFIG | MODEL | PRECISION | RECALL | F1 | ROC-AUC | PR-AUC |
|---|---|---|---|---|---|---|
| **FE No SMOTE** | LR | 0.92 | **0.95** | **0.93** | **0.9933** | **0.9863** |
| | RF | 1.00 | **1.00** | **1.00** | **0.9997** | **0.9995** |
| | XGB | 1.00 | **1.00** | **1.00** | **0.9995** | **0.9991** |
| **FE SMOTE** | LR | 0.93 | 0.95 | 0.94 | 0.9937 | 0.9872 |
| | RF | 1.00 | 1.00 | 1.00 | 0.9996 | 0.9993 |
| | XGB | 1.00 | 1.00 | 1.00 | 0.9994 | 0.9991 |
| **No FE No SMOTE** | LR | 0.95 | 0.81 | 0.88 | 0.9846 | 0.9659 |
| | RF | 0.99 | 0.99 | 0.99 | 0.9986 | 0.9977 |
| | XGB | 0.99 | 1.00 | 0.99 | 0.9992 | 0.9984 |
| **No FE SMOTE** | LR | 0.87 | **0.92** | 0.89 | 0.9874 | 0.9707 |
| | RF | 0.99 | **1.00** | 0.99 | 0.9990 | 0.9981 |
| | XGB | 0.98 | **0.99** | 0.99 | 0.9990 | 0.9981 |

FE + No SMOTE raised LR F1 from 0.88 to 0.93 (this config also uses class_weight='balanced'). RF and XGB reach F1 of about 1.00 (0.998) with FE; SMOTE had negligible effect on them. Adopted: FE + No SMOTE. Best model: RF (F1 0.998, AUC 0.9997).

Table 4.3:Paysim Fraud class (class 1) — all models × all configs.

### 4.4.3 Model Architectures

Three classification algorithms were selected to represent complementary modelling paradigms: a linear model (Logistic Regression), a bagging ensemble (Random Forest), and a boosting ensemble (XGBoost). Each model was configured with hyperparameters informed by the dataset characteristics and validated through systematic experimentation.

**IBM AML Dataset Configuration:**

**Random Forest:** n_estimators=200, max_depth=15, min_samples_leaf=5, class_weight=None, random_state=42. The choice of 200 trees balances computational cost against ensemble diversity; empirical studies (Breiman, 2001) demonstrate diminishing returns beyond this point. max_depth=15 permits deep splits because Random Forest mitigates individual-tree overfitting through bootstrap aggregation and feature sub-sampling at each split. The ensemble averages predictions across 200 independently grown trees, reducing variance without requiring depth restriction. min_samples_leaf=5 prevents terminal nodes from memorising individual transactions, imposing a regularisation floor. class_weight was set to None because SMOTE already balances the training partition to a 1:1 ratio; applying class_weight=balanced simultaneously would double-penalise the majority class, effectively shifting the decision boundary toward over-predicting fraud and inflating false positives.

**XGBoost:** n_estimators=200, max_depth=8, learning_rate=0.05, eval_metric=logloss, random_state=42. max_depth=8 is deliberately shallower than Random Forest because boosting builds trees sequentially, where each tree corrects the residual errors of its predecessor. Deep trees in a boosting framework compound sequential corrections, leading to rapid overfitting to training noise. The depth of 8 strikes a balance between capturing complex feature interactions (which transaction-level AML patterns require) and preventing memorisation. learning_rate=0.05 shrinks the contribution of each tree by 95%, forcing the ensemble to build consensus gradually across 200 iterations rather than relying on early aggressive corrections. eval_metric=logloss directly optimises the log-likelihood of correct class probabilities, which is preferable to accuracy-based metrics on imbalanced data. class_weight was not applied (None), consistent with the SMOTE-based balancing strategy.

**Logistic Regression:** max_iter=1000, class_weight=None, C=1.0, penalty=l2, solver=lbfgs, random_state=42. max_iter=1000 ensures convergence on the 42-feature input space; the default of 100 iterations proved insufficient during preliminary runs, producing ConvergenceWarnings. L2 regularisation (penalty=l2) with C=1.0 penalises large coefficients, preventing any single feature from dominating the

decision boundary, which is particularly important when One-Hot encoded features create sparse binary inputs. The lbfgs solver is well-suited for L2 regularisation on moderately-sized datasets. As for the tree models, class_weight was set to None for the train/test (deployed) Logistic Regression, because SMOTE already balances the training partition. In the 5-fold cross-validation runs, however, Logistic Regression was configured with class_weight='balanced', so its K-Fold results (Section 5.2.3) describe this weighted variant.

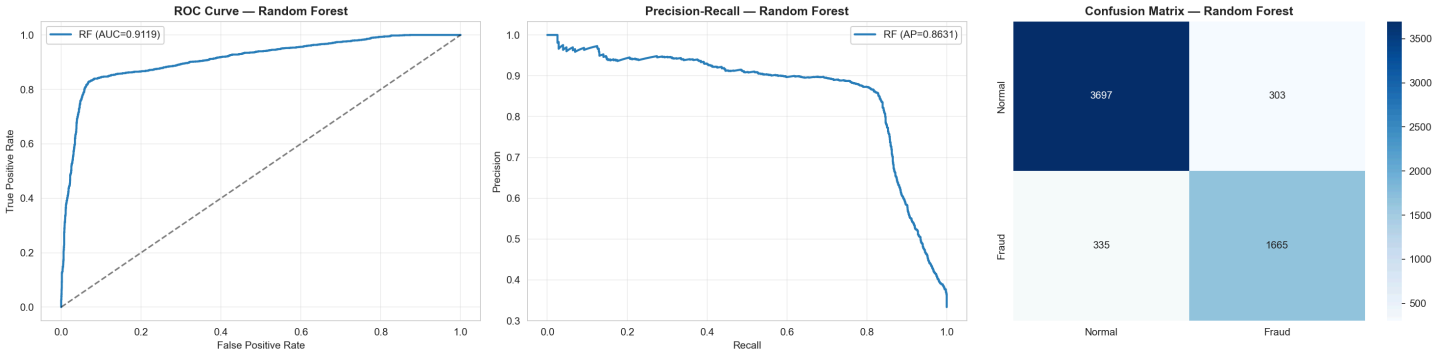



Figure 4.12: IBM AML – Random Forest: Confusion Matrix

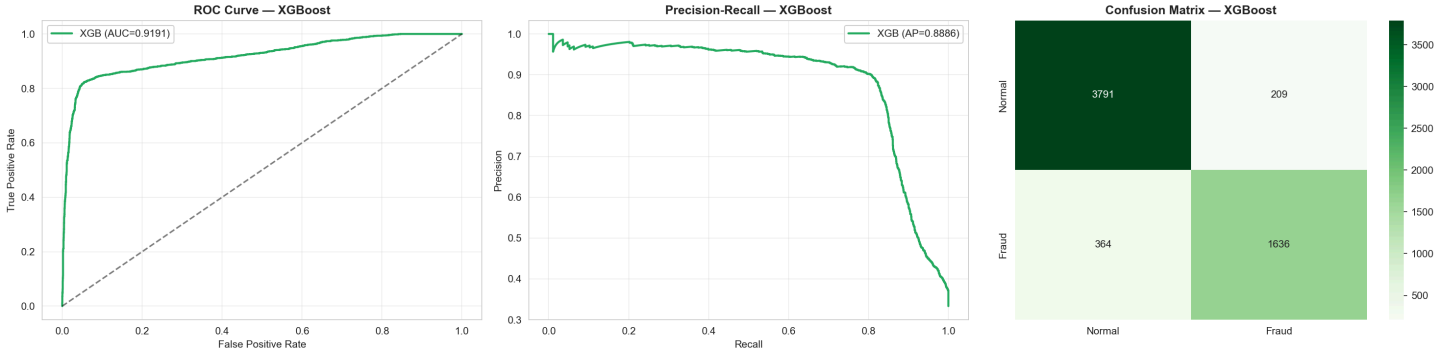



Figure 4.13: IBM AML – XGBoost: Confusion Matrix

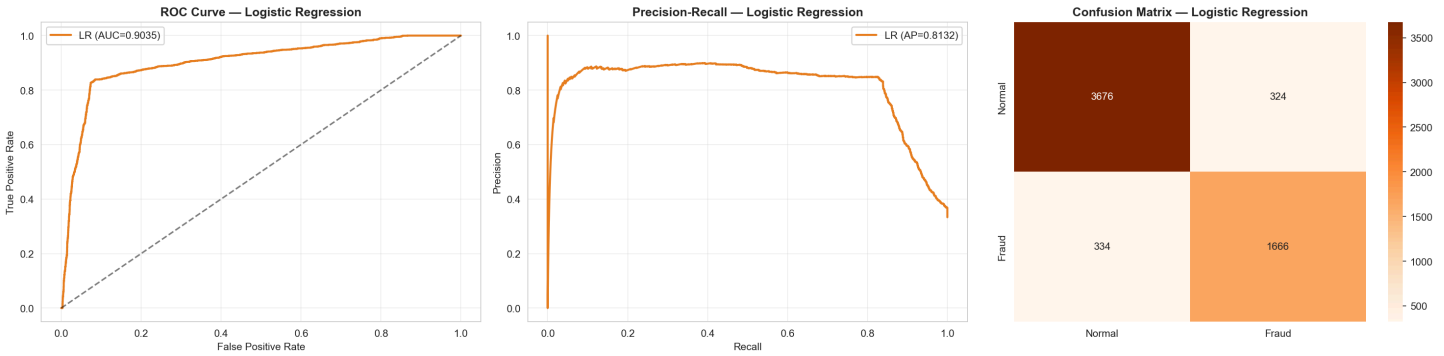



Figure 4.14: IBM AML – Logistic Regression: Confusion Matrix

**XGBoost (Green):**

1.Overall Best Performer.

2.Achieved the highest ROC-AUC (AUC = 0.9191) and the highest average precision in identifying actual fraud (AP = 0.8886).

3.Key Advantage: It produced the lowest number of false positives (false alarms) at only 209 cases, making it the most effective model for minimizing customer disruption.

**Random Forest (Blue):**

1.Excellent and Balanced Performance.

2.High accuracy that closely follows XGBoost (AUC = 0.9119, AP = 0.8631).

3.It generated a moderate number of false positives (303) and missed a similar number of fraud cases (335 false negatives).

**Logistic Regression (Orange):**

1.Lowest overall precision, but with one specific advantage.

2.Recorded the lowest general metrics (AUC = 0.9035, AP = 0.8132).

3.Generated the highest number of false alarms (324).

4.Only Advantage: It was marginally better at minimizing false negatives (missed fraud), missing 334 cases (compared to XGBoost's 364), but this comes at a significant cost to overall precision.

**Credit Card and PaySim Dataset Configuration:**

For the Credit Card and PaySim datasets, where SMOTE was not applied, class_weight='balanced' was set for Logistic Regression and Random Forest. This assigns misclassification penalties inversely proportional to class frequency, increasing the cost of missing a fraudulent transaction, and replaces the role that SMOTE served on IBM AML. XGBoost was trained without any class weighting (no scale_pos_weight). All other hyperparameters remained identical to the IBM AML configuration to ensure that performance differences across datasets reflect genuine data complexity rather than hyperparameter confounds.

1) **Credit Card Dataset**

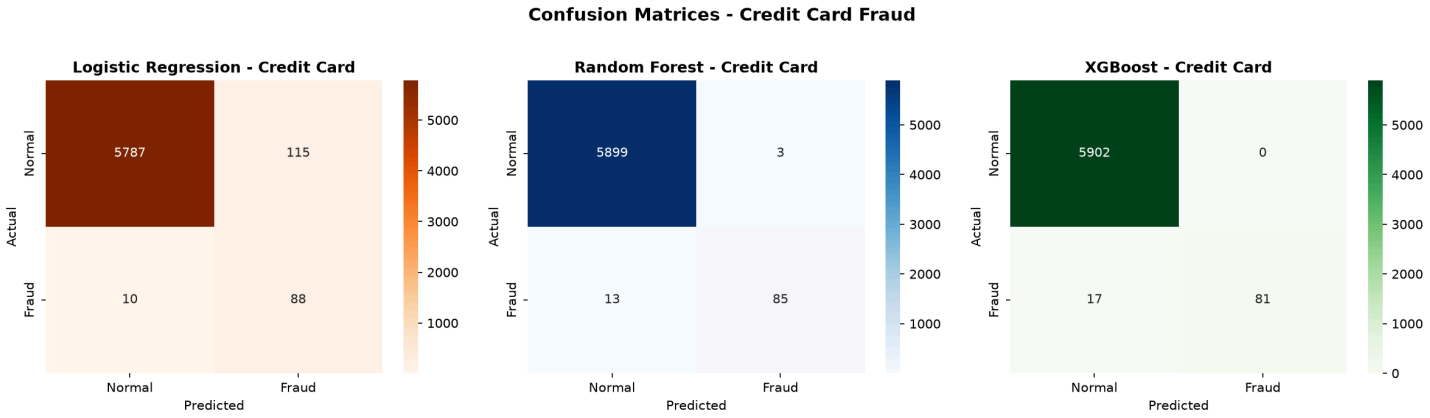



**Figure 4.15: Credit Card Dataset – Confusion Matrices for All Three Models**

Logistic Regression is the most sensitive, detecting the highest number of actual fraud cases (88), but it generates a significant number of false alarms (115). Conversely, XGBoost and Random Forest prioritize precision; they reduce false alarms to almost zero (0 and 3, respectively), ensuring legitimate transactions are not blocked, although they miss slightly more fraudulent cases.

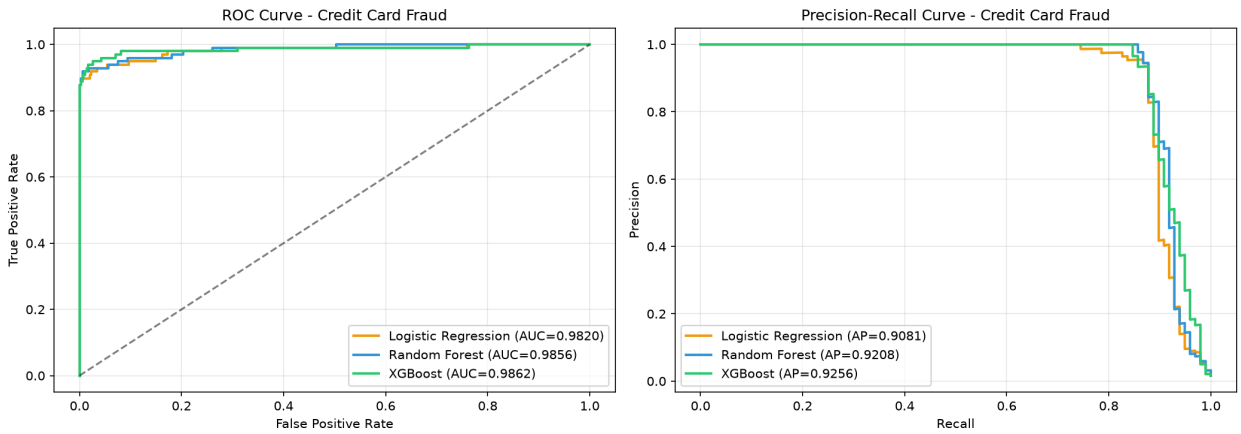



**Figure 4.16: Credit Card Dataset – ROC and Precision-Recall Curves**

All three models demonstrate exceptional performance, with ROC-AUC scores exceeding 0.98. However, XGBoost stands out as the most robust model, achieving the highest overall scores in both AUC (0.9862) and Average Precision (AP = 0.9256). The PR curve clearly shows that the tree-based models (XGBoost and Random Forest) outperform Logistic Regression, indicating they are much better equipped to handle the severe class imbalance typical of financial fraud without sacrificing precision.

2) **PaySim Dataset**

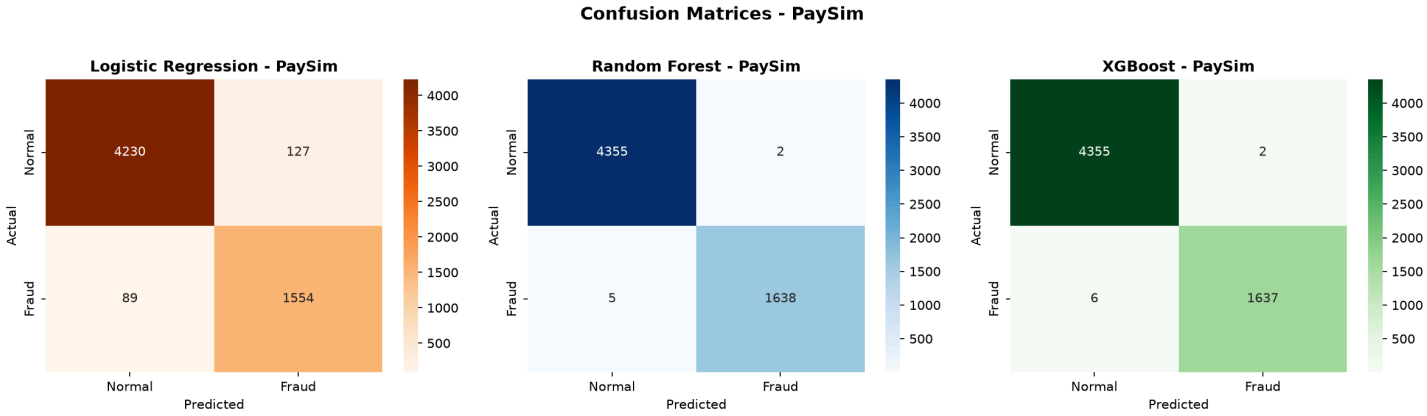



**Figure 4.17: PaySim Dataset – Confusion Matrices for All Three Models**

The tree-based models, Random Forest and XGBoost, demonstrate exceptional and nearly identical performance. Random Forest identified 1638 fraudulent transactions and missed 5, XGBoost identified 1637 and missed 6, and both recorded only 2 false positives. In contrast, Logistic Regression produced significantly more false alarms (127) and missed more actual fraud cases (89). This highlights the superior capability of

both Random Forest and XGBoost in handling the complex transactional patterns present in the PaySim dataset.

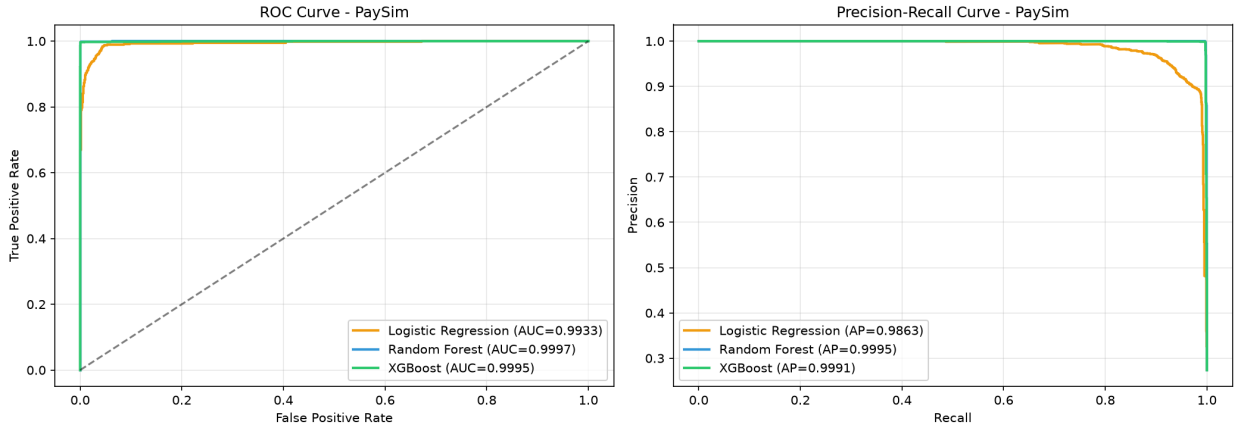



**Figure 4.18: PaySim Dataset – ROC and Precision-Recall Curves**

All models exhibit near-perfect performance on this dataset. However, the tree-based models, Random Forest and XGBoost, are virtually indistinguishable and achieve near-flawless scores (with both AUC and AP exceeding 0.999). Their curves completely overlap, indicating identical, optimal performance. Logistic Regression performs slightly lower (AUC = 0.9933, AP = 0.9863) but still shows excellent predictive capability. The results confirm that Random Forest and XGBoost are exceptionally well-suited for capturing the specific fraud patterns within the PaySim data.

### 4.4.4 Validation Strategy

Two complementary validation approaches were employed for each model to provide both a fast baseline estimate and a robust stability assessment:

**Train/Test Split (80/20):** A stratified split with random_state=42 partitioned each dataset into 80% training and 20% testing subsets, preserving the class distribution in both partitions. For the IBM AML dataset, this yielded 24,000 training transactions (expanded to 32,000 after SMOTE with 16,000 per class) and 6,000 test transactions containing 2,000 fraud cases. The scaler was fit exclusively on the training data and used to transform both partitions, preventing information leakage through feature scaling. SMOTE was applied only to the scaled training data.

**Stratified 5-Fold Cross Validation:** The dataset was divided into five non-overlapping folds with preserved class ratios. In each iteration, four folds served as training data and one fold as validation. A fresh StandardScaler and SMOTE instance were created independently within each fold, ensuring that no

information from synthetic samples or scaling parameters leaked into the validation fold. This provides a more robust performance estimate by averaging metrics over five different data splits, reducing the variance inherent in a single train/test division and exposing any data-split-dependent performance anomalies.

K-Fold cross-validation was used only to evaluate performance and stability. A retraining of all three models on 100% of the sample was also run in the notebook, but the models serialised for the application are the train/test-split models trained on the 80% partition, so the test metrics reported in this chapter and shown in the application correspond exactly to the deployed models.

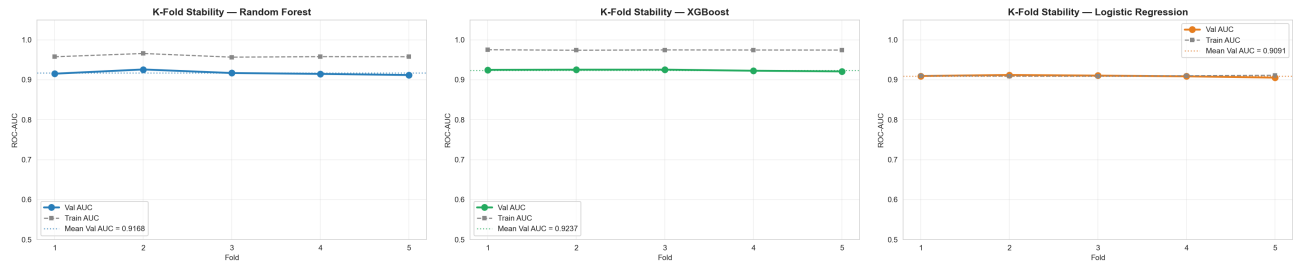



**Figure4.19: IBM AML – 5-Fold Cross-Validation: AUC Learning Curves for All Models**

### Logistic Regression Model:

Stability: The model is highly stable. We can observe that the training data line (Train AUC) and the validation data line (Val AUC) are nearly identical across all five folds.

Performance: The Mean Val AUC is 0.9091.

Conclusion: The model shows no signs of overfitting, meaning it is highly reliable and its real-world performance can be predicted with precision. Although its overall accuracy is the lowest among the three models, it provides an excellent baseline and high transparency for interpreting the results.

### Random Forest Model:

Stability: It is well-stable across the five folds, with only very slight fluctuations in the validation scores.

Performance: The Mean Val AUC increases here to 0.9168.

Conclusion: There is a clear and consistent gap between the training performance (which approaches 0.96) and the validation performance. This indicates a minor degree of overfitting; however, the model

outperforms Logistic Regression in predictive capability and offers a solid balance between accuracy and stability.

**XGBoost Model:**

Stability: The lines remain perfectly stable across all folds without any sharp drops or spikes, demonstrating the consistent performance of the algorithm.

Performance: This model achieves the highest Mean Val AUC at 0.9237.

Conclusion: This is the strongest model in terms of overall performance and its capacity to detect complex patterns and distinguish between classes. The gap between the training line (around 0.97) and the validation line indicates slight overfitting, but its final predictive performance remains superior to the others.

## 4.5 Risk Scoring System

### 4.5.1 Transaction Risk Score (0-100)

Each transaction receives a risk score from 0 to 100, computed as the weighted ensemble average of fraud probabilities from the three production models (LR, RF, XGB). Each model produces a probability via predict_proba, and the ensemble average is scaled to the 0-100 range. Equal weights (1/3 each) are assigned to all three models to avoid introducing subjective bias toward any single classifier. The ensemble approach provides robustness: if one model produces an anomalous probability due to a distributional shift, the other two models moderate the final score.

#### 4.5.2 Customer Risk Score (0-100)

Customer-level risk is a weighted aggregation of the transaction risk scores of an account, computed in the application's Risk Engine layer. It is distinct from the model layer: the three ML models produce transaction-level fraud probabilities from the selected input features, and the Risk Engine aggregates these per account. The 60+ engineered features are not part of this score; they drive the rule-based explanations (Section 4.5.4) and the dashboard.

The Customer Risk Score combines three components: Average Transaction Risk (55% weight), the mean ensemble risk score across all account transactions, which captures the overall behavioural profile; Maximum Transaction Risk (20% weight), the highest single transaction risk score, so that even one highly suspicious transaction elevates the customer risk; and High-Risk Transaction Ratio (25% weight), the percentage of transactions scoring 60 or above, which identifies accounts with persistent rather than isolated suspicious activity. Behavioural flags such as

cross-bank transfers, currency mismatches, night activity and monthly spikes are not added to the score; they are shown as explanations next to it (Section 4.5.4), because they were excluded from model inputs to prevent leakage and serve as descriptive post-hoc indicators.

### 4.5.3 Risk Level Classification

Transactions and customers are classified into four risk levels based on their computed scores: LOW (0-30), requiring no action and representing routine legitimate activity; MEDIUM (30-60), triggering routine monitoring with periodic review; HIGH (60-85), requiring compliance review and enhanced due diligence; and CRITICAL (85-100), necessitating immediate review and Suspicious Activity Report (SAR) consideration. These are fixed thresholds chosen by the team on the 0-100 scale; they were not statistically calibrated, and production use would require calibrating them to the real fraud rate (Section 5.5).

#### 4.5.4 Explainable AI

Every risk score is accompanied by human-readable explanations generated by a rule-based alerting system that operates independently from the ML models. While the models produce a single fraud probability based on the 42 selected input features, the explainability engine examines the full set of engineered features and flags any indicator that exceeds predefined thresholds, regardless of the model's overall verdict. This means that alert cards may appear even when the transaction risk score is LOW. This is by design: the model evaluates the holistic pattern across all features and may determine that a combination of individually suspicious indicators does not constitute fraud, while the rule-based alerts surface each indicator independently so that compliance officers can exercise their own judgement.

For IBM AML transactions, alert categories include: COUNTERPARTY alerts for cross-bank transfers, CURRENCY alerts for payment-receiving currency mismatches, AMOUNT alerts when the transaction exceeds the account's 95th percentile or three times the account average, VELOCITY alerts when the inter-transaction interval is faster than the account's historical average, TIMING alerts for off-hours transactions (00:00-04:59), and BEHAVIOR alerts for month-over-month spikes in amount, transaction count, or receiver diversity. Each alert type has a fixed severity level (MEDIUM or HIGH).

For PaySim, explanations cover transactions that use more than 90% of the sender's balance, large transfers above 500,000 with a significant balance change, and destination type indicating whether funds were sent to a customer account rather than a merchant. For Credit Card, explanations reference transaction amounts above 1,000 and PCA features (V14, V17, V12 or V10) with an absolute value above 3.

These explanations are displayed in Step 3 (Risk Assessment) of the application as colour-coded alert cards, with green borders for LOW, amber borders for MEDIUM, and red borders for HIGH and CRITICAL flags. This layered approach — where models provide a probabilistic score and rules provide interpretable justifications — supports regulatory requirements for transparent and auditable decision-making, aligning with Financial Action Task Force (FATF) guidelines on the use of technology in AML compliance.

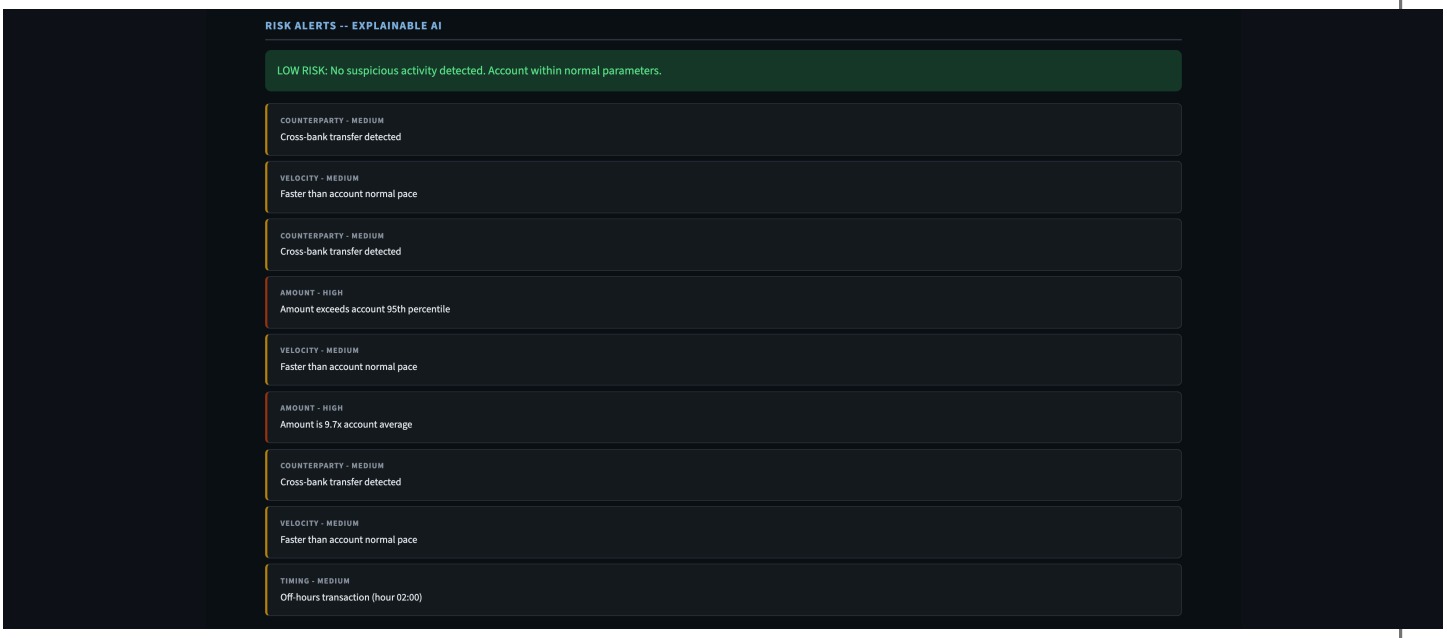



Figure 4.20: Explainable AI Alert Cards: Severity-Coded Transaction Risk Justifications

## 4.6 Application Implementation

### 4.6.1 Technology Stack

The system was built using Streamlit 1.58 as the web framework for rapid prototyping with native Python data support, Plotly for interactive visualisation with hover, zoom, and export capabilities, scikit-learn 1.9.0 and XGBoost 3.2.0 for model training and inference, Apache Parquet via pyarrow for columnar compressed data storage (the 30,000-row, 89-column IBM AML feature file takes about 17 MB as CSV and 4 MB as Parquet), joblib for model serialisation, and custom CSS with the DM Sans font for a banking-grade professional appearance.

### 4.6.2 Application Structure

The application follows a step-based navigation pattern with five sequential pages:

Step 1 (Select Dataset): Three dataset cards (AML Transactions, Credit Card, PaySim) with availability status. Each card loads the corresponding data, models, and scaler.

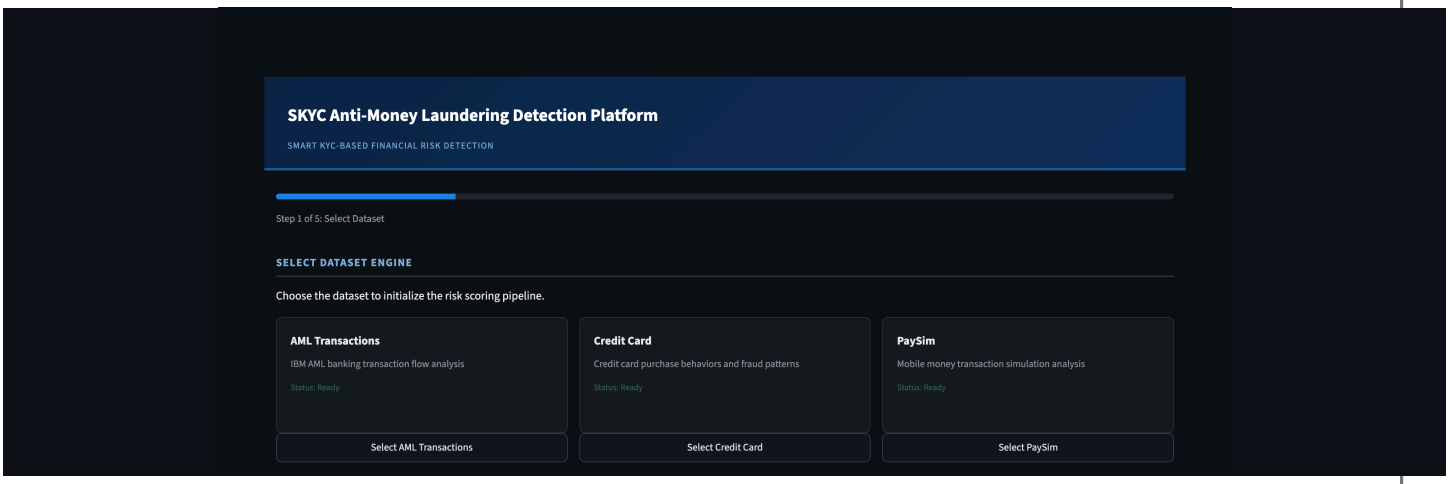



**Figure 4.21: Application Step 1: Dataset Selection Interface**

**Step 2 (Select Account):** Dropdown of all unique accounts for the selected dataset. For the Credit Card dataset, which does not originally contain an account identifier, a synthetic Account_ID column was generated and appended to the dataset to enable account-level grouping and navigation consistent with the other datasets. This column is used exclusively in the application interface for selection and display purposes and is automatically excluded from model inputs during inference, as the model reads its expected feature list from a stored JSON configuration file and ignores any additional columns.

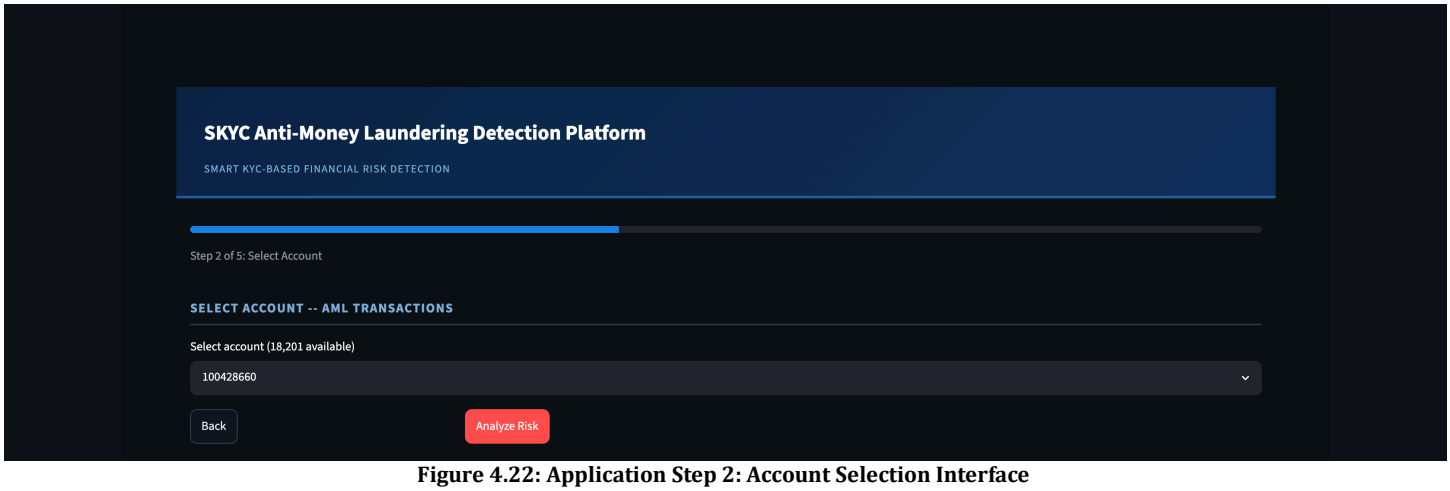



**Figure 4.22: Application Step 2: Account Selection Interface**

Step 3 (Risk Assessment): Customer risk score displayed as a gauge, highest transaction risk, total transactions, fraud rate, explainable AI alerts with severity-coded reason cards (colour-coded by risk level), and a full transaction table with per-model probability scores.

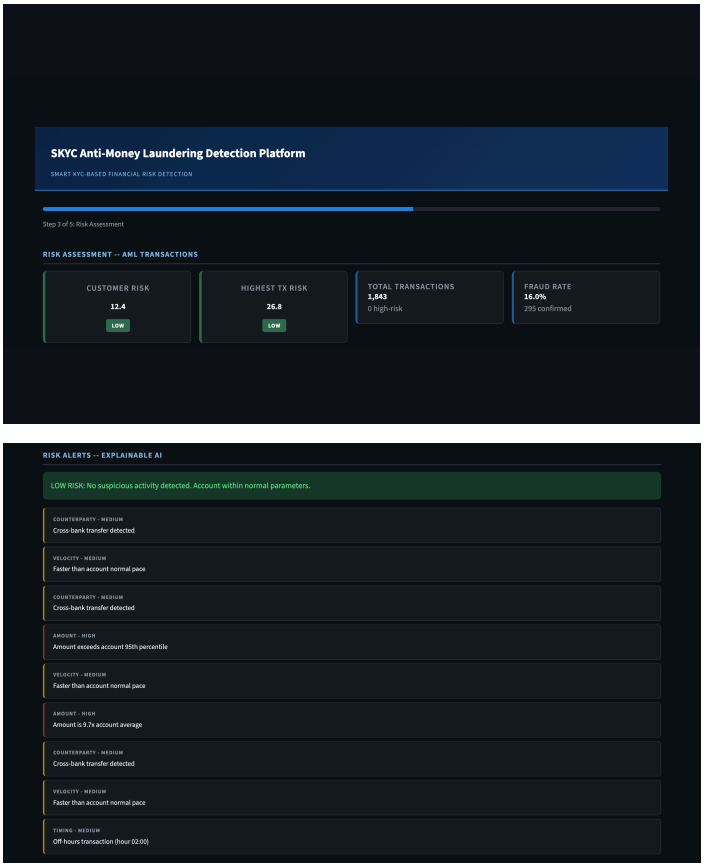



**Figure 4.23: Application Step 3 – Risk Assessment: Customer Risk Gauge and Summary**




**Figure 4.24: Application Step 3 – Risk Assessment: Per-Transaction Risk Table**

Step 4 (Analytics Dashboard): Dataset-specific feature engineering visualisations. AML shows payment format risk distribution, hourly transaction patterns, night activity analysis, spike indicators, cross-bank and currency mismatch analysis, and monthly tracking charts. PaySim shows transaction type risk

distribution, balance difference analysis, and destination type analysis. Credit Card shows an amount vs risk scatter plot and a bar chart of the PCA features most correlated with the risk score.




**Figure 4.25: Application Step 4 – Analytics Dashboard: Behavioural Feature Visualisations**

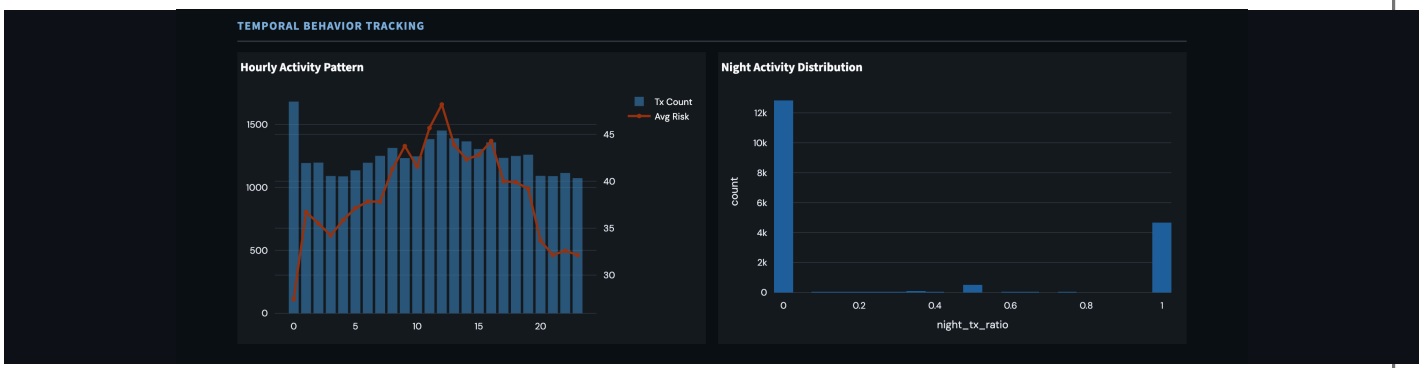



**Figure 4.26: Application Step 4 – Analytics Dashboard: Monthly and Temporal Analysis**

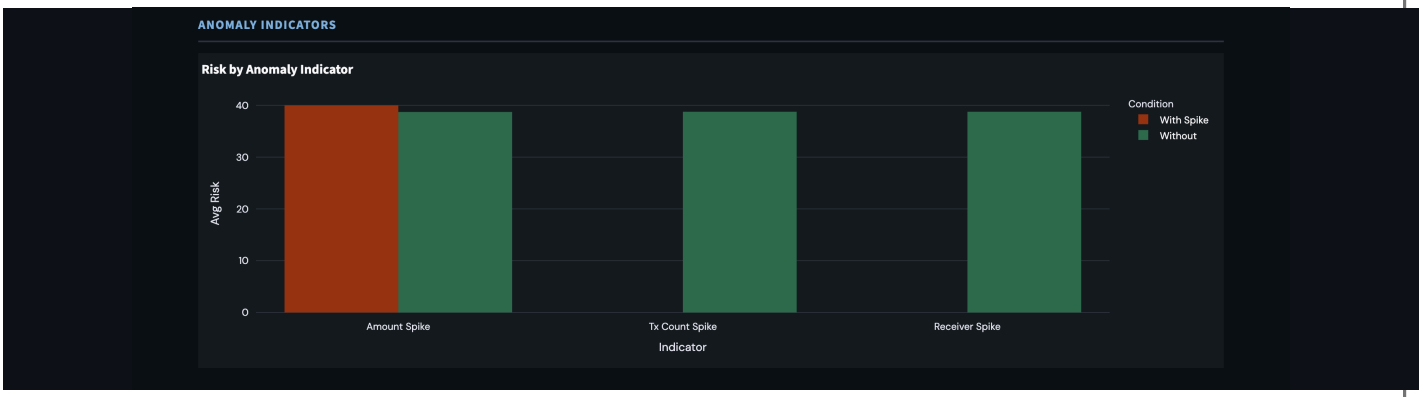



**Figure 4.27: Application Step 4 – Analytics Dashboard: Network and Velocity Features**

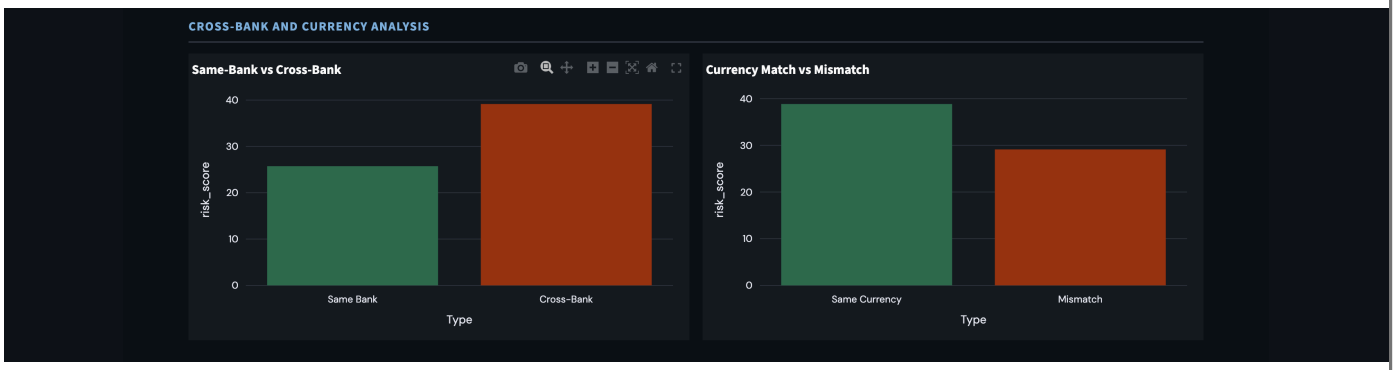



**Figure 4.28: Application Step 4 – Analytics Dashboard: Additional Feature Plots**

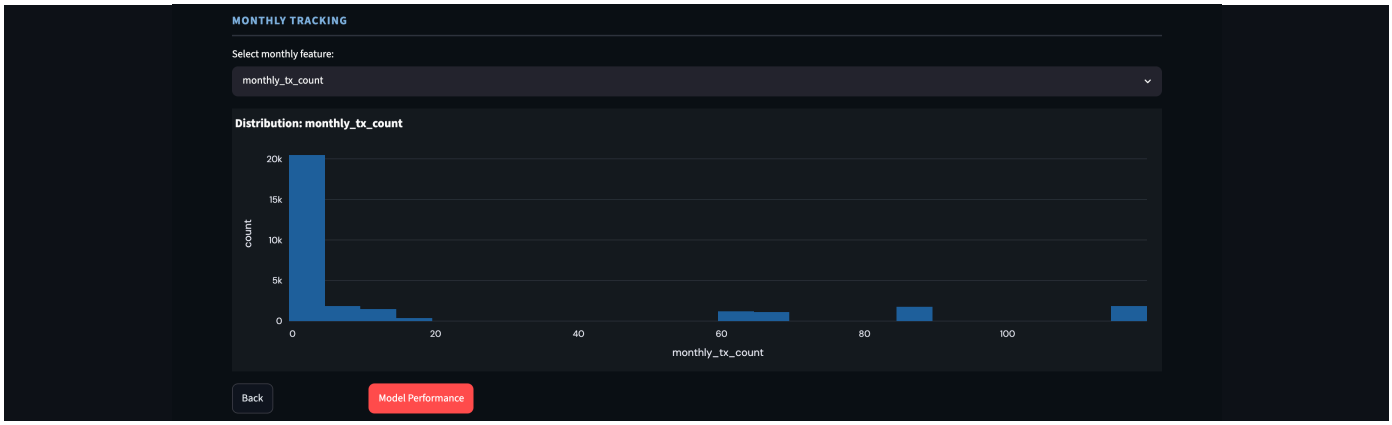



**Figure 4.29: Application Step 4 – Analytics Dashboard: Monthly Tracking**

Step 5 (Model Performance): Per-model metrics table (Precision, Recall, F1, ROC-AUC, PR-AUC), grouped bar charts comparing all three models, feature importance visualisation from Random Forest, and a cross-dataset comparison table showing performance across all 9 models (3 models x 3 datasets).

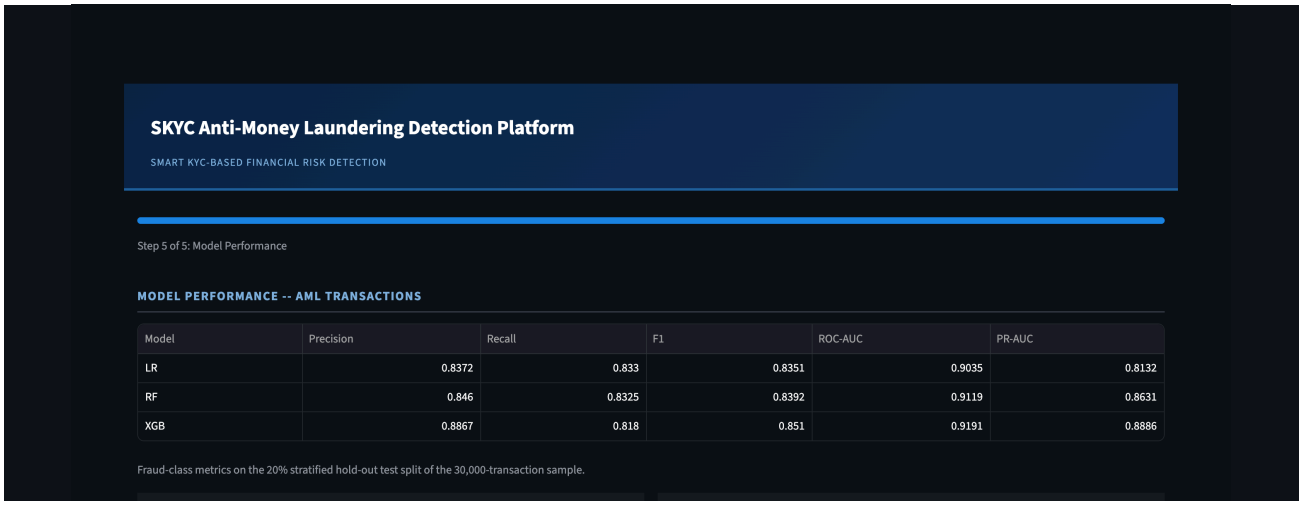



**Figure 4.30: Application Step 5 – Model Performance**

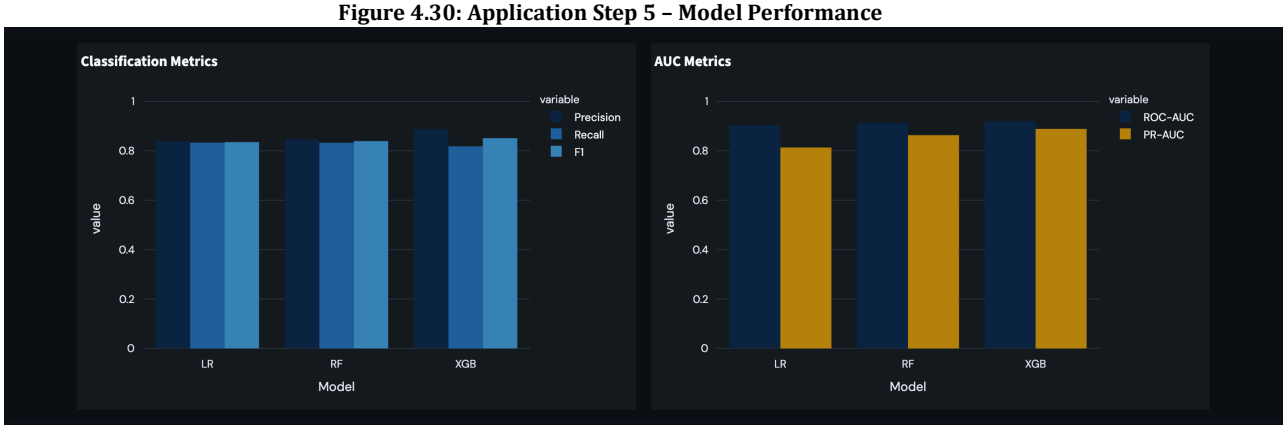



**Figure 4.31: Application Step 5 – Model Performance: Metric Charts**

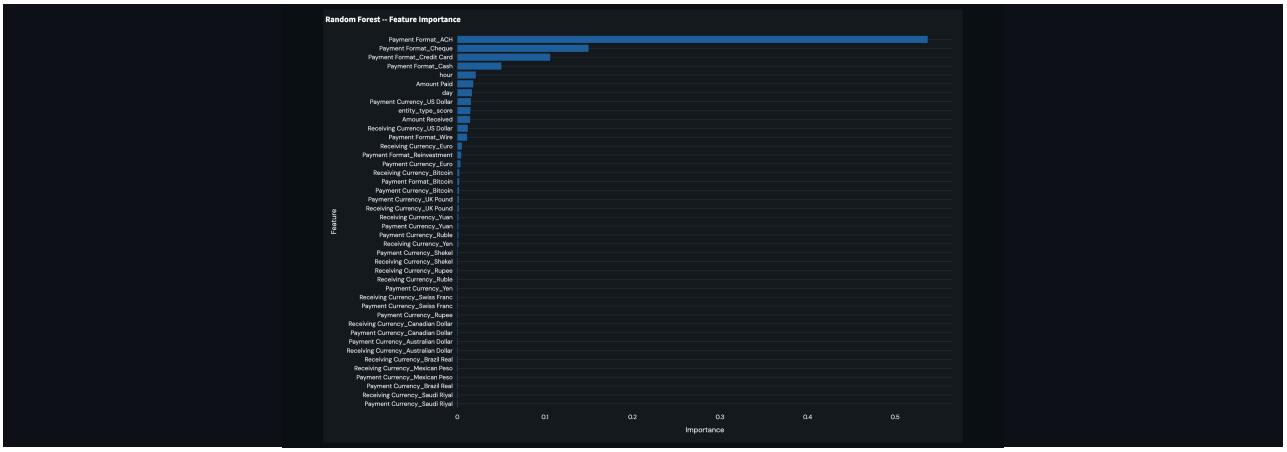



**Figure 4.32: Feature Importance Visualisation**

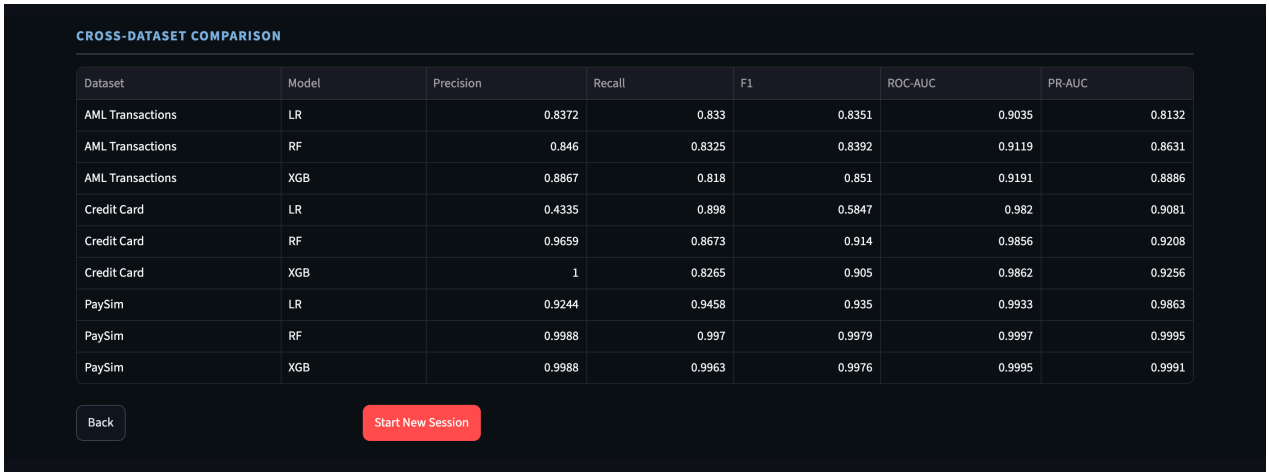



**Figure 4.33: Application Step 5 – Cross-Dataset Comparison**

## 4.7 Challenges Encountered and Solutions

### 4.7.1 Memory and Computational Constraints

The IBM AML dataset posed the most significant computational challenge, with the combined HILarge_Trans.csv and LI-Large_Trans.csv files totalling over 355 million transactions and exceeding 40 GB in raw CSV format. Attempting to load either file directly into a Pandas DataFrame caused kernel crashes due to insufficient RAM, as the data exceeded the available system memory. This was resolved by implementing chunk-based processing: each CSV was read in batches (5 million rows for HI, 1 million rows for LI), converted to Apache Parquet format, and stored as 213 individual Parquet files (36 HI + 177 LI). Parquet's columnar compression reduced storage requirements by approximately 75% and enabled selective column loading during subsequent preprocessing stages, eliminating the need to load entire files into memory.

### 4.7.2 Feature Engineering Iteration

Iterating on the 213 Parquet files for feature engineering required scanning all files for each account, which was computationally expensive. To accelerate development, a caching mechanism was implemented: after filtering transactions for the 237,294 selected accounts (187,294 fraud + 50,000 normal), the resulting 81,125,801-row subset was saved as a single consolidated Parquet file. Subsequent feature engineering operations read from this cached file rather than re-scanning all 213 source files, reducing processing time from several hours to minutes.

### 4.7.3 Variable Scoping in Jupyter Notebooks

During development in Jupyter notebooks, a variable scoping issue arose in the conditional cache logic: the merged DataFrame was assigned inside an if-else block (cache hit vs cache miss), but subsequent cells referenced the variable outside that scope, causing NameError exceptions when the execution order was non-linear. This was resolved by restructuring the notebook to ensure that the DataFrame was always assigned in the global scope regardless of whether the cache was used, and by adding explicit validation checks confirming the DataFrame existed before proceeding to feature engineering.

### 4.7.4 Class Imbalance at Multiple Scales

The original IBM AML dataset had a fraud rate of only 0.092% (326,150 fraud out of 355,768,786 total), which is too extreme for direct model training. This was addressed through a multi-stage sampling strategy: first retaining all fraud accounts, then selecting 50,000 normal accounts, then drawing a 30,000transaction sample with a 1:2 fraud-to-normal ratio (33.33% fraud). Even after sampling, SMOTE was applied to the training partition to achieve a 1:1 ratio. For Credit Card (0.172% original fraud rate), the sample retained all 492 fraud cases. For PaySim (0.13% original fraud rate), all 8,213 fraud cases were retained. Each dataset required a different balancing strategy calibrated to its post-sampling fraud rate.

### 4.7.5 Unsupervised Model Exploration and Abandonment

During the initial design phase, unsupervised anomaly detection models were explored as complementary components. Isolation Forest, One-Class SVM, and Autoencoder architectures were implemented and evaluated. However, all three approaches were ultimately abandoned for the following reasons. Isolation Forest produced a high false positive rate on the IBM AML dataset, flagging over 15% of normal transactions as anomalous, which would be impractical for compliance teams to review. One-Class SVM was computationally prohibitive even on the 30,000-transaction sample, requiring over 30 minutes per training run and making iterative experimentation infeasible. The Autoencoder required extensive hyperparameter tuning (number of layers, compression ratio, reconstruction threshold) without yielding improvements over supervised baselines. More fundamentally, since all three datasets provide groundtruth fraud labels, supervised learning is the natural and more effective approach; unsupervised methods are most valuable when labels are scarce or unavailable, which was not the case in this project. The unsupervised exploration is documented as a planned extension for future deployment scenarios where labelled data may not be available.

# CHAPTER 5: RESULTS AND DISCUSSION

## 5.1 Comparison with Related Work

To contextualise the performance of the SKYC system, a comparison was conducted with published results from studies using the same three datasets. This section examines how the SKYC approach differs from and compares to existing work on Kaggle and in the academic literature.

### 5.1.1 Credit Card Dataset

The Kaggle Credit Card Fraud Detection dataset (284,807 transactions, 492 fraud) is one of the most widely studied benchmarks in the fraud detection literature. A comprehensive comparative study by **IIETA (2025)** evaluated Random Forest and XGBoost with SMOTE on this dataset, reporting RF achieving F1=0.872 and AUC=0.978, while XGBoost reached F1=0.837 and AUC=0.983**. A Frontiers in AI study (2025)** reported RF achieving F1=0.8256 and AUC=0.9759. An advanced hybrid approach combining Autoencoder with XGBoost achieved F1=0.9407 (ResearchGate, 2025). A supervised vs unsupervised comparison study using 5-fold cross validation reported XGBoost AUC=0.989 and RF AUC=0.988 on the full dataset.

The SKYC system achieved XGBoost Precision=1.00, F1=0.91 and AUC=0.9862, with Random Forest achieving F1=0.91 and AUC=0.9856 on the fraud class; XGBoost made no false positives on the test set. These results are competitive with the published baselines, but the comparison is indicative only: our test split comes from a 30,000-transaction sample with a 1.64% fraud rate (about ten times the original 0.172%), which raises precision. SMOTE was not used; Logistic Regression and Random Forest used class_weight='balanced' and XGBoost used no class weighting.

### 5.1.2 PaySim Dataset

The PaySim dataset (6,362,620 transactions, 8,213 fraud) has been studied in several fraud detection frameworks. **A rule-based machine learning model study** reported MCC=0.993, AUC=0.991, and F1=0.998 on PaySim, representing one of the strongest published baselines. **A Fraud Detection System study (2026)** using CRISP-DM methodology with GridSearchCV-tuned models reported strong but slightly lower results. Most published studies on PaySim achieve F1 scores between 0.95 and 0.99 depending on feature engineering and sampling strategy.

The SKYC system achieved near-perfect performance on PaySim, with Random Forest and XGBoost both reaching Precision=0.999, F1=0.998 and AUC of 0.9995-0.9997 under the FE+No SMOTE

configuration (working-sample fraud rate 27.38%), which matches the strongest published baselines. The SKYC advantage lies in the engineered features (balance_diff_orig, balance_diff_dest, amount_vs_balance, is_night), which capture balance manipulation patterns that raw features alone do not expose. Without these features, tree-model F1 drops to 0.99, confirming their contribution.

### 5.1.3 IBM AML Dataset

The IBM Transactions for Anti-Money Laundering dataset is substantially less studied than the Credit Card and PaySim datasets, with limited published baselines. The original IBM paper (**Altman et al., 2023**) evaluated Graph Neural Network architectures (GCN, GraphSAGE, PNA, GIN) on this dataset, demonstrating that GNNs exploiting account connectivity can detect laundering transactions**. A GitHub project by steveee27** applied Random Forest and XGBoost to the HI-Large dataset for training and LILarge for testing, but did not report standardised metrics on a held-out test set with the same data distribution. The Kaggle competition page lists fewer than 50 public notebooks, most of which use the small or medium datasets rather than the Large variants used in SKYC.

The SKYC system achieved XGBoost AUC=0.9237 (K-Fold mean) and F1=0.85 (test split) on the fraud class using tabular features only. These figures are not directly comparable with Altman et al. (2023), who report minority-class F1 on the original, highly imbalanced data, whereas SKYC is evaluated on a sample with 33.33% laundering; at the original rate precision would be far lower. Within these limits, the results show that useful AML signal can be obtained from tabular features with standard ML models, without the overhead of graph-based methods, and SKYC adds a complete application layer (risk scoring, explainability, dashboard) that published baselines do not offer.

### 5.1.4 Key Differentiators of the SKYC Approach

Several aspects distinguish the SKYC system from related work. First, the multi-dataset design evaluates the same three models across three datasets with a unified methodology, enabling direct comparison of model performance across different data modalities (tabular AML, PCA-transformed credit card, synthetic mobile money). Most published studies evaluate on a single dataset. Second, the systematic four-configuration experiment (FE/NO_FE x SMOTE/NO_SMOTE) provides empirical evidence for preprocessing decisions rather than applying FE and SMOTE as defaults. Third, the dual validation strategy (single split + 5-fold) quantifies both performance and stability, revealing overfitting levels that single-split studies cannot detect. Fourth, the working application with unified risk scoring and explainability goes beyond model evaluation to deliver a usable prototype.

## 5.2 IBM AML Dataset Results

### 5.2.1 Train/Test Split Performance

The three supervised models were evaluated on the IBM AML dataset using an 80/20 stratified train/test split. The training partition contained 24,000 transactions, expanded to 32,000 after SMOTE balancing (16,000 per class). The test partition contained 6,000 transactions, including 2,000 fraud cases (33.33%). XGBoost achieved the highest ROC-AUC (0.9191) and F1-Score (0.8510) on the fraud class, with the strongest Precision (0.89) indicating fewer false positives. Random Forest followed closely with AUC=0.9119 and F1=0.8392, achieving the highest Recall (0.83) alongside Logistic Regression. Logistic Regression, despite being a linear model, achieved competitive performance (AUC=0.9035, F1=0.84), demonstrating that the 42-feature input set captures meaningful linear separability between normal and fraudulent transactions.

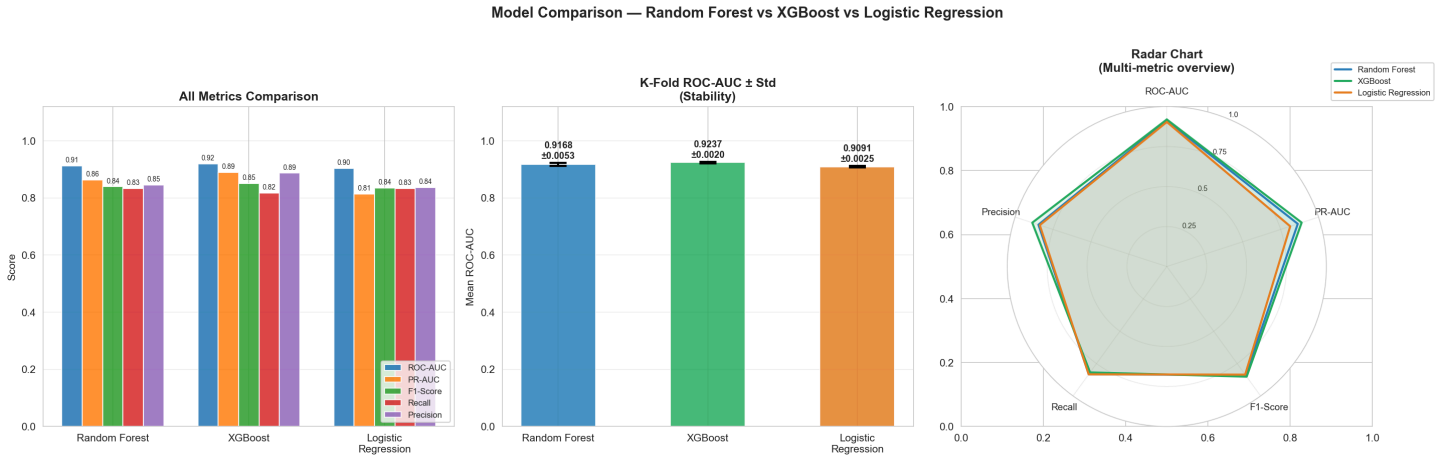



Figure 5.1: IBM AML – Model Comparison: Test Metrics, K-Fold ROC-AUC and Radar Chart

### 5.2.2 Overfitting Analysis (Train/Test Split)

Logistic Regression exhibited the smallest generalisation gap (0.86%), indicating excellent generalisation with minimal memorisation, which is expected from a linear model with L2 regularisation. Random Forest showed a 4.70% gap, within the widely accepted threshold of 5% (Hastie et al., 2009). XGBoost exhibited moderate overfitting (5.65%), attributable to its sequential boosting mechanism which corrects residual errors and can memorise training noise when trees are sufficiently deep. However, the test performance remains strong (0.9191 AUC), indicating that the overfitting does not compromise practical utility. The overfitting hierarchy (LR < RF < XGB) is consistent with model complexity: linear models have the least

capacity to memorise, bagging ensembles moderate through averaging, and boosting ensembles accumulate corrections that can overfit.

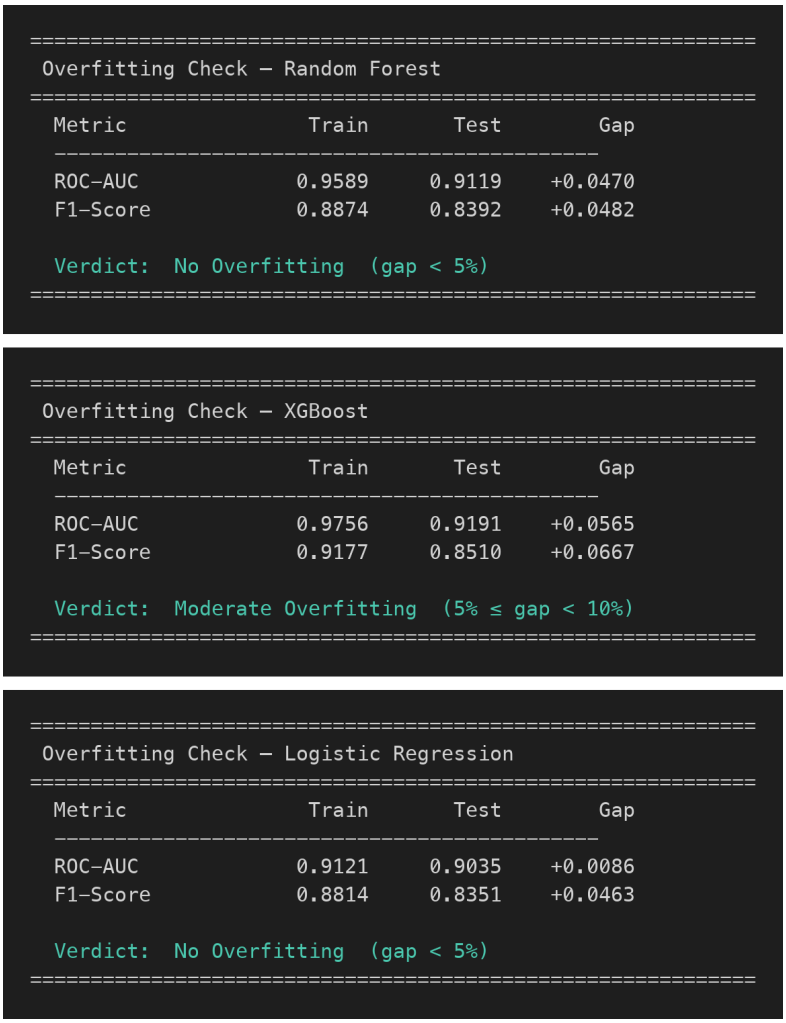



**Figure 5.2: IBM AML – Overfitting Analysis: Train vs. Test AUC Gap per Model**

### 5.2.3 K-Fold Cross Validation Results

XGBoost achieved the highest mean K-Fold AUC (0.9237) with the lowest standard deviation (0.0020), indicating both superior performance and good stability across data splits; its performance varies by only about 0.2% across partitions, confirming that the model has learned genuine patterns rather than split-specific artefacts. Random Forest showed higher variance (0.0053 std) with a mean AUC of 0.9168. Logistic Regression was also very stable (0.0025 std) with a slightly lower mean AUC (0.9091), reflecting the classic bias-variance trade-off: lower complexity yields lower variance but a slightly higher irreducible bias.

```
================================================================================================================
 FINAL MODEL COMPARISON — Train/Test Split vs Stratified K-Fold (k=5)
================================================================================================================
              Model  TT ROC-AUC  TT F1  TT Precision  TT Recall  TT PR-AUC  TT Gap  KF ROC-AUC  KF F1  KF Precision  KF Recall  KF Gap
      Random Forest      0.9119 0.8392        0.8460     0.8325     0.8631  0.0470      0.9168 0.8497        0.8569     0.8427  0.0422
            XGBoost      0.9191 0.8510        0.8867     0.8180     0.8886  0.0565      0.9237 0.8616        0.8980     0.8281  0.0507
Logistic Regression      0.9035 0.8351        0.8372     0.8330     0.8132  0.0086      0.9091 0.8453        0.8471     0.8435  0.0007
================================================================================================================
```

Figure 5.3: IBM AML – K-Fold Cross-Validation vs. **Train/Test Split**

### 5.2.4 K-Fold Overfitting Analysis

The K-Fold overfitting analysis confirmed the single-split findings with remarkable consistency. Logistic Regression demonstrated virtually zero overfitting (0.07% gap between train and validation AUC), Random Forest remained within acceptable bounds (4.22% gap), and XGBoost showed mild overfitting (5.07% gap) that is consistent across folds rather than being an artefact of a single split. The consistency of overfitting levels between single-split and K-Fold analyses provides strong evidence that the observed performance is genuine and not an artefact of a particular data partition. The fact that XGBoost's overfitting level remained stable at approximately 5% across both validation methods indicates that this is a structural property of the model-data interaction rather than a random occurrence.

```
============================================================
🔍 K-Fold Overfitting — Random Forest
============================================================
  Metric                    Train          Val          Gap
  ------------------------------------------------------------
  ROC-AUC                  0.9591       0.9167     +0.0425
  F1-Score                 0.8854       0.8498     +0.0356

  Verdict: ✅ No Overfitting  (gap < 5%)
============================================================
```

```
============================================================
🔍 K-Fold Overfitting — XGBoost
============================================================
  Metric                    Train          Val          Gap
  ------------------------------------------------------------
  ROC-AUC                  0.9744       0.9243     +0.0502
  F1-Score                 0.9151       0.8607     +0.0544

  Verdict: 🟡 Moderate Overfitting  (5% ≤ gap < 10%)
============================================================
```

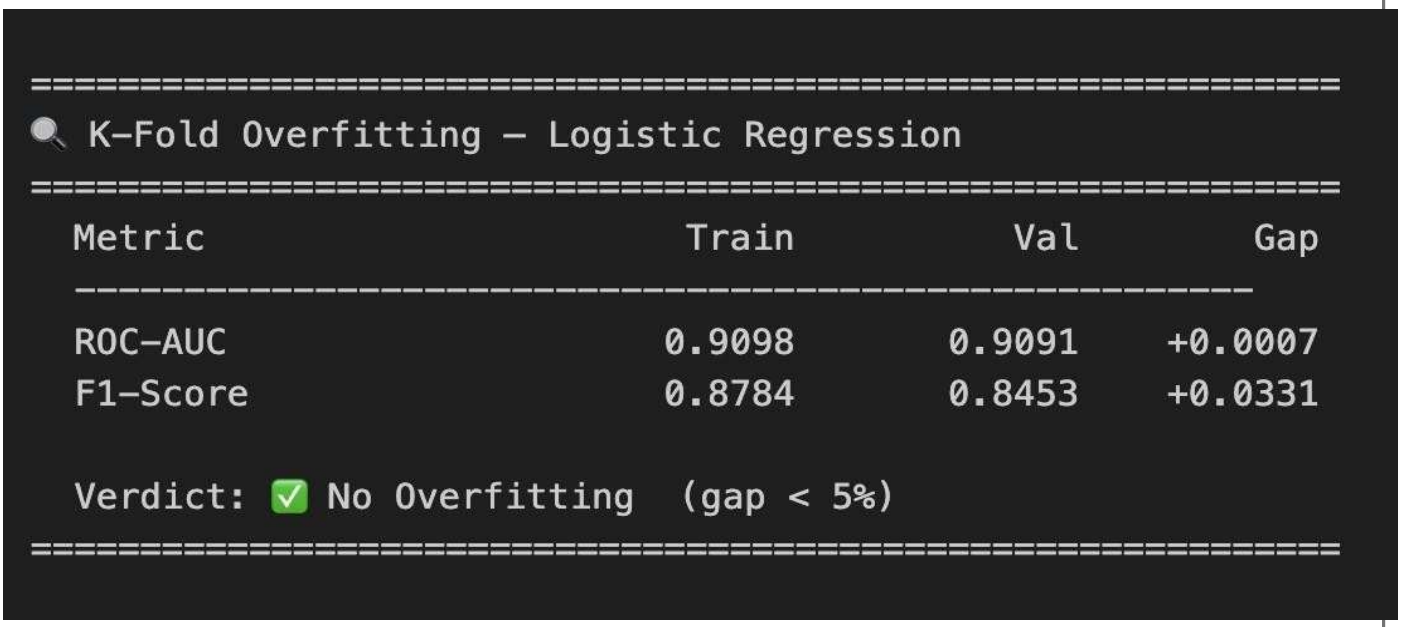



Figure 5.4: IBM AML – K-Fold Overfitting Analysis: Train vs. Validation AUC

## 5.3 Credit Card and PaySim Experimental Comparison

### 5.3.1 Four-Configuration Experiment Design

Four configurations were evaluated to assess the impact of Feature Engineering (FE) and SMOTE: FE+SMOTE, FE+NO_SMOTE, NO_FE+SMOTE and NO_FE+NO_SMOTE. For Credit Card, FE and NO_FE are functionally identical because no feature engineering was applied to the PCA-transformed features, reducing the experiment to a SMOTE comparison. For PaySim, all four configurations differ because the engineered features (balance_diff_orig, balance_diff_dest, amount_vs_balance, is_night) are new. Class weighting was not held constant, however: FE+NO_SMOTE uses class_weight='balanced' for LR and RF while the NO_FE runs use none, so the FE effect is partly confounded with weighting.

### 5.3.2 Credit Card Results

The No SMOTE configuration gave higher F1 and ROC-AUC than SMOTE for all three models. XGBoost achieved Precision=1.00, Recall=0.83, F1=0.91 and AUC=0.9862 without SMOTE, the best overall performance, and Random Forest achieved F1=0.91 and AUC=0.9856. Adding SMOTE raised XGBoost Recall to 0.89 but reduced its Precision to 0.87. Logistic Regression had low Precision in both configurations (0.43 without and 0.42 with SMOTE, Recall 0.90–0.91) because class_weight='balanced' already shifts its decision boundary toward the

fraud class; without any reweighting (NO_FE + NO_SMOTE run), LR reaches Precision 0.99 at Recall 0.83. Any rebalancing therefore trades precision for recall at the default 0.5 threshold, and tree-based models (RF, XGB) are less affected because they partition the feature space locally rather than learning a global boundary.

```
======================================================================
 Credit Card Fraud – Model Comparison
======================================================================
              Model  Precision  Recall  F1-Score  ROC-AUC  PR-AUC
Logistic Regression     0.4335  0.8980    0.5847   0.9820  0.9081
      Random Forest     0.9659  0.8673    0.9140   0.9856  0.9208
            XGBoost     1.0000  0.8265    0.9050   0.9862  0.9256
======================================================================
```

Figure 5.5: Credit Card Dataset –Model Comparison

### 5.3.3 PaySim Results

Feature engineering coincided with a clear improvement for Logistic Regression, whose F1 rose from 0.88 to 0.93 (part of this gain may come from the class weighting used in the FE + No SMOTE run). Random Forest and XGBoost reached near-perfect scores under FE + No SMOTE (F1 = 0.998, Precision = 0.999, Recall 0.996–0.997), with AUC = 0.9997 and 0.9995 respectively. Without FE, these models still achieved F1 = 0.99 and AUC = 0.999, confirming that the engineered features provide incremental improvement on already-strong baselines. SMOTE had negligible impact across all models, which is expected given the working sample's fraud rate of 27.38%. The FE + No SMOTE configuration was adopted for the application as it achieves the highest absolute performance with the simplest pipeline.

```
================================================================
 PaySim – Model Comparison with fe
================================================================
              Model  Precision  Recall  F1-Score  ROC-AUC  PR-AUC
Logistic Regression     0.9244  0.9458    0.9350   0.9933  0.9863
      Random Forest     0.9988  0.9970    0.9979   0.9997  0.9995
            XGBoost     0.9988  0.9963    0.9976   0.9995  0.9991
================================================================
```

**Figure 5.6: PaySim Dataset –Model Comparison**

### 5.3.4 Cross-Dataset Comparison

**IBM AML** presented the most challenging classification problem, with the lowest scores across all metrics (best ROC-AUC: 0.9237 from XGBoost K-Fold). The dataset is synthetic but generated to mimic realistic laundering typologies, in which fraudulent transactions resemble legitimate activity through layering and structuring. **Credit Card** (real, anonymised transactions) achieved strong performance (best ROC-AUC: 0.9862) due to the discriminative power of its PCA-transformed features. **PaySim** achieved near-perfect scores (best ROC-AUC: 0.9997) because its simulator generates fraud by simple rules that tree-based models can partition almost perfectly. The gradient (PaySim > Credit Card > IBM AML) thus follows the difficulty of the fraud patterns; since the samples have different fraud rates (27.38%, 1.64%, 33.33%), precision and F1 should be compared across datasets with caution.

```
================================================================================================
 CROSS-DATASET COMPARISON – All 3 Datasets × 3 Models
================================================================================================
    Dataset              Model  Precision  Recall  F1-Score  ROC-AUC  PR-AUC
    IBM AML Logistic Regression     0.8372  0.8330    0.8351   0.9035  0.8132
Credit Card Logistic Regression     0.4335  0.8980    0.5847   0.9820  0.9081
     PaySim Logistic Regression     0.9244  0.9458    0.9350   0.9933  0.9863
    IBM AML       Random Forest     0.8460  0.8325    0.8392   0.9119  0.8631
Credit Card       Random Forest     0.9659  0.8673    0.9140   0.9856  0.9208
     PaySim       Random Forest     0.9988  0.9970    0.9979   0.9997  0.9995
    IBM AML             XGBoost     0.8867  0.8180    0.8510   0.9191  0.8886
Credit Card             XGBoost     1.0000  0.8265    0.9050   0.9862  0.9256
     PaySim             XGBoost     0.9988  0.9963    0.9976   0.9995  0.9991
================================================================================================
```

**Figure 5.7: Cross-Dataset Model Performance Comparison (IBM AML, Credit Card, PaySim)**

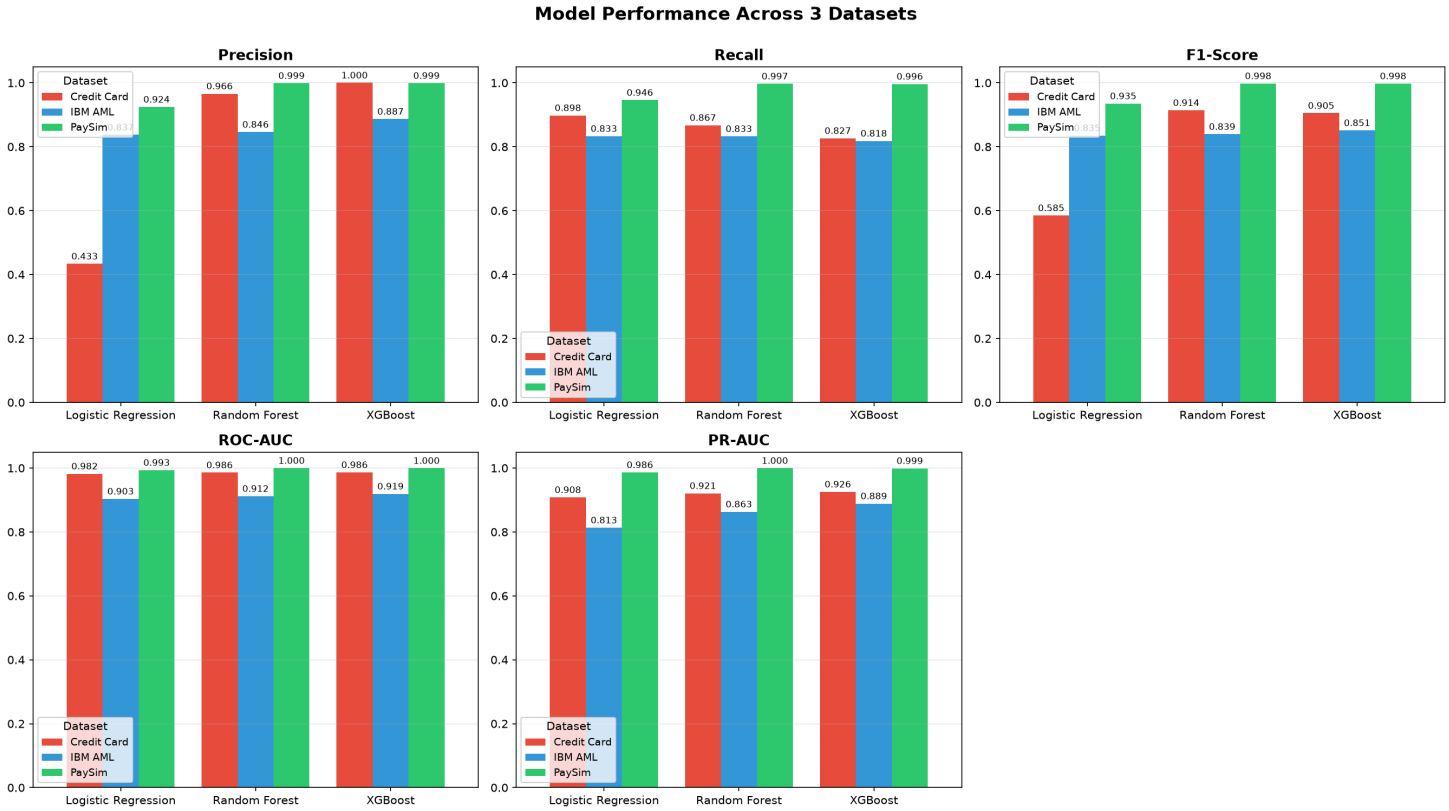



**Figure 5.8: Model Performance Across 3 Datasets**

```
 Average Performance per Dataset:
============================================================
             Precision  Recall  F1-Score  ROC-AUC  PR-AUC
Dataset
Credit Card     0.7998  0.8639    0.8012   0.9846  0.9182
IBM AML         0.8566  0.8278    0.8418   0.9115  0.8550
PaySim          0.9740  0.9797    0.9768   0.9975  0.9950
============================================================

 Average Performance per Model:
============================================================
                     Precision  Recall  F1-Score  ROC-AUC  PR-AUC
Model
Logistic Regression     0.7317  0.8923    0.7849   0.9596  0.9025
Random Forest           0.9369  0.8989    0.9170   0.9657  0.9278
XGBoost                 0.9618  0.8803    0.9179   0.9683  0.9378
============================================================
```

Figure 5.9: Average Performance per Dataset and per Model

## 5.4 Feature Importance Analysis

Random Forest feature importance for the **IBM AML dataset** is dominated by the One-Hot encoded Payment Format columns, which together account for about 86% of the total importance: Payment Format_ACH alone contributes 54%, followed by Cheque (15%), Credit Card (11%) and Cash (5%). In XGBoost, ACH accounts for 71%. This reflects a strong property of the IBM AML data: laundering transactions are concentrated in particular payment formats, especially ACH. The currency columns contribute about 5% in total, while hour (2.1%), Amount Paid (1.8%), day (1.7%) and entity_type_score (1.5%) contribute little. The heavy reliance on one feature means the models largely learn payment-format patterns, which should be considered when interpreting the results.

For **Credit Card,** the PCA features V10, V14, V4, V12 and V17 were the most important for Random Forest, and V14 alone accounted for 47% of XGBoost importance, consistent with published literature identifying these components as the most discriminative for fraud separation. For

**PaySim**, balance_diff_orig and amount_vs_balance dominated importance (56% together in Random Forest; balance_diff_orig 59% in XGBoost), confirming balance manipulation as the main fraud signal.

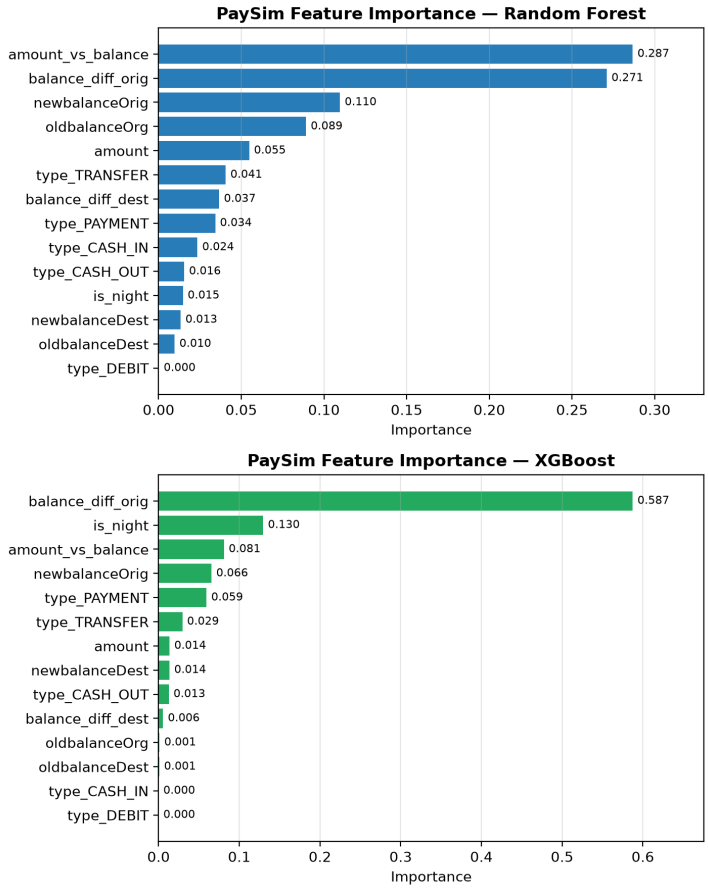



Figure 5.10: PaySim – Feature Importance Rankings

## 5.5 Limitations

Several limitations should be acknowledged. First, the 30,000-transaction sample represents a small fraction of the full 81-million-record IBM AML filtered dataset. While K-Fold validation confirms stability within this sample, the sample may not capture all transaction patterns present in the full dataset, particularly rare laundering typologies that occur in fewer than 1 in 10,000 transactions. Second, the IBM AML fraud rate of 33.33% in the working sample is substantially higher than typical real-world AML rates (below 0.1%), due to the deliberate oversampling of fraudulent transactions to facilitate model training. Production deployment would require recalibration of model thresholds to account for the true prior probability of fraud. Third, engineered features were excluded from model inputs to prevent temporal and label leakage, but this limits model expressiveness compared to production systems that can implement proper temporal splitting with features computed only from strictly past data. Fourth, the

PaySim dataset is synthetic, generated by a simulation engine that embeds specific fraud rules; the nearperfect performance scores reflect the model recovering these rules rather than learning genuine fraud behaviour, which may not generalise to real mobile money transactions. Fifth, model serialisation with scikit-learn 1.9.0 creates a version dependency that must be maintained in the deployment environment to avoid deserialisation errors.

# CHAPTER 6: CONCLUSION AND FUTURE WORK

## 6.1 Conclusion

This project presented SKYC, a Smart KYC-Based Anti-Money Laundering Detection Platform that integrates machine learning with behavioural analytics for comprehensive financial crime detection. The system was designed, implemented, and evaluated across three financial datasets: IBM AML (355 million transactions, filtered to 30,000), Credit Card Fraud (284,807 transactions, sampled to 30,000), and PaySim (6.3 million transactions, sampled to 30,000), employing nine supervised machine learning models and a unified risk scoring engine.

The key contributions of this work include: first, a comprehensive feature engineering pipeline producing over 60 behavioural features spanning monthly, daily, hourly, and yearly temporal granularities, as well as network, velocity, and anomaly indicators, powering the analytics dashboard without introducing leakage into model training; second, a systematic experimental methodology comparing four preprocessing configurations (FE/NO_FE x SMOTE/NO_SMOTE) across two datasets, showing that SMOTE adds little recall (0–6 points) while lowering precision, that class weighting sharply lowers Logistic Regression precision at the default threshold, and that feature engineering coincides with clear gains on raw transactional features (PaySim: +5 points LR F1) but not on pre-transformed data (Credit Card); third, a dual validation strategy combining single train/test splits with Stratified 5-Fold Cross Validation, with K-Fold analysis showing that Logistic Regression exhibited virtually zero overfitting (0.07% gap), Random Forest maintained acceptable generalisation (4.22% gap), and XGBoost showed moderate but manageable overfitting (5.07% gap); and fourth, a working web application with dataset-specific analytics, rule-based explanations and compliance-oriented dashboards, supporting three datasets with independent model pipelines, a unified risk scoring engine (0-100 scale), and human-readable justifications for every flagged transaction.

The best-performing model on the primary IBM AML dataset was XGBoost, achieving 0.9237 mean

K-Fold ROC-AUC with a standard deviation of 0.0020, demonstrating both strong discrimination ability and good stability across data splits. On Credit Card, XGBoost achieved AUC=0.9862 with perfect Precision (1.00) on the fraud class. On PaySim, Random Forest achieved AUC=0.9997 with F1=0.998, demonstrating that the feature engineering pipeline captures the essential fraud signals in mobile money transactions.

## 6.2 Future Work

Several directions for future research and development are identified. Temporal Split Validation would implement time-based train/test splitting instead of random splitting, better simulating real-world deployment where models are trained on historical data and evaluated on future transactions, and would also enable the safe inclusion of temporal engineered features as model inputs since the temporal ordering would prevent leakage. Graph Neural Networks could model the transaction network as a graph, with accounts as nodes and transactions as edges, capturing multi-hop relationships such as chains of transactions across multiple intermediary accounts that tabular models cannot express; the IBM paper (Altman et al., 2023) has already demonstrated the potential of GNNs on this dataset.

Online Learning capabilities would allow models to update incrementally with new transactions without full retraining, adapting to evolving fraud patterns in real time through techniques such as incremental gradient boosting. Full Dataset Training using distributed computing frameworks (Dask, Spark, or Ray) would enable training on the complete 80-million-record filtered dataset, potentially capturing rare fraud patterns not present in the 30,000-transaction sample; preliminary estimates suggest that Random Forest training on 80 million records would require approximately 128 GB RAM with Dask, which is feasible on cloud infrastructure.

Advanced Explainability through SHAP (SHapley Additive exPlanations) values would provide perfeature contribution analysis for each individual prediction, offering more granular and mathematically grounded explanations than the current rule-based system. Real-Time Scoring via a REST API (FastAPI or Flask) would enable sub-second transaction scoring for integration with banking core systems, supporting the regulatory requirement for real-time monitoring. Federated Learning could enable privacypreserving model training across multiple institutions without sharing raw transaction data, addressing the regulatory constraints of cross-border AML cooperation where data sharing between jurisdictions is restricted. Finally, Deep Learning Models such as LSTM and Transformer architectures

could capture long-range temporal dependencies in transaction histories that tree-based models cannot, particularly for detecting slow-moving layering schemes that unfold over weeks or months.

## REFERENCES


[1] Altman, E., et al. (2023). Realistic Synthetic Financial Transactions for Anti-Money Laundering Models. Available at: https://doi.org/10.52202/075280-1300

[2] Bahnsen, A. C., Aouada, D., Stojanovic, A., and Ottersten, B. (2016). Feature engineering strategies for credit card fraud detection. Available at: https://doi.org/10.1016/j.eswa.2015.12.030

[3] Breiman, L. (2001). Random forests. Machine Learning. Available at: https://doi.org/10.1023/a:1010933404324

[4] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Available at: https://doi.org/10.1613/jair.953

[5] Chen, T., and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Available at: https://doi.org/10.1145/2939672.2939785

[6] Dal Pozzolo, A., Caelen, O., Johnson, R. A., and Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. Available at: https://doi.org/10.1109/ssci.2015.33

[7] Dreiseitl, S., and Ohno-Machado, L. (2002). Logistic regression and artificial neural network classification models. Available at: https://doi.org/10.1016/s1532-0464(03)00034-0

[8] FATF. (2023). International Standards on Combating Money Laundering and the Financing of Terrorism and Proliferation. Available at: https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Fatf-recommendations.html

[9] Hastie, T., Tibshirani, R., and Friedman, J. (2009). The Elements of Statistical Learning. Available at: https://doi.org/10.1007/978-0-387-84858-7

[10] Hilal, W., Gadsden, S. A., and Yawney, J. (2022). Financial fraud: A review of anomaly detection techniques and recent advances. Available at: https://doi.org/10.1016/j.eswa.2021.116429

[11] IIETA. (2025). Comparative Analysis of Machine Learning Models for Credit Card Fraud Detection Using SMOTE... Available at: https://doi.org/10.18280/ijsse.150504

[12] Jullum, M., Loland, A., Huseby, R. B., Blix, P., and Gramme, N. (2020). Detecting money laundering transactions with machine learning. Available at: https://doi.org/10.1108/jmlc-07-2019-0055

[13] Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. Available at: https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf

[14] Liu, F. T., Ting, K. M., and Zhou, Z. H. (2008). Isolation forest. Available at: https://doi.org/10.1109/icdm.2008.17

[15] Lopez-Rojas, E. A., Elmir, A., and Axelsson, S. (2016). PaySim: A financial mobile money simulator for fraud detection. Available at: https://doi.org/10.1504/ijspm.2018.093756

[16] Lundberg, S. M., and Lee, S. I. (2017). A unified approach to interpreting model predictions. Available at: https://dl.acm.org/doi/10.5555/3295222.3295230

[17] Ngai, E. W., Hu, Y., Wong, Y. H., Chen, Y., and Sun, X. (2011). The application of data mining techniques in financial fraud detection. Available at: https://doi.org/10.1016/j.dss.2010.08.006

[18] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Available at: https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html

[19] Sterne, J. A. C., White, I. R., et al. (2009). Multiple imputation for missing data in epidemiological and clinical research: potential and pitfalls. BMJ, 338, b2393. Available at: https://doi.org/10.1136/bmj.b2393

[20] Weber, M., et al. (2018). Scalable graph learning for anti-money laundering. Available at: https://arxiv.org/abs/1812.00076